Contents

Notes on Cosmological Simulations **1**
Introduction . . . 1
Anatomy of Cosmological Simulations . . . 1
The Major Cosmological Codes . . . 2
The N-Body Problem . . . 2
Direct Summation . . . 2
Boundaries and Singularities . . . 3
Periodic Boundary Conditions . . . 3
Gravitational Softening . . . 3
The Integrator . . . 4
The Euler Method . . . 4
Velocity Verlet . . . 4
Symplectic Integration . . . 4
The Kick-Drift-Kick Integrator . . . 5
The Particle-Mesh Method . . . 6
Step 1. Finding the Potential . . . 6
Step 2. From Potential to Force . . . 9
Advanced Interpolation . . . 10
The Flaws of Nearest Grid Point (NGP) . . . 10
Cloud-in-Cell (CIC) . . . 10
Implementation: "Splatting" Mass and "Gathering" Forces . . . 11
Deconvolving the Mass Assignment . . . 12
The P^{3}M Algorithm . . . 13
Combining PP for Short-Range and PM for Long-Range . . . 13
The Subtractive Scheme . . . 13
Calculating the Mesh-Force Correction . . . 13
Choosing the Cutoff Radius (r_c) . . . 14
The Switching Function . . . 14
Fourier-Split PM . . . 15
Fixing the Grid in Frequency Space . . . 15
The Exact Analytical Complement . . . 16
An Expanding Space . . . 16
The Hubble Flow . . . 16
Comoving Coordinates . . . 17
The Equations of Motion . . . 17
Cosmological Models and the Friedmann Equations . . . 17
An Einstein-de Sitter Universe . . . 19
The ΛCDM Model . . . 19
General Relativity and Cosmic Expansion . . . 20
Natural Units . . . 22
Deriving the Gravitational Constant (G) . . . 23
Hubble: Physical vs. Code Units . . . 24
Translating Natural Units to Physical Reality . . . 24
The Cosmic Timeline . . . 25
The Scale Factor (a) and Redshift (z) . . . 25
Eras and epochs . . . 25
Initial Conditions . . . 27
The Unperturbed State . . . 27

The Zel'dovich Approximation . . . 27
Gravitational softening . . . 32
Determining the Optimal Softening Length . . . 32
The Physical Softening Cap . . . 32
Gravity Validation and Accuracy . . . 33
Conservation of Energy and Momentum . . . 33
The Two-Body Problem and Kepler's Laws . . . 34
Sources of Error . . . 35
Hydrodynamics . . . 35
The Hybrid Simulation Approach . . . 36
The Euler Equations . . . 36
An Operator-Split Hydro-Solver . . . 37
Coupling Hydrodynamics to the Expanding Universe . . . 43
Sub-Grid Coupling . . . 43
Initial Conditions for the Gaseous Component . . . 45
Gas Physics . . . 46
Temperature . . . 46
Gravitational Compression and Shocks . . . 46
Radiative Cooling . . . 46
Stiff Equations . . . 48
Implicit Integration . . . 49
Coupling Cooling to the Simulation . . . 50
The Optically Thin Approximation . . . 51
The Subgrid Clumping Factor . . . 51
Metallicity and the Cosmic Ultraviolet Background . . . 53
Metallicity (Z) . . . 53
Ultraviolet Background (UVB) . . . 54
Numerical methods . . . 54
Meshless Finite Mass (MFM) Hydrodynamics . . . 56
Volume Partitioning and Density . . . 56
Gradient Estimation and Spatial Reconstruction . . . 58
Effective Faces . . . 59
The Riemann Solver and Flux Computation . . . 60
Dual Energy Formalism . . . 60
Adaptive Gravitational Softening . . . 61
Initializing the Meshless Finite Mass (MFM) Gas . . . 62
Validation of the Hydrodynamic Solver . . . 63
Conservation in a Closed Box . . . 63
The Linear Soundwave Test . . . 63
The 1D Shock-Tube . . . 64
The Sedov-Taylor Blast Wave (Point Explosion) . . . 66
The Kelvin-Helmholtz Instability . . . 66
The Evrard Spherical Collapse . . . 66
Sources of Error . . . 67
Adaptive Timestep . . . 67
The Courant-Friedrichs-Lewy (CFL) Condition for hydrodynamics . . . 67
The Cooling Timestep Limiter . . . 68
The Gravitational Timestep . . . 68
The Global Timestep . . . 69
Subcycled Operator Splitting . . . 69
Adaptive Multirate Operator Splitting . . . 69
Decoupling the Thermodynamics . . . 70
Accuracy and Symplecticity in a Subcycled Architecture . . . 71
Spatial Partitioning and the LBVH . . . 71
The Oct-Tree Concept . . . 72
The Barnes-Hut Algorithm . . . 72
The Multipole Expansion . . . 72
The Linear Bounding Volume Hierarchy (LBVH) . . . 74
Morton Codes and the Z-Order Curve . . . 74
The Karras Tree Construction . . . 74

Stack-Based Traversal . . . 75
The TreePM Architecture . . . 76
Individual particle timesteps . . . 76
Bounding the Local Clock . . . 77
The Power-of-Two Hierarchy . . . 77
The Active Timeline and Integration . . . 77
Asymmetric Fluxes and Conservation Trade-offs . . . 77
The Wake-Up Failsafe . . . 78
Analytical Requirements for Resolution and Volume . . . 78
Terminology . . . 78
Expected Discrepancies in the Physics of Small Boxes . . . 79
The Box Size . . . 80
Full-size Representative Simulations . . . 82
Architectures of Cosmological Simulations . . . 83
Large-Volume "Uniform Box" Simulations . . . 84
Zoom-in Simulations . . . 84
Constrained Simulations . . . 84
The Physics Split: Gravity-Only vs. Hydrodynamics . . . 84
Diagnosing Cosmological Simulations . . . 85
Cosmic Expansion History . . . 85
Structure Growth and Linear Theory . . . 85
The 1-Point Density PDF . . . 86
The Matter Power Spectrum . . . 87
Global Gas Energy Inventory . . . 88
Maximum Gas Density Evolution . . . 89
Maximum Temperature Evolution . . . 90
The Temperature-Density Phase Diagram . . . 90
Cold Dense Gas Fraction (The Star Formation Precursor) . . . 91
The Invariant Layzer-Irvine Energy . . . 91
Appendix . . . 93
Cosmological Unit Conversion Cheatsheet . . . 93

Notes on Cosmological Simulations

This is a living document—a collection of knowledge that I have gathered while learning about cosmological simulations. It is not a formal text but rather a journal, an attempt to solidify concepts by structuring and explaining them in my own way.

Along the way, I have been developing a proof-of-concept N-body/hydrodynamics simulation, which allowed me to understand algorithms by implementing them, and to appreciate physical principles by seeing their effects in a virtual universe. The explanations in this document are reflected in this practical work.

This is my best effort to present this knowledge in the way that I would have found most helpful at the start of my learning process.

Víctor Álamo
vialamo@gmail.com
https://github.com/vialamo/nbody

Introduction

Because astrophysicists cannot experiment on stars or galaxies in a laboratory, they use computers to build virtual patches of the cosmos. By seeding a simulation volume with the initial conditions of the Big Bang and stepping it forward in time using the laws of physics, we can watch 13.8 billion years of cosmic evolution unfold in a matter of days.

The Origins

The endeavor to simulate the cosmos began in the 1970s. Early N-body simulations used just a few hundred particles to study how galaxies might cluster together under gravity. By the 1980s, researchers were running the first 3-dimensional models of Cold Dark Matter (CDM). These early simulations treated entire galaxies as single points of mass and modeled only the force of gravity, ignoring the complex fluid dynamics of gas.

Applications

Projects like the Millennium Simulation, Illustris, and EAGLE utilize supercomputing clusters to track billions—and sometimes trillions—of particles. These virtual universes serve three main purposes:

- **Testing the Standard Model:** We can easily alter the parameters of a simulation—adding more dark energy, changing the mass of dark matter particles, or altering the laws of gravity. By comparing the resulting "mock universe" to the real one, we can prove or disprove fundamental theories of physics.
- **Calibrating Telescope Data:** Massive modern observatories (like the James Webb Space Telescope and the Euclid satellite) gather an overwhelming amount of data. Simulations provide the theoretical maps required to interpret those observations, helping astronomers distinguish between optical illusions (like redshift-space distortions) and true physical structures.
- **Probing the Unobservable:** While a telescope can only capture a snapshot of a galaxy at one specific moment in its life, a simulation allows us to watch the processes of galaxy mergers, black hole feeding, and cosmic web formation from beginning to end.

Anatomy of Cosmological Simulations

A cosmological code contains different ingredients to mimic the composition and behavior of the universe.

- **Collisionless Dark Matter :** Dark matter accounts for roughly 85% of the matter in the universe and dominates its gravitational landscape. Because it does not interact with light or experience thermodynamic pressure, we model it as "collisionless" particles. This is handled by an **N-body solver**, which tracks millions of discrete, massive particles moving purely under the influence of gravity.

- **Baryonic Gas:** Normal, visible matter behaves differently from dark matter. Gas clouds crash into each other, heat up, form shockwaves, and exert pressure. To simulate this, we require a **Hydrodynamics solver**. This typically involves tracking the conservation of mass, momentum, and energy as the fluid flows through space, while gravity links the dark matter and the gas together.
- **The Expanding Background:** A cosmological simulation takes place in an expanding universe. Both the dark matter and the hydrodynamic gas must be subjected to a cosmological background model to properly account for the dilution of density and the slowing of velocities due to cosmic expansion.

The Major Cosmological Codes

The astrophysical community relies on a handful of open-source software packages to run these supercomputer simulations.

Codes are split by their approach to fluid dynamics: some treat gas as a collection of individual moving particles (**Lagrangian** methods like SPH), while others treat gas as a fluid flowing through a fixed or adaptive 3D grid (**Eulerian** methods like AMR).

Here is a summary of the most prominent cosmological codes and the underlying engines that drive them:

Code Name	Gravity Solver	Hydrodynamics Solver	Notable Simulations	Website / Repository
GADGET	TreePM (Tree-based + Particle-Mesh)	SPH (Smoothed Particle Hydrodynamics)	Millennium, EAGLE	MPA Garching - Gadget-4
RAMSES	AMR-coupled Particle-Mesh	AMR (Adaptive Mesh Refinement)	Horizon-AGN	ramses.cnrs.fr / GitHub
ENZO	Particle-Mesh	AMR (Adaptive Mesh Refinement)	Renaissance Simulations	enzo-project.org
AREPO	TreePM	Moving Voronoi Mesh (Unstructured grid)	Illustris, IllustrisTNG	arepo-code.org
SWIFT	Fast Multipole Method (Tree)	Modern SPH / Meshless Finite Mass	Flamingo	swiftsim.com / GitHub

The N-Body Problem

The **N-body problem** is the task of predicting the dynamical evolution of a system composed of N particles that interact through mutual gravitational attraction. Each particle experiences the combined gravitational influence of all others, and because these forces depend on the instantaneous positions of every particle, the motion of any one particle cannot be determined independently.

In Newtonian gravity, the equation of motion for particle i with position vector $\mathbf{x}_i$, velocity $\mathbf{v}_i$, and mass m_i is:

$$m_i \frac{d^2\mathbf{x}_i}{dt^2} = -Gm_i \sum_{\substack{j=1 \\ j\neq i}}^{N} m_j \frac{\mathbf{x}_i - \mathbf{x}_j}{|\mathbf{x}_i - \mathbf{x}_j|^3}$$

This coupled system of $3N$ second-order differential equations (N per axis) has no general analytic solution for $N > 2$, requiring numerical methods instead.

Direct Summation

The **direct-summation algorithm** is a numerical method for solving the N-body problem, which explicitly computes the gravitational force on each particle from every other particle. The procedure for a single time step can be described as follows:

1. **Select a particle**, say particle A.
2. **Loop over all other particles** $(B, C, D, \ldots)$.

3. **Compute the pairwise force** on A from each other particle using Newton's law of gravitation:

$$\mathbf{F}_{AB} = -G\frac{m_A m_B}{r_{AB}^2}\hat{\mathbf{r}}_{AB}$$

   where $\mathbf{r}_{AB} = \mathbf{x}_A - \mathbf{x}_B$ and $\hat{\mathbf{r}}_{AB} = \mathbf{r}_{AB}/|\mathbf{r}_{AB}|$.
4. **Sum all pairwise forces** to obtain the total force on particle A:

$$\mathbf{F}_A = \sum_{\substack{B=1 \\ B\neq A}}^{N} \mathbf{F}_{AB}$$

5. **Update** particle A's position and velocity using this total force.
6. **Repeat** the process for every particle in the system.

Although conceptually simple and exact, the direct-summation method is computationally prohibitive for large N. To compute the total force on one particle, we must evaluate $N-1$ pairwise interactions. Doing this for all N particles requires approximately $N(N-1) \approx N^2$ force evaluations per time step.

In computational complexity terms, this corresponds to $O(N^2)$ scaling — meaning that doubling the number of particles multiplies the total computational cost by roughly four. This quadratic growth rapidly becomes intractable.

Because of this steep scaling, the direct method is impractical for cosmological simulations. To overcome this, we rely on **approximation schemes** —such as the **Particle-Mesh (PM)**, explained later— that reduce computational cost.

Boundaries and Singularities

Periodic Boundary Conditions

To simulate a small, representative patch of an infinite, uniform universe, simulations employ **periodic boundary conditions**. This method treats the simulation space as a seamless, repeating tile.

A particle exiting one face immediately re-enters from the opposite face. This means that when calculating the force between two particles, the "wrap-around" distance must be considered. We always use the shortest path between the two particles. This is known as the **Minimum Image Convention**, and it ensures that no particle ever feels an artificial "edge of the universe."

Gravitational Softening

Newton's law of gravity, $F \propto 1/r^2$, has a mathematical singularity: as the distance r between two particles approaches zero, the force between them approaches infinity. In a simulation that moves in discrete time steps, these immense forces can cause particles to be catapulted away at unrealistic speeds, compromising the simulation's stability and energy conservation.

To prevent this, we introduce gravitational softening. Instead of treating particles as infinitely small points, this technique treats them as extended, spherical clouds of mass. We modify Newton's law of gravity by introducing a softening length, ϵ (epsilon). The classic representation of this is the Plummer force law:

$$F = \frac{Gm_1 m_2 r}{(r^2+\epsilon^2)^{3/2}}$$

When particles are far apart, they feel the normal $1/r^2$ force. However, when their separation becomes comparable to or smaller than ϵ, they begin to pass "inside" each other's density clouds. As r approaches zero, the force is softened and gradually drops to zero. This mimics reaching the center of a mass cloud, where the gravitational pull from all sides cancels out.

A simple rule of thumb is to base the softening length on the **mean inter-particle spacing**, d. For a box with side L and N particles, it's calculated as:

$$d = \frac{L}{N^{1/3}}$$

The softening length is then a small fraction of d, such as $\epsilon = 0.03d$. The gravitational softening will be explained in more detail in a later section.

Key Literature & Further Reading
Bagla, J. S., & Padmanabhan, T. (2004). *Cosmological N-Body Simulations.* arXiv:astro-ph/0411730. Available at https://arxiv.org/pdf/astro-ph/0411730.pdf

The Integrator

The Euler Method

To move the particles through time, we need an "integrator"—an algorithm that takes the current state of a particle (its position and velocity) and predicts its state a small moment later. A straightforward approach is the **Euler method**.

The Euler method assumes that the velocity and acceleration are constant over one small time step, Δt. It calculates the force on the particle at its current position to find its acceleration, and then takes a linear step forward.

The update equations are:

1. **Update Position:** $\mathbf{x}_{n+1} = \mathbf{x}_n + \mathbf{v}_n \Delta t$
2. **Update Velocity:** $\mathbf{v}_{n+1} = \mathbf{v}_n + \mathbf{a}_n \Delta t$

While simple, the Euler method's is blind to any changes that occur during the step. This error, while tiny on each step, is **systematic**. Over thousands of steps, it accumulates, failing to conserve energy. This makes the Euler method unsuitable where long-term stability is important.

Velocity Verlet

Velocity Verlet is an integrator that accounts for the fact that forces change *during* a time step. Instead of just using the acceleration from the beginning of the step, it uses the average of the accelerations from the beginning and the end of the step.

The algorithm proceeds in three steps:

1. **Calculate the New Position:** First, advance the position using the current velocity and acceleration.
$$\mathbf{x}(t + \Delta t) = \mathbf{x}(t) + \mathbf{v}(t)\Delta t + \frac{1}{2}\mathbf{a}(t)\Delta t^2$$
2. **Calculate the New Acceleration:** With the new position, calculate the new force vector $\mathbf{F}(\mathbf{x}(t + \Delta t))$ and from it, the new acceleration.
$$\mathbf{a}(t + \Delta t) = \frac{\mathbf{F}(\mathbf{x}(t + \Delta t))}{m}$$
3. **Calculate the New Velocity:** Finally, update the velocity using the **average** of the old acceleration $\mathbf{a}(t)$ and the new acceleration $\mathbf{a}(t + \Delta t)$.
$$\mathbf{v}(t + \Delta t) = \mathbf{v}(t) + \frac{\mathbf{a}(t) + \mathbf{a}(t + \Delta t)}{2}\Delta t$$

This final step of averaging the accelerations corrects the systematic drift of the Euler method, and makes Velocity Verlet a **symplectic integrator**. This enables the algorithm to produce stable trajectories, conserving energy remarkably well over long periods.

Symplectic Integration

A **symplectic integrator** is designed to respect the underlying geometry of physics, a property that allows it to conserve a system's total energy over very long periods. This is best understood by comparing how different integrators handle a simple gravitational problem, like a planet orbiting a star.

- A **non-symplectic** integrator, like the Euler method, consistently makes an error in the same direction. It always "cuts the corner" of the orbit, pushing the planet slightly outwards. These errors add up, causing the planet's energy to systematically increase and its orbit to spiral away.

- A **symplectic** integrator, like Verlet, makes errors that are correlated. On one step, it might slightly overshoot the true orbit, but on a later step, it will undershoot it. The errors effectively cancel each other out over time. The simulated planet executes a stable "wobble" along the correct orbital path. The shape of the orbit might oscillate, but its average size and energy remain correct in the long term.

The deeper reason for this stability lies in a concept from classical mechanics called **phase space**. Phase space is an abstract map where every point represents the complete state of a particle—both its **position** and its **momentum**. For a system where energy is conserved, a rule known as **Liouville's Theorem** states that the "area" (or volume) of any group of states in phase space must stay constant as the system evolves.

Symplectic integrators **preserve this phase space volume**. The bounded energy error (the "wobble") is a direct consequence of this property.

The Kick-Drift-Kick Integrator

The introduction of cosmic expansion adds a velocity-dependent term to the equations of motion (the "Hubble drag", explored in a later section). This new term creates a challenge for the standard Verlet algorithm because its symplectic nature is defined for forces that depend only on position, not velocity. To handle this new term gracefully, we adopt a different formulation known as a **Leapfrog** scheme. The most common implementation, the **Kick-Drift-Kick (KDK)** integrator, is a common choice in cosmological simulations.

The KDK scheme advances the system from a time t to $t + \Delta t$ in three stages:

1. First "Kick" (Velocity Half-Step)

The velocities are "kicked" forward by half a time step using the acceleration from the beginning of the step.

$$\mathbf{v}(t + \tfrac{1}{2}\Delta t) = \mathbf{v}(t) + \mathbf{a}(t)\frac{\Delta t}{2}$$

2. "Drift" (Position Full-Step)

The positions then "drift" for a full time step using the more accurate **mid-step velocity**.

$$\mathbf{x}(t + \Delta t) = \mathbf{x}(t) + \mathbf{v}(t + \tfrac{1}{2}\Delta t)\Delta t$$

3. Second "Kick" (Velocity Second Half-Step)

Finally, the new acceleration, $\mathbf{a}(t + \Delta t)$, is computed from the forces at the new positions, $\mathbf{x}(t + \Delta t)$, and the **mid-step velocity**, $\mathbf{v}(t + \frac{1}{2}\Delta t)$. The acceleration is then used to complete the velocity update for the second half of the time step.

$$\mathbf{v}(t + \Delta t) = \mathbf{v}(t + \tfrac{1}{2}\Delta t) + \mathbf{a}(t + \Delta t)\frac{\Delta t}{2}$$

This staggered formulation is more robust for handling the time-varying and velocity-dependent forces present in a cosmological simulation.

Symmetry and Second-Order Accuracy

In numerical physics, the "order of accuracy" dictates how fast errors shrink when we take smaller timesteps (Δt). The Forward Euler method is only *first-order* accurate ($O(\Delta t)$); if we cut the timestep in half, the error only drops by half.

The KDK scheme achieves higher accuracy through its **symmetry**. By splitting the velocity kick into two equal halves that bracket the position drift, the algorithm becomes **time-reversible**. For conservative forces like gravity, this means that if we paused the simulation, reversed all the velocities, and ran the KDK math backward, the particles would trace their steps back to their starting positions.

In numerical calculus, any integration scheme that is symmetric and time-reversible causes the leading, first-order Taylor series error terms to cancel each other out. Because the first-order error is gone, the largest surviving error scales with the square of the timestep ($O(\Delta t^2)$). This means KDK is **second-order accurate**—if we cut the timestep in half, the integration error drops by a factor of four.

Key Literature & Further Reading

Springel, V. (2005). *The cosmological simulation code GADGET-2. Monthly Notices of the Royal Astronomical Society*, 364(4), 1105-1134. arXiv:astro-ph/0505010. Available at https://arxiv.org/abs/astro-ph/0505010

The Particle-Mesh Method

Instead of calculating the gravitational pull between every pair of particles, the Particle-Mesh (PM) method simplifies the problem by describing the mass distribution on a regular grid. From this "mass map", the gravitational potential and forces can be solved on the grid itself. These are the steps:

1. **Potential calculation:** First, the gravitational potential (Φ) is calculated for the entire grid. The potential is a scalar "landscape" that describes the depth of the gravitational well at every point.
2. **Force calculation:** Second, the force ($\mathbf{F}$) is determined by finding the steepest downhill slope (the gradient) of that potential landscape.

This `Mass -> Potential -> Force` pipeline is the foundation of the PM method. The following sections break down how each part of this process is achieved.

Step 1. Finding the Potential

The process of finding the potential begins by describing the mass distribution on the grid.

Mass Assignment (NGP)

Mass assignment is the procedure for transferring the mass of the continuously positioned particles onto the discrete nodes of the grid.

The simplest and most intuitive way to do this is the **Nearest Grid Point (NGP)** scheme: for each particle, we find the single grid point (or cell center) that it is closest to, and assign the particle's *entire mass* to that one point.

The result is an array representing the mass density field, $\rho_{i,j,k}$. Mathematically, the density in a given cell (i, j, k) is the sum of the masses of all particles within that cell, divided by the cell's volume:

$$\rho_{i,j,k} = \frac{1}{L^3} \sum_{p \in \text{cell}(i,j,k)} m_p$$

Where m_p is the mass of a particle p, and L is the side length of a grid cell.

While NGP is very simple, it introduces inaccuracies. As we will explore in a later section, more sophisticated schemes like Cloud-in-Cell (CIC) can be used to create a smoother and more accurate density field.

Poisson's Equation

The potential field Φ can be determined from the mass density field, $\rho_{i,j,k}$, through the **Poisson's Equation**.

$$\nabla^2 \Phi = 4\pi G \rho$$

Where:

- ρ is the mass density grid — the **input**.
- Φ is the gravitational potential field — the **output**.
- G is the gravitational constant.
- ∇^2 (the **Laplacian**) is a mathematical operator that measures how much a function curves around a point—the **net curvature**.

In this equation, mass acts as the source of curvature: where there is mass, the potential bends inward, forming gravitational wells. Where $\rho = 0$, there's no net curvature. This may seem counterintuitive, as the potential field forms a curved, gravitational well even in the empty space around a mass. The key is that the Laplacian, $\nabla^2\Phi$, measures the **net curvature**. In the smooth $1/r$ shape of a potential, the radial inward bending of the field is balanced by a natural geometric spreading effect in three dimensions. These two effects cancel each other out, resulting in zero net curvature.

Possion's equation allows us to transition from Newton's "particle-to-particle" worldview to a continuous "field" worldview. As we will see in the next section, writing gravity in this form allows us to take advantage of the Convolution Theorem and Fast Fourier Transforms (FFTs) to solve the gravitational field in a fraction of the time. Here is the step-by-step derivation from Newton's law of gravity to Poisson's equation.

Step 1: The Gravitational Field of a Point Mass We start with the standard Newtonian gravitational potential for a single point mass M at the origin:

$$\Phi = -\frac{GM}{r}$$

The gravitational field (the acceleration vector $\boldsymbol{g}$) is simply the negative gradient of this potential. Taking the derivative with respect to r gives us the classic inverse-square law:

$$\boldsymbol{g} = -\nabla\Phi = -\frac{GM}{r^2}\hat{\boldsymbol{r}}$$

Step 2: The Gravitational Flux (Gauss's Law) Imagine wrapping that point mass inside a spherical surface (a Gaussian surface) with radius r. We want to calculate the total "flux" of the gravitational field pointing inward through that surface.

To do this, we integrate the field $\boldsymbol{g}$ over the surface area A of the sphere ($A = 4\pi r^2$):

$$\oint_S \boldsymbol{g} \cdot d\boldsymbol{A} = \left(-\frac{GM}{r^2}\right)(4\pi r^2) = -4\pi GM$$

Notice how the r^2 terms perfectly cancel out. The total flux through the surface depends *only* on the mass enclosed inside it, regardless of the size or shape of the boundary.

Step 3: Transitioning to a Continuous Fluid In a cosmological simulation, we don't just have one point mass; we map billions of particles onto a continuous density grid, represented by $\rho(\boldsymbol{r})$.

The total enclosed mass M is simply the volume integral of the density field:

$$M = \int_V \rho(\boldsymbol{r})dV$$

Substitute this into our flux equation from Step 2:

$$\oint_S \boldsymbol{g} \cdot d\boldsymbol{A} = -4\pi G \int_V \rho(\boldsymbol{r})dV$$

Step 4: The Divergence Theorem Now we use the **Divergence Theorem**. This theorem states that the flux of a vector field across a closed boundary is exactly equal to the volume integral of the *divergence* ($\nabla\cdot$) of that field inside the boundary.

Applying it to our gravitational field gives:

$$\oint_S \boldsymbol{g} \cdot d\boldsymbol{A} = \int_V (\nabla \cdot \boldsymbol{g})dV$$

Step 5: Equating the Integrals Because Step 3 and Step 4 are calculating the same flux, we can set them equal to each other:

$$\int_V (\nabla \cdot \boldsymbol{g})dV = \int_V (-4\pi G\rho)dV$$

Because this must be true for *any* arbitrary volume V we choose in our simulation box, the integrands themselves must be identical. We can strip away the integrals to get the differential form of Gauss's Law:

$$\nabla \cdot \boldsymbol{g} = -4\pi G\rho$$

Step 6: The Poisson Equation Remember from Step 1 that the gravitational field is the negative gradient of the potential ($\boldsymbol{g} = -\nabla\Phi$).

Substitute that definition back into our differential equation:

$$\nabla \cdot (-\nabla\Phi) = -4\pi G\rho$$

The divergence of a gradient ($\nabla \cdot \nabla$) is the **Laplacian operator**, denoted as ∇^2. Pulling the negative sign out and canceling it on both sides gives us the final equation:

$$\nabla^2\Phi = 4\pi G\rho$$

Solving Poisson's equation means finding the global shape of Φ given all the local sources ρ. As we'll see next, the Fast Fourier Transform (FFT) offers an efficient way to compute it.

The FFT and the Convolution Theorem

Given the mass density grid, ρ, and the rule connecting it to the potential, Poisson's Equation, the challenge now is to solve it. Although Poisson's equation is written as a differential equation (∇^2), solving it in real space means integrating Newton's law: calculating the potential at every grid point by summing the $1/r$ influence from all other grid points. This computationally expensive task operation is known as a **convolution**.

Fortunately, there is a more efficient mathematical tool that can solve this problem: the **Fast Fourier Transform (FFT)**. This algorithm is used to take advantage of the **Convolution Theorem**.

The Fourier Transform The **Fourier Transform** is a mathematical operation that rewrites any spatial field in terms of its **spatial frequencies** — the underlying wave patterns that make it up.

Suppose we have a density field $\rho(\mathbf{x})$ defined across space. Instead of describing it point by point, we can express it as a sum of smooth oscillating patterns — *plane waves* — each with its own wavelength, direction, and amplitude. The Fourier Transform tells us **how each wave** contributes to the total field.

Formally, it is written as:

$$\hat{\rho}(\mathbf{k}) = \int \rho(\mathbf{x}) e^{-i\mathbf{k}\cdot\mathbf{x}} d^3x$$

Here, $\mathbf{x}$ represents position in real space, and $\mathbf{k}$ is the **wavevector**, describing one of those plane waves. Each component (k_x, k_y, k_z) measures how rapidly the field oscillates along that direction, while its magnitude

$$k = |\mathbf{k}| = \frac{2\pi}{\lambda}$$

tells us the *spatial frequency*.

The result of the transform, $\hat{\rho}(\mathbf{k})$, is a **complex number**. Its magnitude $|\hat{\rho}(\mathbf{k})|$ gives the *amplitude*, and its phase $\arg(\hat{\rho}(\mathbf{k}) = \arctan(\text{Im}/\text{Re}))$ specifies the *offset* — where the wave's peaks and troughs occur in space.

Thus, while $\rho(\mathbf{x})$ is a **real-valued function of position**, $\hat{\rho}(\mathbf{k})$ is a **complex-valued function of wavevector**. They describe the same field, but from two complementary perspectives: one in **Real space**, one in **Frequency (or k-) space**. This dual representation is very useful in simulations.

> In what follows, we'll use the shorthand notation ρ_k and Φ_k to denote the discrete Fourier-transformed fields, corresponding to $\hat{\rho}(\mathbf{k})$ and $\hat{\Phi}(\mathbf{k})$ in the continuous case, but defined only at the discrete $\mathbf{k}$ values of our simulation grid.

The Convolution Theorem This theorem is the heart of the entire PM method. It states:

> A slow and complicated **convolution** in real space becomes a fast and simple element-by-element **multiplication** in frequency space.

This allows us to replace the slow, brute-force calculation with a faster three-step process:

1. **Transform to Frequency Space:** We use the **FFT** to transform our mass grid, ρ, into its frequency representation, ρ_k. A convenient property of the FFT is that it automatically treats the finite grid as if it were periodic. It represents the data as a sum of simple waves that fit perfectly end-to-end within the box, which is mathematically equivalent to assuming the grid repeats infinitely like a tiled pattern.

   To compute the gravitational potential, we must understand how a single unit of mass influences the space around it. In linear mathematics, the tool for this is a Green's function. Broadly speaking, a Green's function represents the "impulse response" of a system to a single, localized point of input. Once you know how a system reacts to one point, you can determine its reaction to any complex input by summing together the individual point responses.

   For our specific case, the "system" is the gravitational field governed by Poisson's equation. Therefore, our specific Green's function, $G(\mathbf{r})$, is defined as the solution to Poisson's equation for a unit point source:

$$\nabla^2 G(\mathbf{r}) = 4\pi G\delta(\mathbf{r}).$$

   Where $\delta(\mathbf{r})$ is the Dirac delta function, representing the mass density of an idealized point source located at the origin—a mathematical spike that is infinitely dense at that single spot and zero everywhere else, containing exactly one unit of total mass. In three dimensions, solving this gives the familiar Newtonian potential for a point mass: $G(\mathbf{r}) = -G/|\mathbf{r}|$. This mathematical kernel acts as the system's response to a point mass, linking the density field to the potential through Poisson's equation.

   When we move to frequency space, derivatives become multiplications by $-k^2$, so the corresponding frequency-space form of the Green's function is

$$\mathcal{F}\{\nabla^2 G(\mathbf{r})\} = -k^2 G_k$$

$$\mathcal{F}\{4\pi G\delta(\mathbf{r})\} = 4\pi G$$

$$G_k = -\frac{4\pi G}{k^2}.$$

   This is the function we use to compute the potential in the Fourier domain. This function has a mathematical singularity at the $k = 0$ **mode**. This mode (also known as the DC component) represents the **average potential** of the entire simulation box.

   However, since physical forces depend only on the **gradient** of the potential ($\mathbf{F} = -\nabla\Phi$), not its absolute value, this average potential is physically arbitrary. To avoid the division-by-zero, we can redefine the $k = 0$ component of the potential to zero. Beyond just fixing a numerical error, this is mathematically equivalent to subtracting the mean background density of the universe from the source term. This is a standard technique in cosmology (often related to the "Jeans swindle"), which ensures that gravity is driven only by local *perturbations* (overdensities and underdensities) rather than the infinite mass of a periodic universe. It makes the calculation perfectly well-defined without affecting the final relative forces.

2. **Multiply:** In frequency space, the Poisson equation becomes an element-wise multiplication:

$$\Phi_k = G_k \cdot \rho_k$$

3. **Transform Back:** We take the resulting potential in frequency space, Φ_k, and use the **Inverse Fast Fourier Transform (IFFT)** to convert it back into the real-space potential grid, Φ, that we wanted.

With this process, we replace a slow algorithm that scales as $O(M^6)$ (for a 3D grid with $M \times M \times M$ cells) with one that scales as $O(M^3 \log M)$. This is what makes the Particle-Mesh method convenient.

Step 2. From Potential to Force

Now that we have the potential grid, Φ, we must calculate the force grid. The physical relationship is universal: force is the negative gradient of the potential.

$$\mathbf{F} = -\nabla\Phi$$

On a discrete grid, we can't take a true derivative. We approximate it using a **finite difference**. A common and accurate method is the **central difference**, which calculates the slope at a point by looking at the values of its neighbors on either side. For the x-component of the force at grid cell (i, j, k), the formula is:

$$F_{x,i,j,k} \approx -\frac{\Phi_{i+1,j,k} - \Phi_{i-1,j,k}}{2L}$$

With the force calculated at every point on the grid, the final step is to **interpolate** this force back to each particle's continuous position. This is done using the same scheme we used for mass assignment (e.g., NGP or CIC), completing the Particle-Mesh calculation.

Advanced Interpolation

The Flaws of Nearest Grid Point (NGP)

In a previous section, we introduced the Nearest Grid Point (NGP) scheme as the simplest way to assign mass to the grid. The primary flaw of NGP is that the force a particle feels is **discontinuous**. The jerky, stepwise force from an NGP grid is a poor and unphysical approximation to a real smooth gravitational field. This leads to several significant problems:

1. **Poor Energy Conservation:** Symplectic integrators can only conserve energy if the force is the smooth gradient of a potential. The sudden "jumps" in force at the cell boundaries introduce small, systematic errors into the integration. These errors accumulate over time, causing the total energy of the simulation to **drift**, rather than just oscillating around the true value.
2. **Break of translational invariance:** As particles move through the grid, the forces between them violently snap on and off in staircase-like steps depending on when they cross cell boundaries. This injects artificial kinetic energy into the system.
3. **Grid-Imposed Artifacts:** The force field has an artificial, grid-like pattern. Particles can feel an unphysical pull along the grid axes (x and y) that is stronger than the pull along the diagonals. This can cause particles to artificially cluster along grid lines.

Because of these flaws, NGP is rarely used in simulations.

Cloud-in-Cell (CIC)

To achieve a stable simulation that conserves energy, we need a smooth way to connect the particles to the grid. A widely used method for this is the **Cloud-in-Cell (CIC)** interpolation scheme.

Particles as Clouds

Instead of treating each particle as an infinitesimal point, the CIC method treats each particle as a small, **cubic "cloud"** of mass, the same size as a grid cell. As this particle-cloud moves through the simulation space, it overlaps with the **eight** nearest grid points that form the corners of its current cell.

The mass of the particle is then distributed, or "splatted," onto these eight grid points. The amount of mass assigned to each point is proportional to the **volume of overlap** between the particle's cloud and the region surrounding each grid point. This is a form of **trilinear interpolation**. A particle in the exact center of a cubic cell would distribute 12.5% of its mass to each of the eight corners. A particle mostly in one corner of a cell would give most of its mass to that corner's node.

This process results in a smoother and more realistic mass density grid. A small movement by a particle leads to a small, continuous change in the mass distribution on the grid, eliminating the sudden "jumps" of the NGP method.

At its core, CIC is a linear interpolation scheme. Higher-order schemes (like TSC or PCS) exist and provide smoother forces, but are not in the scope of this text.

Symmetric Interpolation

After the forces have been calculated on the grid, we must interpolate them back to the particle's continuous position. The rule for momentum conservation is symmetry: the **force interpolation scheme must be consistent with the mass assignment scheme**.

CIC follows this rule if the force on the particle is calculated by taking a weighted average of the forces from the **same eight grid points**, using the **same volume-based weights** that were used to distribute the mass.

This symmetry ensures that Newton's third law ($\mathbf{F}_{ij} = -\mathbf{F}_{ji}$) is obeyed for any pair of particles. When these forces are fed into a symplectic integrator, the system's total energy is conserved remarkably well. This makes CIC a common choice for modern codes.

Implementation: "Splatting" Mass and "Gathering" Forces

The conceptual idea of treating particles as "clouds" translates into a two-part algorithm. These two parts are often called **"splatting"** (distributing the particle mass onto the grid) and **"gathering"** (interpolating the force from the grid back to the particle).

For simplicity, the following explanation is for a 2D case.

The "Splat" Step: Mass Assignment

This process is performed for every particle to create the final mass density grid, ρ.

1. Find the Reference Grid Point and Fractional Position

First, for a given particle p with position $\mathbf{x}_p = (x_p, y_p)$, we find the integer index of the grid point at its "bottom-left," denoted (i, j). We then find the particle's fractional position within that cell, (δ_x, δ_y), where both values range from 0 to 1. Let L be the side length of a grid cell.

The reference grid index is found using the floor function, $\lfloor \cdot \rfloor$:

$$i = \lfloor x_p/L \rfloor$$

$$j = \lfloor y_p/L \rfloor$$

The fractional position within the cell is then:

$$\delta_x = (x_p/L) - i$$

$$\delta_y = (y_p/L) - j$$

2. Calculate the Weights

Next, we calculate four weights based on these fractional positions. Each weight corresponds to the fractional area of the particle's "cloud" that overlaps with the four surrounding grid cells located at (i, j), $(i + 1, j)$, $(i, j + 1)$, and $(i + 1, j + 1)$.

The weights for the corners are:

$$w_{i,j} = (1 - \delta_x)(1 - \delta_y)$$

$$w_{i+1,j} = \delta_x(1 - \delta_y)$$

$$w_{i,j+1} = (1 - \delta_x)\delta_y$$

$$w_{i+1,j+1} = \delta_x\delta_y$$

Notice that these four weights always sum to 1.

3. Distribute the Mass

Finally, we add the mass of the particle, m_p, scaled by the appropriate weight, to each of the four corresponding grid points. This process is repeated for all particles in the simulation. The total mass on a given grid point is the sum of the contributions from all particles whose "clouds" overlap it.

The contribution from a single particle p to the mass at each of the four nodes is:

$$\Delta M_{i,j} = m_p \cdot w_{i,j}$$

$$\Delta M_{i+1,j} = m_p \cdot w_{i+1,j}$$

$$\Delta M_{i,j+1} = m_p \cdot w_{i,j+1}$$

$$\Delta M_{i+1,j+1} = m_p \cdot w_{i+1,j+1}$$

To get the final mass density grid, ρ, the total mass accumulated at each node is divided by the cell area, L^2. All grid indices are taken modulo the grid size to correctly handle the periodic boundaries.

The "Gather" Step: Force Interpolation

This step occurs after the forces have been calculated on the grid (creating an acceleration field, $\mathbf{a}_{\text{grid}}$) and is the mirror image of the splatting process.

To find the force on a particle, we use the **same** indices and weights we calculated for it in the splatting step. We then perform a weighted average of the acceleration values from the four surrounding grid points to find the acceleration at the particle's precise location, $\mathbf{a}_p$.

Let the acceleration field on the grid be $\mathbf{a}_{i,j} = (a_{x,i,j}, a_{y,i,j})$ and the four CIC weights for a given particle be $w_{i,j}$, $w_{i+1,j}$, $w_{i,j+1}$, and $w_{i+1,j+1}$.

The x-component of the interpolated acceleration for the particle, $a_{x,p}$, is calculated as:

$$a_{x,p} = a_{x,i,j} \cdot w_{i,j} + a_{x,i+1,j} \cdot w_{i+1,j} + a_{x,i,j+1} \cdot w_{i,j+1} + a_{x,i+1,j+1} \cdot w_{i+1,j+1}$$

The y-component, $a_{y,p}$, is calculated in the same way using the y-components of the grid acceleration field.

The final force on the particle, $\mathbf{F}_p$, is its mass, m_p, times this interpolated acceleration vector:

$$\mathbf{F}_p = m_p \mathbf{a}_p$$

Deconvolving the Mass Assignment

The "Splat-Gather" procedure introduces an artifact into the gravity solver. In mathematics, any time we take a set of discrete points and "smear" them across a spatial domain, we are performing a **convolution**. The CIC mass assignment smears a point mass into a shape equivalent to convolving a 1D uniform boxcar (a "top-hat") with itself, creating a triangular density cloud.

In Fourier analysis, the **Convolution Theorem** dictates that a complex convolution in real space is identical to a simple multiplication in frequency space. The Fourier transform of a 1D top-hat function is the sinc function, defined as:

$$\text{sinc}(x) = \frac{\sin(x)}{x}$$

Because the CIC shape is the convolution of two top-hats, its Fourier transform is sinc^2. In a 3D grid with cell size Δx, the continuous CIC assignment acts as a mathematical filter, $W(\mathbf{k})$, that multiplies the true density field in frequency space:

$$W(\mathbf{k}) = \text{sinc}^2\left(\frac{k_x \Delta x}{2}\right) \cdot \text{sinc}^2\left(\frac{k_y \Delta x}{2}\right) \cdot \text{sinc}^2\left(\frac{k_z \Delta x}{2}\right)$$

This creates a numerical problem. The simulation applies this CIC filter *twice*—once when splatting the mass onto the grid ($\rho_{\text{grid}} = \rho_{\text{true}} * W$), and once when gathering the forces back to the particles ($\mathbf{F}_{\text{particle}} = \mathbf{F}_{\text{grid}} * W$).

Because the filter is applied twice, the total numerical dampening applied is $W(\mathbf{k})^2$. This means the gravitational potential is accidentally multiplied by a factor of $\mathbf{sinc}^4$ along every axis. Since the sinc function decays as frequencies get higher, this accidental sinc^4 multiplication artificially weakens the gravitational pull at intermediate scales (typically distances of 1 to 3 grid cells).

To fix this, we must undo the accidental CIC convolution during the gravity calculation. This process is known as **deconvolution**.

When we solve for the gravitational potential in Fourier space, we calculate the value of the sinc function for every specific wavevector (k_x, k_y, k_z). We then divide the potential by the sinc^4 penalty. The equation for the basic Particle-Mesh potential in Fourier space becomes:

$$\Phi_k = \rho_k \cdot \left(\frac{-4\pi G}{k^2}\right) \cdot \frac{1}{W(\mathbf{k})^2}$$

Key Literature & Further Reading

Bagla, J. S., & Padmanabhan, T. (2004). *Cosmological N-Body Simulations.* arXiv:astro-ph/0411730. Available at https://arxiv.org/pdf/astro-ph/0411730.pdf

The P³M Algorithm

Combining PP for Short-Range and PM for Long-Range

We have seen that the Particle-Mesh (PM) method is efficient for calculating the gravitational field of a large number of particles. However, its speed comes at the cost of accuracy at small scales. The grid is good at capturing the overall "blurry" shape of the gravitational field, but it's innacurate at resolving the sharp, fine details of the force between two particles that are very close to each other. This inaccuracy at short ranges is the primary weakness of the pure PM method.

On the other hand, the direct Particle-Particle (PP) calculation is the exact opposite. While it is perfectly accurate at all scales, its weakness, is that its $O(N^2)$ complexity makes it too slow for a large number of particles.

This presents a classic trade-off: speed or accuracy. The **Particle-Particle Particle-Mesh (P³M)** algorithm combines both methods, using each one only where it excels.

The P³M method splits the force calculation into two parts based on a **cutoff radius**, r_c:

1. **Long-Range Force (PM):** The smooth pull from all the **distant** particles (those farther than r_c) is calculated efficiently using the Particle-Mesh method.
2. **Short-Range Force (PP):** The sharp, strong force from the few **nearby** particles (those closer than r_c) is calculated using the direct Particle-Particle method.

The Subtractive Scheme

Simply adding these two forces together would be incorrect, as the PM method already includes an inaccurate estimate of the short-range forces. Instead, we use the PP calculation to *correct* the PM force at short distances. This is often done with a **subtractive scheme**:

$$\mathbf{F}_{\text{total}} = \mathbf{F}_{\text{PM}} + \left(\mathbf{F}_{\text{PP}}^{\text{short}} - \mathbf{F}_{\text{PM}}^{\text{short}}\right)$$

The process is straightforward:

1. First, we calculate the baseline $\mathbf{F_{PM}}$ for all particles. This gives us the correct long-range force everywhere but an incorrect "blurry" force for nearby pairs.
2. Then, for any pair of particles closer than the cutoff radius, we calculate the **true force** between them, $\mathbf{F}_{\mathbf{PP}}^{\mathbf{short}}$.
3. We also calculate an approximation of the **blurry, inaccurate force** that the PM method produced for that same pair, $\mathbf{F}_{\mathbf{PM}}^{\mathbf{short}}$.
4. Finally, we subtract the inaccurate mesh force and add the correct direct force. This replaces the blurry grid force with the sharp, accurate PP force, but only where it matters—at short distances.

P³M tries to get the best of both worlds by using the fast PM algorithm for the vast majority of interactions (the weak pulls from distant particles) and reserving the slow but accurate PP algorithm only for the few interactions between close neighbors.

Calculating the Mesh-Force Correction

To implement the subtractive scheme, we need a mathematical function for $\mathbf{F}_{\text{PM}}^{\text{short}}$ that approximates the "blurry" force produced by the grid at short distances. We can't get this from the final grid itself, as it contains the combined influence of all particles.

Instead, we model this effect with a standard gravitational force formula that has been **softened** with a special parameter, ϵ_{PM}, chosen specifically to mimic the resolution of the Particle-Mesh grid.

The vector force that approximates the mesh's influence between two particles with masses m_1 and m_2 is given by:

$$\mathbf{F}_{\text{PM}}^{\text{short}} = \frac{G m_1 m_2 \mathbf{r}}{\left(r^2 + \epsilon_{\text{PM}}^2\right)^{3/2}}$$

The terms in this formula are:

- $\mathbf{r}$ is the vector separating the two particles.
- r is the magnitude of that vector, $r = \|\mathbf{r}\|$.

- G, m_1, m_2 are the gravitational constant and the particle masses.
- ϵ_{PM} is a **softening length** specifically chosen to match the grid's resolution. An effective choice is to set this value to be proportional to the grid cell length, L. For example:

$$\epsilon_{\text{PM}} \approx 0.5 \cdot L$$

This formula creates a force that is significantly weakened at short distances (when $r \lesssim \epsilon_{\text{PM}}$), which successfully mimics the behavior of the full PM/FFT calculation.

Choosing the Cutoff Radius (r_c)

The choice of the cutoff radius, r_c, involves a trade-off between accuracy and computational speed.

- A **small** cutoff radius means the PM method handles most of the work, but we risk losing accuracy if the cutoff is smaller than the region where the PM force is unreliable.
- A **large** cutoff radius ensures high accuracy at short ranges, but it forces the slow PP calculation to do much more work.

The optimal choice is linked to the resolution of the Particle-Mesh grid. The PM method's accuracy degrades significantly at distances smaller than about 2 to 3 grid cell sizes. Therefore, the cutoff radius must ensure the PP method is used throughout this entire "inaccurate zone."

A rule of thumb is to set the cutoff radius to be a few times the grid cell length, L:

$$r_c \approx 2.5 \cdot L$$

The Switching Function

Using a hard cutoff radius—where the correction is fully active if $r < r_c$ and instantly zero if $r \geq r_c$—can create an abrupt "jolt" in the force. This discontinuity, however small, can introduce numerical errors and impact the long-term energy conservation of the simulation.

We must ensure the total force is smooth at all distances. This is accomplished by introducing a **switching function**, $S(r)$, that smoothly "fades out" the short-range correction as the particle separation, r, approaches the cutoff radius, r_c.

The total force is then calculated as:

$$\mathbf{F}_{\text{total}} = \mathbf{F}_{\text{PM}} + S(r) \cdot \left(\mathbf{F}_{\text{PP}}^{\text{short}} - \mathbf{F}_{\text{PM}}^{\text{short}}\right)$$

The switching function $S(r)$ operates over a small **transition zone**, typically defined between a starting radius, r_{start}, and the cutoff radius, r_c. It has the following properties:

1. For $r \leq r_{\text{start}}$, the function is $S(r) = 1$. The correction is fully applied.
2. For $r \geq r_c$, the function is $S(r) = 0$. The correction is fully turned off.
3. In the transition zone, $r_{\text{start}} < r < r_c$, the function smoothly decreases from 1 to 0.

To ensure the force changes smoothly, the *derivative* of the switching function should also be zero at the start and end of the transition. An effective way to achieve this is with a cubic polynomial.

First, we define a normalized distance, x, that goes from 0 to 1 across the transition zone:

$$x = \frac{r - r_{\text{start}}}{r_c - r_{\text{start}}}$$

Then, a polynomial that satisfies the smoothness conditions is:

$$S(x) = 2x^3 - 3x^2 + 1$$

Using this function to taper the correction term eliminates the jolt at the cutoff. It creates a continuous and differentiable total force, which leads to superior long-term energy conservation.

Key Literature & Further Reading
Shirokov, A., & Bertschinger, E. (2005). *GRACOS: Scalable and Load Balanced P3M Cosmological N-body Code.* arXiv:astro-ph/0505087. Available at https://arxiv.org/abs/astro-ph/0505087

Fourier-Split PM

In our previous P³M approach, we attempted to correct the short-range errors of the Particle-Mesh grid by subtracting an analytical approximation of the grid's force ($F_{\text{PM}}^{\text{short}}$) and replacing it with the exact Newtonian force (F_{PP}). This scheme assumes that the grid's short-range force is smooth, isotropic (equal in all directions), and predictable (modeled as a softened $1/r^2$ curve).

In reality, the force produced by a finite-difference grid at sub-cell distances ($r < \Delta x$) is **anisotropic** and polluted by grid artifacts, like **artificial repulsion** caused by the Gibbs phenomenon.

The Gibbs phenomenon arises when a localized, sharp function—such as a dense point-mass is projected onto a grid. A discrete spatial grid can only resolve frequencies up to the Nyquist limit ($k_{\text{Nyq}} = \pi/\Delta x$). If the code solves Poisson's equation in Fourier space by multiplying the discrete density modes by the continuous analytical Green's function ($-4\pi G/k^2$), it effectively applies a cutoff to all frequencies above k_{Nyq}.

Multiplying by a sharp frequency cutoff in Fourier space is equivalent to convolving the potential with a sinc function ($\sin(x)/x$) in real space. Because the sinc function oscillates between positive and negative values, the resulting gravitational potential does not decay monotonically as $-1/r$. Instead, it "rings," creating concentric ripples of artificial potential hills and troughs.

This is the cause of artificial repulsion. The slope of these ripples can overpower the Newtonian slope, flipping the net gradient, and causing particles to repel each other.

Because the PM force is noisy, direction-dependent, and occasionally repulsive, a smooth, isotropic analytical formula ($F_{\text{PM}}^{\text{short}}$) can't match it. When we subtract the smooth formula from the grid, the subtraction fails to cancel out the error. Instead, it leaves a jagged, unphysical residual force field. This "mesh artifact" causes particles to artificially cluster along the x, y, and z grid axes and introduces jolts that slowly destroy the long-term energy conservation.

Fixing the Grid in Frequency Space

A good solution is to eliminate the error before it reaches real space. The Fourier-Split PM method forces the grid's potential to be smooth by intervening in the middle of the Fast Fourier Transform (FFT) pipeline.

Recall that in frequency space (k-space), solving Poisson's equation is a simple multiplication:

$$\Phi_k = \rho_k \cdot \left(\frac{-4\pi G}{k^2} \right)$$

In the Fourier-Split method, we introduce a blurring function into this equation by multiplying the potential by a **Gaussian decay filter**:

$$\Phi_k = \rho_k \cdot \left(\frac{-4\pi G}{k^2} \right) \cdot \exp(-k^2 r_s^2)$$

Here, r_s is the **Gaussian smoothing scale**, a tuning parameter typically set to roughly 1.25 to 1.5 times the grid cell width (Δx).

In Fourier analysis, high frequencies (large k) correspond to small-scale, sharp, jagged details. Because the exponent is negative and proportional to k^2, the term $\exp(-k^2 r_s^2)$ plummets rapidly to zero for high-frequency modes.

We should keep the Cloud-in-Cell (CIC) deconvolution we established earlier to fix the sinc^4 introduced by the mass assignment and force interpolation steps. Combining both of these frequency-space corrections gives us the final equation for the gravitational potential:

$$\Phi_k = \rho_k \cdot \left(\frac{-4\pi G}{k^2} \right) \cdot \frac{\exp(-k^2 r_s^2)}{W(\mathbf{k})^2}$$

By applying this filter, we erase the grid's ability to resolve any structure smaller than r_s. The resulting gravitational field is no longer jagged, it no longer suffers from aliasing along the cell boundaries, and it is non-repulsive. The long-range grid force is now guaranteed to be isotropic and smooth.

However, by blurring the grid, we have also suppressed the strength of gravity at short distances. To recover the true physics, we must now define an exact short-range force to complement the newly smoothed grid.

The Exact Analytical Complement

In the classical P³M scheme, the short-range correction was an educated guess designed to subtract the grid's errors. In the Fourier-Split PM method, the short-range correction is an exact derivation.

Because we explicitly defined the shape of the long-range potential using a Gaussian filter in frequency space ($\exp(-k^2 r_s^2)$), we can use the inverse Fourier transform to find the analytical shape of that smoothed potential in real space. For a point mass M, the Gaussian-smoothed potential is:

$$\Phi_{\text{long-range}}(r) = -\frac{GM}{r}\text{erf}\left(\frac{r}{2r_s}\right)$$

Where erf is the standard Error Function. Since the true Newtonian potential is $\Phi(r) = -GM/r$, the short-range potential we *missed* by blurring the grid is simply the difference between the true potential and the long-range potential:

$$\Phi_{\text{short-range}}(r) = -\frac{GM}{r}\left[1 - \text{erf}\left(\frac{r}{2r_s}\right)\right] = -\frac{GM}{r}\text{erfc}\left(\frac{r}{2r_s}\right)$$

Here, erfc is the **Complementary Error Function**. To find the actual force that our Particle-Particle (PP) solver needs to apply, we take the negative gradient ($-\nabla$) of this short-range potential. Taking the derivative of the erfc function introduces an additional exponential term, giving us the final equation for the short-range force between two particles:

$$F_{\text{PP}}(r) = \frac{Gm_1m_2}{r^2}\left[\text{erfc}\left(\frac{r}{2r_s}\right) + \frac{r}{r_s\sqrt{\pi}}\exp\left(-\frac{r^2}{4r_s^2}\right)\right]$$

Observe the behavior of this equation:

- As $r \to 0$ (particles are very close), the terms in the brackets approach 1.0, recovering the standard $1/r^2$ Newtonian gravity.
- As $r \gg r_s$ (particles are far apart), the erfc and exponential terms rapidly drop to zero, ensuring the PP force vanishes where the PM grid takes over.

The sum of the Gaussian-filtered grid force and this erfc-damped PP force equals the true $1/r^2$ Newtonian force.

Key Literature & Further Reading
Bagla, J. S. (2002). *TreePM: A code for cosmological N-body simulations.* Journal of Astrophysics and Astronomy, 23(4), 185-196. Available at https://arxiv.org/pdf/astro-ph/9911025

An Expanding Space

Up to this point, our simulation has taken place in a static box. However, the universe is not static; it is expanding. To model the formation of structure, we must incorporate this expansion into our simulation.

This is achieved by switching from familiar "proper" coordinates to **comoving coordinates**. Instead of tracking particles in a fixed box, we track them on a virtual grid that expands along with the universe itself.

The Hubble Flow

The cosmic expansion is described by the **Hubble-Lemaître Law**. This law states that, on average, every galaxy is moving away from every other galaxy. The farther away a galaxy is, the faster it appears to recede. This is not a motion *through* space, but rather the expansion *of* space itself. This uniform expansion is the **Hubble Flow**. The velocity of this recession, $\mathbf{v}_{\text{Hubble}}$, for an object at a proper distance $\mathbf{r}$ is given by:

$$\mathbf{v}_{\text{Hubble}}(t) = H(t)\mathbf{r}(t)$$

Where $H(t)$ is the Hubble parameter at time t. This flow is the background upon which all other motions are superimposed.

Comoving Coordinates

Computationally, it's convenient to factor out the expansion when tracking particles. We do this by defining a **scale factor**, $a(t)$, which describes the relative size of the universe at any time t. By convention, $a = 1$ today. In the past, a was smaller.

We can now define two types of coordinates:

- **Proper Coordinates (r):** The real, physical distance between two objects that we would measure with a ruler at time t. This distance grows as the universe expands.
- **Comoving Coordinates (x):** The coordinates of an object on the virtual, expanding grid. If an object is moved *only* by the Hubble Flow, its comoving coordinates **do not change**.

The relationship between them is:

$$\mathbf{r}(t) = a(t)\mathbf{x}$$

A particle's true velocity is a combination of the Hubble Flow and its own motion relative to the expanding grid. This local motion, caused by the gravitational pull of nearby structures, is called the **peculiar velocity**, $\mathbf{v}_{\text{pec}}$.

The Equations of Motion

To understand how the equations of motion change, we start with the standard physical law in **proper coordinates**: a particle's physical acceleration is equal to the true gravitational acceleration at its location.

$$\frac{d^2\mathbf{r}}{dt^2} = \mathbf{g}_{\text{proper}}(\mathbf{r})$$

Here, $\mathbf{g}_{\text{proper}}(\mathbf{r})$ is the "real" gravitational acceleration created by the physical distribution of matter in the expanding universe.

Our goal is to rewrite this equation using **comoving coordinates**, $\mathbf{x}$, which are related by $\mathbf{r}(t) = a(t)\mathbf{x}$. After performing the necessary calculus to account for the fact that the scale factor $a(t)$ is changing over time, we arrive at the new equation of motion for a particle's comoving acceleration, $\frac{d^2\mathbf{x}}{dt^2}$:

$$\frac{d^2\mathbf{x}}{dt^2} = \frac{\mathbf{g}_{\text{comoving}}(\mathbf{x})}{a^3} - 2H(t)\frac{d\mathbf{x}}{dt}$$

Let's break down these two terms:

- **Modified Gravity:** The first term, $\frac{\mathbf{g}_{\text{comoving}}(\mathbf{x})}{a^3}$, represents the force of gravity. The term $\mathbf{g}_{\text{comoving}}(\mathbf{x})$ is the gravitational acceleration that our force solvers calculate—it's the acceleration that would exist in a *static* universe if the particles were at their current comoving positions. The division by the scale factor cubed, a^3, is the cosmological correction. As the universe expands by a factor of a, the volume of any given region increases by a^3. This dilutes the physical density of matter as $\rho \propto 1/a^3$. Since gravity is sourced by density, its strength weakens accordingly, and this term captures that effect.
- **Hubble Drag:** The second term, $-2H(t)\frac{d\mathbf{x}}{dt}$, is a new velocity-dependent "friction" term. The term $H(t)$ is the Hubble parameter ($\frac{1}{a}\frac{da}{dt}$), and $\frac{d\mathbf{x}}{dt}$ is the particle's **peculiar velocity** (its local motion relative to the expanding grid). This "Hubble drag" acts to slow down these peculiar velocities. In an expanding universe, a particle's local motion is constantly being damped by the stretching of space itself.

Cosmological Models and the Friedmann Equations

The equation of motion we just derived relies on the scale factor, $a(t)$, and the Hubble parameter, $H(t)$. To integrate the particle trajectories, we need to know how these values evolve. Their behavior is dictated by the laws of General Relativity.

General Relativity is governed by **Einstein's Field Equations**. These equations describe the balance between the geometry of the cosmos and the "stuff" inside it. They are summarized in tensor notation:

$$G_{\mu\nu} + \Lambda g_{\mu\nu} = \frac{8\pi G}{c^4} T_{\mu\nu}$$

The left side represents the canvas of spacetime itself: $G_{\mu\nu}$ measures the geometric curvature, $g_{\mu\nu}$ is the metric defining how distances are measured, and Λ is the cosmological constant representing the inherent energy of the

vacuum. The right side represents the contents: $T_{\mu\nu}$ is the stress-energy tensor, which tallies up all the mass, light, and fluid pressure in a given region. As physicist John Archibald Wheeler summarized: *"Spacetime tells matter how to move; matter tells spacetime how to curve."* However, solving it directly for an irregular, clumpy universe filled with scattered galaxies is impossible.

In the 1920s, Alexander Friedmann simplified Einstein's field equations for a universe that is assumed to be uniform and isotropic on large scales. The result, known as the Friedmann equation, acts as the master blueprint for cosmic expansion.

The geometric cosmological constant (Λ) from Einstein's equations is typically treated as an effective "vacuum energy density" (ρ_Λ). This allows us to group dark energy alongside normal matter and radiation into a single total density term (ρ_{tot}), yielding a unified equation:

$$H(t)^2 = \left(\frac{1}{a}\frac{da}{dt}\right)^2 = \frac{8\pi G}{3}\rho_{tot} - \frac{kc^2}{a^2}$$

Where:

$$\rho_{tot} = \rho_m + \rho_r + \rho_\Lambda$$

$$\rho_\Lambda = \frac{\Lambda c^2}{8\pi G}$$

$$\rho_r = \frac{\epsilon}{c^2}$$

$$\epsilon = \frac{4\sigma}{c}T^4$$

Here, G is the gravitational constant, k is a constant representing the overall geometric curvature of space, and ρ_{tot} is the combined density of matter (ρ_m), radiation (ρ_r), and the inherent vacuum energy of space itself (ρ_Λ). For the radiation component, ϵ represents the thermodynamic energy density of the photon gas filling the universe, determined by the Stefan-Boltzmann constant (σ), the speed of light (c), and the temperature of the Cosmic Microwave Background (T).

Observations of the real universe indicate that our cosmos is geometrically "flat," meaning $k = 0$. In a flat universe, the expansion rate is balanced by the total energy density. This equilibrium point is known as the critical density (ρ_c). Evaluated at any time t, it is defined by the Hubble parameter and the gravitational constant:

$$\rho_c(t) = \frac{3H(t)^2}{8\pi G}$$

Because the critical density acts as the universal balancing point, cosmologists express the composition of the universe using dimensionless density parameters (Ω_m for matter, Ω_r for radiation, and Ω_Λ for dark energy). These are defined as the ratio of a given component's density to the critical density:

$$\Omega_m = \frac{\rho_m}{\rho_c}, \quad \Omega_r = \frac{\rho_r}{\rho_c}, \quad \Omega_\Lambda = \frac{\rho_\Lambda}{\rho_c}$$

In a flat universe, where the total density equals the critical density ($\rho_{tot} = \rho_c$), the sum of these fractions must equal one:

$$\Omega_m + \Omega_r + \Omega_\Lambda = 1$$

For our simulation to be a consistent representation of reality, the mean matter density must equal the matter fraction of this critical density evaluated at the present day:

$$\bar{\rho}_m = \Omega_m \rho_{c,0}$$

While the complete Friedmann framework accounts for the presence of radiation (Ω_r), it is common practice in cosmological simulations to set $\Omega_r = 0$. This approximation is justified by the physics of cosmic expansion: while matter density dilutes as a^{-3} due to the increasing volume of space, radiation density dilutes much faster, scaling as a^{-4}, because the expansion of space also stretches the physical wavelength of the photons, and a photon's energy is inversely proportional to its wavelength ($E \propto 1/\lambda$). Consequently, by the time a standard

N-body simulation initializes (typically around a redshift of $z = 100$), the gravitational influence of radiation has become negligible. Codes often ignore it, assuming that $\Omega_m + \Omega_\Lambda = 1$.

Note that the total matter density parameter, Ω_m, is actually a composite of baryonic matter (Ω_b) and dark matter (Ω_{dm}). Baryonic matter accounts for all the "normal" matter in the universe—such as the cosmic gas, stars, and planets—while dark matter represents the invisible, collisionless mass that provides the dominant gravitational framework for structure formation. This is expressed as $\Omega_m = \Omega_b + \Omega_{dm}$. In a gravity-only N-body simulation, our particles represent this combined total mass. However, when we later introduce hydrodynamics to the code, we must separate these fractions, as the baryonic gas experiences fluid pressure and thermal dynamics, while the dark matter responds exclusively to gravity.

A **cosmological model** is a specific "recipe" of these cosmic ingredients. By defining the total density ρ and its composition (Ω_m, Ω_r and Ω_Λ) and plugging them into the Friedmann equation, we can solve for the trajectory of the expansion.

For the purposes of our simulation, there are two primary models of interest: the classic, matter-dominated model (Einstein-de Sitter) and the modern, dark-energy-driven model (ΛCDM).

An Einstein-de Sitter Universe

The classic model for a matter-dominated universe is the **Einstein-de Sitter (EdS)**. This solution to Einstein's Friedmann equations describes a flat, expanding universe containing only matter ($\Omega_m = 1$) and no dark energy ($\Omega_\Lambda = 0$):

$$H(t)^2 = \frac{8\pi G}{3}\rho(t)$$

In an EdS universe, the expansion of space is constantly being decelerated by the gravitational pull of its own mass. This is described by a power-law relationship between the scale factor, $a(t)$, and cosmic time, t:

$$a(t) \propto t^{2/3}$$

The Hubble parameter also becomes a function of time:

$$H(t) = \frac{1}{a(t)}\frac{da(t)}{dt} = \frac{2}{3t}$$

The ΛCDM Model

In 1998, observations revealed that the expansion of our universe is not slowing down due to gravity; it is accelerating.

The **ΛCDM$ (Lambda Cold Dark Matter)** model introduces a new component: **Dark Energy**, represented by the cosmological constant, Λ. Dark energy acts as a repulsive negative pressure inherent to space itself, pushing the universe apart.

In a flat ΛCDM universe, the total density is made up of matter ($\Omega_m \approx 0.3$) and dark energy ($\Omega_\Lambda \approx 0.7$), such that $\Omega_m + \Omega_\Lambda = 1$. The Friedmann equation expands to include this new term:

$$H(t)^2 = H_0^2\left(\frac{\Omega_m}{a(t)^3} + \Omega_\Lambda\right)$$

Where H_0 is the **Hubble Constant**—the rate at which the universe is expanding right now, that is, the value of the Hubble parameter, $H(t)$, measured at the present.

Notice that the matter density dilutes as the universe expands ($1/a^3$), but the dark energy density (Ω_Λ) remains constant. This creates a cosmic tug-of-war. In the early universe, when $a(t)$ was very small, the dense matter term dominated the Friedmann equation. The universe behaved almost like an EdS model, decelerating as gravity pulled matter together. However, as space expanded and matter diluted, the outward push of dark energy eventually overtook the fading pull of gravity. Today, dark energy dominates, and the expansion is accelerating.

The solution for the scale factor $a(t)$ in a flat ΛCDM universe uses a hyperbolic sine function:

$$a(t) = \left(\frac{\Omega_m}{\Omega_\Lambda}\right)^{1/3} \sinh^{2/3}\left(\frac{3}{2}H_0\sqrt{\Omega_\Lambda}t\right)$$

In a ΛCDM universe, the accelerated stretching of space pulls matter apart faster than gravity can pull it together, eventually halting the hierarchical growth of the cosmos.

General Relativity and Cosmic Expansion

In our simulation model, the scale factor $a(t)$ multiplies the distance between every single coordinate in the comoving grid. When dark matter clumps together to form a dense halo, the grid underneath it continues to expand, and the particles must generate inward "peculiar velocities" to fight against this background stretching and remain gravitationally bound.

However, under Einstein's General Relativity, **space does not expand independently of the matter inside it.** In this section we will understand why our simulation's approach still yields the correct physics.

Let's look at the Einstein Field Equations:

$$G_{\mu\nu} + \Lambda g_{\mu\nu} = \frac{8\pi G}{c^4}T_{\mu\nu}$$

The left side describes how the geometry of spacetime bends and stretches in response to mass and energy. The foundational building block of that geometry is the metric tensor, $g_{\mu\nu}$.

In simple terms, a metric is a mathematical ruler. It defines how distances and time intervals are measured in a curved universe.

In standard Newtonian physics, if we want to measure the spatial distance (ds) between two points, we use the 3D Pythagorean theorem: $ds^2 = dx^2 + dy^2 + dz^2$. However, if those coordinates are sitting in a spacetime warped by gravity or cosmic expansion, that flat-space formula no longer works. The metric tensor ($g_{\mu\nu}$) acts as the correction factor, modifying the Pythagorean theorem to account for the stretching of space and the dilation of time.

This relationship is expressed by multiplying the metric tensor against the differential coordinate steps (dx^μ and dx^ν). Using the Einstein summation convention—where repeating indices imply a sum over all four spacetime dimensions—the universal formula for measuring spacetime intervals is written as:

$$ds^2 = g_{\mu\nu}dx^\mu dx^\nu$$

We can think of $g_{\mu\nu}$ as a 4×4 matrix, and the coordinates as a vector: (dt, dx, dy, dz). In a flat, empty universe (the "Minkowski" metric of Special Relativity), this matrix consists of the speed of light for the time component, 1s for the spatial components, and 0s everywhere else:

$$g_{\mu\nu} = \begin{pmatrix} -c^2 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

If we multiply this matrix through the ds^2 equation, we get the classic flat-space distance formula:

$$ds^2 = -c^2dt^2 + dx^2 + dy^2 + dz^2$$

If we freeze time ($dt = 0$), this recovers the standard 3D Pythagorean theorem.

But when mass or cosmic expansion is introduced, the 1s might be replaced by time-dependent variables, or the 0s might become complex fractions. This alters the underlying geometry, meaning the same coordinate points will yield a different distance, ds.

Solving the Einstein's equations means finding the correct metric—the "ruler"—for a specific arrangement of mass. Different arrangements require different rulers.

The FLRW Metric: A Uniform Universe

To model the global expansion of the universe, cosmologists use the **FLRW metric** (Friedmann–Lemaître–Robertson–Walker). To derive this metric, we assume the universe is a smooth, uniform fluid with no localized clumps, stars, or galaxies. This yields the following geometry:

$$ds^2 = -c^2dt^2 + a(t)^2(dx^2 + dy^2 + dz^2)$$

In this equation, the spatial coordinates (dx, dy, dz) are multiplied by the scale factor $a(t)$. This is the expanding space: because the scale factor depends on time, the physical distance (ds) between two stationary coordinate points increases as the clock ticks. This describes the cosmic voids between distant galaxy clusters.

The Schwarzschild Metric: A Clumped Universe

However, the universe is not a smooth mist. Matter collapses into dense, localized structures like stars, galaxies, and clusters, leaving vacuums between them. If we plug a dense, spherical clump of mass into the Einstein Field Equations surrounded by empty space, the FLRW metric is no longer a valid solution. Instead, the geometry of space is described by the **Schwarzschild metric**:

$$ds^2 = -\left(1 - \frac{2GM}{rc^2}\right)c^2dt^2 + \left(1 - \frac{2GM}{rc^2}\right)^{-1}dr^2 + r^2(d\theta^2 + \sin^2\theta d\phi^2)$$

Note that **there is no time-dependent scale factor $a(t)$ attached to the spatial coordinates** (dr, $d\theta$, $d\phi$). The metric depends solely on the mass (M) and the radius from it (r).

This shows that the space inside a dense, gravitationally bound system is static. The space between the Earth and the Sun, or between stars in the Milky Way, is not expanding. Gravity does not "compensate" for expansion in these regions; the expansion simply does not exist within this localized geometry. Physicists often model the universe as an "Einstein-Straus vacuole"—a "Swiss cheese" universe where the cheese is the smooth, expanding FLRW space, and the holes are static, non-expanding Schwarzschild regions where mass has concentrated.

Validity of the Simulation's Approximation

This brings us to a contradiction. Our model applies the expanding FLRW ruler ($a(t)$) universally. According to General Relativity, that's wrong: we are forcing space to expand inside our simulated galaxy clusters. To keep those clusters from ripping apart, our Newtonian integrator generates artificial "peculiar velocities," making the particles swim inward to fight against a background expansion that shouldn't be there.

To prove that despite violating relativistic geometry, our Newtonian "cheat" actually mimics the geometry of spacetime we'll use **Cosmological Perturbation Theory**.

Cosmological Perturbation Theory

Cosmological Perturbation Theory demonstrates how and when Einstein's equations simplify into our model. To do so, physicists define a metric that represents an expanding universe containing slight localized "wrinkles" or "dimples" of gravity. This is known as the **Conformal Newtonian Gauge**:

$$ds^2 = -\left(1 + \frac{2\Phi}{c^2}\right)c^2dt^2 + a^2(t)\left(1 - \frac{2\Phi}{c^2}\right)(dx^2 + dy^2 + dz^2)$$

Notice the composition of this metric:

1. The global expansion factor $a^2(t)$ still multiplies the spatial coordinates.
2. We have introduced Φ, the local gravitational potential representing the "wrinkles" in spacetime caused by clumps of dark matter.
3. The speed of light, c, is explicitly included to anchor the relativity.

To prove that our N-body simulation is valid, cosmologists insert this "wrinkled" metric into the complex Einstein Field Equations and apply three mathematical limits to mirror the conditions of a standard galaxy cluster:

1. **Weak Gravity:** The local gravitational potential is tiny compared to the speed of light squared ($\Phi \ll c^2$).
2. **Slow Motion:** The peculiar velocities of the particles are much slower than the speed of light ($v \ll c$).

3. **Sub-Horizon Scales:** The comoving size of the simulation domain (L) is much smaller than the observable universe ($L \ll c/H$).

When these three limits are applied, the majority of the relativistic terms cancel out. What remains is an equation defining how the gravitational potential Φ behaves within the expanding grid:

$$\nabla^2\Phi = 4\pi G a^2(\rho - \bar{\rho})$$

This is the cosmological Poisson equation, and it is the foundation of our gravity solver. Notice the right side of the equation: ρ is the actual density in a local region, and $\bar{\rho}$ is the average background density of the universe. The term $(\rho - \bar{\rho})$ proves that in an expanding universe, gravity is only generated by the *overdensity*—the amount of mass that exceeds the background average.

Subtracting this background density is a "trick" known as the "Jeans Swindle": the uniform background mass exerts no net gravity, can be ignored, and we can apply Poisson's equation only to the local density variations.

The perturbation theory we just applied provides the justification for using the Jeans Swindle in our model. In the limit of slow-moving particles and weak gravity, the shifting spacetime of the universe reduces to a modified Newtonian Poisson equation where only the local overdensities drive the structural collapse.

Limitations of the approximation

These three limits define the boundaries of what we can simulate:

- **Breaking the slow-motion limit ($v \ll c$):** If a particle were accelerated to a significant fraction of the speed of light (e.g., material ejected in a relativistic jet), our Newtonian integrator would calculate the wrong trajectory. Under Newtonian mechanics, a continuous force causes continuous acceleration, which would eventually push our simulation particles faster than the speed of light. In reality, relativity dictates that as an object approaches c, its momentum scales non-linearly, requiring infinite energy to accelerate further. Our code lacks the Lorentz transformations required to enforce this cosmic speed limit.
- **Breaking the weak gravity limit ($\Phi \ll c^2$):** If a supermassive black hole formed in our box, the local gravitational potential Φ would approach c^2. Our code operates on the assumption that space is a flat, rigid Cartesian grid, and that gravity is merely a force vector pulling particles through it. In reality, extreme gravity severely warps the geometry of spacetime, creating phenomena like event horizons and severe local time dilation. A simple $1/r^2$ Newtonian force calculation fails to capture the non-Euclidean behavior of matter falling into deeply curved gravitational wells.
- **Breaking the sub-horizon limit ($L \ll c/H$):** If we attempted to simulate a massive domain 10,000 Megaparsecs across, our treatment of large-scale gravity would fail. Our code's Poisson solver calculates the gravitational potential of the entire grid at a single, frozen timestep. It assumes gravity travels at infinite speed. For a 100 Mpc box, the light-travel delay is negligible compared to the slow movement of galaxies. But for a 10,000 Mpc box, our instantaneous calculation would violate causality, allowing a galaxy on one side of the universe to immediately feel the gravitational pull of a cluster on the far opposite side. To prevent faster-than-light gravity over vast cosmic distances, we would have to use relativistic "retarded potentials" that account for the time it takes for gravitational ripples to travel.

Key Literature & Further Reading

Springel, V. (2005). The cosmological simulation code GADGET-2. *Monthly Notices of the Royal Astronomical Society*, 364(4), 1105-1134. Available at: https://arxiv.org/abs/astro-ph/0505010

Green, S. R., & Wald, R. M. (2012). Newtonian and relativistic cosmologies. *Physical Review D*, 85. Available at: https://arxiv.org/pdf/1111.2997

Natural Units

To simplify the implementation, and for numerical stability, it is common practice to work in a system of **natural units**. Any system of units requires three fundamental quantities: Mass, Length, and Time.

It is standard convention in modern cosmological codes to define define the mass and length units by setting the total mass of the system to $M_{total} = 1$ and the comoving side length of the box to $L = 1$. It is also standard to set the present-day scale factor to $a_0 = 1$.

To complete the system, we must define a code unit of time. While we could choose any arbitrary duration (such as one billion physical years), doing so would cause our calculated value for G to change every time we simulate a different cosmology. Instead, we want to define a time unit that absorbs the cosmological parameters, allowing G to remain a permanent, universal constant in our code.

To do this, we base our time unit on the dynamical timescale of a purely matter-dominated universe. In a classic Einstein-de Sitter model, the physical age of the universe is $\frac{2}{3H_0}$. We adapt this timescale to serve as our base code unit of time (t_{unit}) by dividing it by the square root of the matter fraction:

$$t_{unit} = \frac{2}{3H_{0,\text{phys}}\sqrt{\Omega_m}}$$

Because the physical Hubble parameter has units of inverse time ($1/t$), converting it into our code's internal time units ($H_{0,\text{code}} = H_{0,\text{phys}} \times t_{unit}$) forces the numerical value of the expansion rate in our simulation to become exactly:

$$H_0 = \frac{2}{3\sqrt{\Omega_m}}$$

By anchoring our time unit in this way, we have intentionally entangled the internal expansion rate with the matter density. As we will see, this deliberate choice acts as a counterweight in the Friedmann equations, allowing us to lock the strength of gravity to a single, unchanging number across all flat cosmological models.

It may seem counterintuitive that we derived our time unit from an Einstein-de Sitter universe, when modern simulations almost exclusively model ΛCDM universes. However, this is a deliberate computational trick. Dark energy is a perfectly smooth vacuum energy; it does not cluster, and therefore it does not enter the Poisson equation for local gravity. By defining our time unit using the EdS timescale, we isolate the matter density (Ω_m) and absorb it into the time variable, leaving the gravity solver independent of the cosmology. The effects of dark energy are handled elsewhere in the code—specifically, within the calculation of the global background expansion, $a(t)$.

Deriving the Gravitational Constant (G)

To derive the value of the gravitational constant (G) required for our code, we must ensure that the mean density of our simulation grid matches the physical matter density of a flat universe. As established, this physical matter density is a fraction of the critical density: $\bar{\rho}_m = \Omega_m \rho_c$.

In our natural unit system, the present-day Hubble parameter is defined as $H_0 = \frac{2}{3\sqrt{\Omega_m}}$. If we substitute this definition into the physical matter density equation, a cancellation occurs:

$$\bar{\rho}_m = \Omega_m \left(\frac{3\left(\frac{2}{3\sqrt{\Omega_m}}\right)^2}{8\pi G} \right)$$

When we expand the squared Hubble term, the Ω_m in the denominator cancels out the Ω_m scaling the equation:

$$\bar{\rho}_m = \Omega_m \left(\frac{3\left(\frac{4}{9\Omega_m}\right)}{8\pi G} \right) = \frac{12}{72\pi G} = \frac{1}{6\pi G}$$

Now, we look at the computational side. The mean density of our simulation box is defined by its total mass divided by its comoving volume:

$$\bar{\rho}_m = \frac{M_{total}}{L^3}$$

Equating our mean density to the physical matter density gives us the consistency relation for G:

$$\frac{M_{total}}{L^3} = \frac{1}{6\pi G}$$

$$G = \frac{L^3}{6\pi M_{total}}$$

Because we chose $M_{total} = 1$ and $L = 1$ as natural units, this simplifies to a constant:

$$G = \frac{1}{6\pi}$$

The Ω_m parameter drops out of the calculation. Because the code's definition of the expansion rate (H_0) absorbs the matter fraction, the strength of gravity in code units becomes a constant for any flat cosmology.

Hubble: Physical vs. Code Units

Because we intentionally constructed our time unit to force the internal expansion rate to $H_0 = \frac{2}{3\sqrt{\Omega_m}}$, a sharp reader might wonder how we input the *actual* observed expansion rate of the universe. In cosmology, the matter density (Ω_m) and the true physical Hubble constant are independent parameters.

To resolve this, cosmological codes split the concept of the Hubble parameter into two distinct roles:

- **The Physical Hubble Parameter (h):** In astronomy, the real-world Hubble constant is traditionally written as $H_{0,\text{phys}} = 100 \cdot h$ km/s/Mpc, where h is a dimensionless scaling factor (typically around 0.7). This parameter represents the true physical expansion speed. It is used exclusively during the setup and analysis phases to calculate the real physical size of the primordial density fluctuations, set the initial conditions, and translate the final simulation outputs back into standard physical units like Megaparsecs and Kelvin.
- **The Internal Code-Unit Hubble Parameter (H_0):** Once the simulation starts integrating, it uses the derived internal rate ($H_0 = \frac{2}{3\sqrt{\Omega_m}}$) to step time forward, stretch the scale factor $a(t)$, and apply Hubble drag to the particles on the grid.

Translating Natural Units to Physical Reality

Eventually, we'll need to translate our simulation data back into Megaparsecs, Solar Masses, and Gigayears to compare our "mock universe" with actual telescope observations. We must derive the physical value of 1.0 code unit for Length, Mass, Time, and Velocity.

1. The Length Unit ($[L]$) The length unit represents the comoving size of the virtual patch of space we are simulating. If we decide our simulation box represents a region 100 Megaparsecs across, then 1.0 code unit of distance is defined as $L_{box} = 100$ Mpc.

Because this is a comoving length, the coordinate system expands with the universe. The *physical* distance is $a(t) \times L_{box}$.

2. The Mass Unit ($[M]$) Because we defined the total mass of our system as $M_{total} = 1$, one unit of code mass represents the total physical mass of the entire simulated universe.

To calculate this in Solar Masses ($M_\odot$), we must find the total matter density of the present-day universe and multiply it by the comoving volume of our box. The background matter density is the critical density, derived from fundamental constants:

$$G = 6.674 \times 10^{-11}\ \text{m}^3\ \text{kg}^{-1}\ \text{s}^{-2}$$

$$H_0 = 100h\ \text{km}\ \text{s}^{-1}\ \text{Mpc}^{-1}$$

$$\rho_{crit,0} = \frac{3H_0^2}{8\pi G} \approx 2.775 \times 10^{11} h^2\ M_\odot/\text{Mpc}^3$$

It is important to remember that the critical density ($\rho_{crit,0}$) represents the total mass-energy threshold required to make the universe's spatial geometry flat. In a standard ΛCDM universe, matter only makes up a fraction of this total energy budget (typically $\Omega_m \approx 0.3$), with dark energy making up the remainder. In this case we must multiply the critical density by Ω_m to isolate the physical matter density in our simulated box:

$$[M] = \Omega_m \rho_{crit,0} L_{box}^3$$

If a particle in the code has a mass of m_p, its physical mass is $m_p \times [M]$.

3. The Time Unit ($[T]$) To find out how many years pass when our code's internal clock ticks from $t = 0.0$ to $t = 1.0$, we rely on the time unit we defined earlier. Our code unit of time is defined by the physical Hubble parameter and the matter fraction:

$$[T] = \frac{2}{3H_{0,phys}\sqrt{\Omega_m}}$$

The physical Hubble parameter ($H_{0,phys}$) is typically expressed as $100h$ km s^{-1} Mpc^{-1}. To get our time unit in Gigayears (Gyr), we convert $H_{0,phys}$ to $\approx 0.10227h$ Gyr^{-1}. Substituting this into our equation gives us the translation factor:

$$[T] = \frac{2}{3(0.10227h)\sqrt{\Omega_m}} \text{ Gyr}$$

4. The Velocity Unit ($[V]$) Velocity is derived from distance over time:

$$[V] = \frac{[L]}{[T]}$$

Using the derivations above, the Megaparsecs cancel out, yielding a conversion factor to translate code velocities into kilometers per second:

$$[V] = 150 \cdot h \cdot \sqrt{\Omega_m} \cdot L_{box} \text{ km/s}$$

Once these units are established, all other physical quantities in the simulation—such as the internal energy of the gas, pressure, and temperature—can be derived using dimensional analysis.

The Cosmic Timeline

The Scale Factor (a) and Redshift (z)

Astronomers rely on two interlocked variables to track the evolution of the cosmos: the scale factor (a) and cosmological redshift (z).

The Scale Factor (a) As we established, $a(t)$ tracks the relative, physical size of the coordinate grid. By convention, the scale factor today is set to $a = 1$. When the universe was a quarter of its current size, $a = 0.25$. As we run our code forward from the early universe, a ticks upward toward 1.

Cosmological Redshift (z) As a photon travels across the universe, the space it is traveling through is expanding. This expansion stretches the photon's wavelength, shifting its color toward the red end of the spectrum. We call this stretching **Cosmological Redshift**, denoted by the letter z.

The Link Because the stretching of the light is tied to the stretching of space, redshift and the scale factor are inversely related:

$$a = \frac{1}{1+z}$$

In literature we will almost always see time denoted by z:

- $z = 0$**:** Today ($a = 1$).
- $z = 1$**:** The universe was half its current size ($a = 0.5$).
- $z = 9$**:** The universe was 1/10th its current size ($a = 0.1$).
- $z = 49$**:** A typical starting time for generating simulation's initial conditions ($a = 0.02$).
- $z = 1100$**:** The release of the Cosmic Microwave Background ($a \approx 0.0009$).
- $z \to \infty$**:** The Big Bang ($a \to 0$).

The higher the z, the further back in time we are looking, the further away the object is, and the smaller and denser the universe was when that light was emitted.

Eras and epochs

In cosmology, we divide the 13.8-billion-year history of the universe into distinct "eras." These divisions are defined by the Friedmann equation.

$$H^2(a) = H_0^2 \left(\Omega_{r,0}a^{-4} + \Omega_{m,0}a^{-3} + \Omega_{k,0}a^{-2} + \Omega_{\Lambda,0}\right)$$

Whichever component of the universe (radiation, matter, or dark energy) has the highest energy density dictates the expansion rate and how structures form.

For the purposes of cosmological simulations, the universe's history is defined by three major eras:

The Radiation-Dominated Era (The Big Bang to ~50,000 Years)

In the very early universe, space was small, dense, and hot. During this time, the universe's energy budget was dominated by the kinetic energy of photons and relativistic particles (neutrinos).

- **The Physics:** The initial rate of spatial expansion triggered by the Big Bang was still very high. This rapid expansion acted like a cosmic treadmill: space stretched the dark matter particles away from each other much faster than their local gravity could pull them together. Consequently, the local gravitational collapse of dark matter was frozen—a phenomenon known as the **Mészáros effect**. It was only after the universe expanded enough for this "treadmill" to slow down that local gravity took control and began building cosmic structures.
- **Simulation Context:** We rarely simulate this era directly with N-body codes. Instead, its effects are mathematically "baked in" to our initial conditions.
- **The Transition:** As the universe expanded, radiation diluted faster than matter. Around 50,000 years after the Big Bang, the density of radiation dropped below the density of matter, marking a shift in cosmic physics. Shortly after this transition (at about 380,000 years, or a redshift of roughly z = 1100), the universe cooled enough for the first neutral atoms to form, releasing the Cosmic Microwave Background (CMB).

The Matter-Dominated Era (~50,000 Years to ~9.8 Billion Years)

Once the radiation diluted, the gravity of cold dark matter and baryonic gas took control of the energy budget. The global expansion rate dropped low enough that local gravity could finally fight back. Dark matter overdensities decoupled from the expanding background and collapsed inward, initiating the bottom-up, hierarchical structure formation.

- **The Physics:** With radiation pressure gone and the expansion slowing, gravity was finally free to pull matter together. The tiny primordial ripples left over from the Big Bang collapsed into the cosmic web, forming the first stars, galaxies, and galaxy clusters.
- **Simulation Context:** Simulations typically start right near the beginning of this era (e.g., at redshift z = 49). Because dark energy is negligible here, the universe behaves like an Einstein-de Sitter model where the linear growth factor scales with the size of the universe: $D(t) \propto a(t)$.
- **Key Epochs:** Inside this era, hydrodynamic simulations track two vital milestones:
  - *The Dark Ages:* The time before the first stars ignited, when the universe was filled with cold, neutral hydrogen gas.
  - *Cosmic Dawn & The Epoch of Reionization:* The violent period when the first massive stars and quasars (actively feeding supermassive black holes at the centers of infant galaxies) ignited. These objects emitted such intense ultraviolet radiation that they blasted the surrounding cold, neutral hydrogen back into a hot plasma. This process started locally, blowing expanding "bubbles" of ionized gas around the UV emitters. Over hundreds of millions of years, these bubbles grew and merged until the entire intergalactic medium was completely reionized, changing the fluid dynamics and thermal pressure of the universe.

The Dark Energy-Dominated Era (~9.8 Billion Years to Present)

Matter dilutes as the universe expands, but the density of Dark Energy (the cosmological constant, Λ) remains constant. About 9.8 billion years after the Big Bang (around redshift z = 0.3), matter diluted so much that Dark Energy became the dominant force.

- **The Physics:** Dark Energy acts as a repulsive pressure inherent to the vacuum of space. Under its dominance, the expansion of the universe began to accelerate.
- **Simulation Context:** This late-time acceleration stretches the cosmic web apart faster than gravity can pull it together. This marks the epoch where the growth factor $D(t)$ stalls out and stops tracking the scale factor. Large-scale mergers between galaxy clusters become increasingly rare, and the largest structures begin to freeze in place.

A Note on the "Initial Kick" of the Big Bang

General Relativity cannot tell where the initial expansion come from. While modern theoretical physics attempts to explain this with models like **Cosmic Inflation**, for the purposes of computational cosmology we simply accept it as a given initial condition.

Initial Conditions

The outcome of a cosmological simulation depends on its starting point. To accurately model our universe, we must generate a snapshot that captures the primordial density fluctuations that seeded all future cosmic structure.

In the early universe, these density fluctuations were tiny. Because the variations in the gravitational field were so weak and smooth, the initial clustering of matter was **linear**. This means we can fast-forward through the earliest epochs using the **Zel'dovich Approximation** instead of running an N-body solver. This analytical framework can predict how those tiny primordial ripples displaced matter over millions of years.

However, as matter gradually clumps together, local gravity becomes stronger and non-linear. At this point, the linear Zel'dovich approximation breaks down. That breaking point is our simulation's starting line: we use the Zel'dovich approximation to generate the universe's geometry right up to the edge of the linear regime—typically around $z = 49$, roughly 50 million years after the Big Bang. Where this analytical approximation starts to fail, our numerical solvers take over to compute the non-linear birth of galaxies.

The Unperturbed State

To represent the nearly uniform matter distribution of the early universe, we begin by placing particles on a uniform cubic lattice. For a simulation with N particles in a cubic box of side length L, the initial grid position, $\mathbf{x}_{\text{grid}}$, of a particle is determined by its integer indices (i, j, k).

The position is calculated by determining the number of particles per side, N_s, the spacing between them, d, and then placing each particle at the center of its virtual cubic cell.

The number of particles per side is:

$$N_s = N^{1/3}$$

The spacing between grid points is:

$$d = \frac{L}{N_s}$$

The position vector, $\mathbf{x}_{\text{grid}}$, for the particle at indices (i, j, k) is then given by its components:

$$x = \left(i + \frac{1}{2}\right) d$$

$$y = \left(j + \frac{1}{2}\right) d$$

$$z = \left(k + \frac{1}{2}\right) d$$

Where the indices i, j, and k each run from 0 to $N_s - 1$. The addition of $1/2$ ensures that each particle is placed in the center of its cell, rather than at the corner.

The Zel'dovich Approximation

To create the seeds of galaxies and clusters, we must apply small, correlated **perturbations** to the particle lattice. The **Zel'dovich Approximation** generates a smooth displacement field to "nudge" each particle from its perfect grid position.

We use it to generate a single, self-consistent snapshot of the universe at our chosen start time, $t_{initial}$. This snapshot provides both the initial particle positions and their corresponding initial peculiar velocities. From that moment forward, the N-body simulation takes over, calculating the non-linear evolution.

Mathematically, the Zel'dovich Approximation is an application of **first-order Lagrangian perturbation theory (1LPT)**. For higher accuracy more advanced schemes such as **second-order Lagrangian perturbation theory (2LPT)** are often employed, but not covered in this text.

The Growth Factor

In the Zel'dovich Approximation, the displacement field, $\mathbf{\Psi}$, is not constant. As the universe evolves, the tiny initial overdensities attract more matter, causing the perturbations to grow stronger. In the linear regime, the spatial pattern of the displacement field remains fixed, while its amplitude grows over time. This growth is described by a single function of time, the **linear growth factor**, $D(t)$.

The full, time-dependent displacement field can therefore be written as:

$$\mathbf{\Psi}(\mathbf{x}, t) = D(t)\mathbf{\Psi}_0(\mathbf{x})$$

Here, $\mathbf{\Psi}_0(\mathbf{x})$ is the primordial displacement pattern at some reference time (conventionally, today, where $a = 1$ and $D = 1$), representing what the universe would look like today if gravity had remained linear. $D(t)$ scales this entire pattern up or down depending on the cosmic epoch. In a simple Einstein-de Sitter universe model (or a very early ΛCDM, e.g., $a = 0.02$), the growth factor is conveniently proportional to the scale factor, $D(t) \propto a(t)$.

The $D(t)\mathbf{\Psi}_0(\mathbf{x})$ separability means we only need to compute $\mathbf{\Psi}_0(\mathbf{x})$ once, anchoring it to present-day observational parameters (like σ_8). To generate the actual state of the universe at the starting epoch, we scale this pattern backward in time by multiplying it by the appropriate value of $D(t)$.

Generating the Displacement Pattern

The process of generating the spatial pattern, $\mathbf{\Psi}_0(\mathbf{x})$, begins in Fourier space with the **power spectrum**, $P(k)$. The power spectrum specifies the amplitude of density fluctuations (density contrast) at different spatial scales, or wavenumbers (k).

The density contrast field $\delta(\mathbf{x})$ is defined as:

$$\delta(\mathbf{x}) = \frac{\rho(\mathbf{x}) - \bar{\rho}}{\bar{\rho}} = \frac{\rho(\mathbf{x})}{\bar{\rho}} - 1$$

Where $\bar{\rho}$ is the mean density of the universe.

By taking the Fourier transform of the density contrast field, we convert our 3D grid of densities into a 3D grid of waves, denoted as $\tilde{\delta}(\mathbf{k})$.

The **Matter Power Spectrum**, $P(k)$, is the variance of these Fourier amplitudes as a function of their scale. For a given wavenumber k (which corresponds to a physical spatial scale $\lambda = 2\pi/k$), the power spectrum is:

$$P(k) = \langle|\tilde{\delta}(\mathbf{k})|^2\rangle$$

The spectral index, n, defines the *character* of the initial cosmic structure, controlling the balance of power between large-scale (low frequency) and small-scale (high frequency) fluctuations.

The "flat" or **scale-invariant** spectrum is defined by a spectral index of $n = 1$. This case, known as the Harrison-Zel'dovich spectrum, represents a universe where the initial fluctuations have the same strength on all physical scales. All real-world spectra are described by how they "tilt" away from this baseline.

- If $n = 1$, we have a **scale-invariant** spectrum. This is the theoretical "white noise" or baseline for cosmology.
- If $n < 1$, we have a **"red-tilted"** spectrum. There is more power in large-scale (low k) fluctuations and less power in small-scale (high k) ones, compared to the scale-invariant case. This results in a universe with large, gentle, rolling waves of density.
- If $n > 1$, we have a **"blue-tilted"** spectrum. There is less power on large scales and more power on small scales, compared to the scale-invariant case.

Observations of the early universe show that our cosmos has a **"red-tilted"** spectrum, with a primordial spectral index n_s very close to 1 ($n_s \approx 0.96$). This means the initial density ripples were slightly stronger on larger scales than on small ones, providing the seeds for the vast cosmic web we see today.

To generate the displacement pattern, $\mathbf{\Psi}_0(\mathbf{x})$, we use the following steps:

1. **Define the Wavevectors and Physical Scale.** In a 3D Fourier grid, every wave is defined by a **wavevector**, $\mathbf{k} = (k_x, k_y, k_z)$, which points in the direction the wave is traveling. The magnitude of this vector is the **wavenumber**, $k = |\mathbf{k}|$, which represents the wave's spatial frequency. To simulate the

real universe, we must anchor the discrete grid to a physical scale. By defining the comoving size of the simulation box in Megaparsecs (L_{box}), we can calculate the physical wavenumber for every mode:

$$k_{\text{phys}} = \sqrt{k_x^2 + k_y^2 + k_z^2} \cdot \left(\frac{2\pi}{L_{\text{box}}}\right)$$

This physical wavenumber tells the simulation what size structure each wave represents, from massive superclusters to small dwarf galaxies.

2. **Generate and Shape the Random Field.** We start by creating a grid of random complex numbers, $\delta(\mathbf{k})$, that satisfies **conjugate symmetry**, $\delta(\mathbf{k}) = \delta^*(-\mathbf{k})$, to ensure the final field in real space is real-valued. Each Fourier mode is then scaled so its amplitude follows the **ΛCDM power spectrum**. While the primordial universe started with a nearly scale-invariant spectrum (k^{n_s}), the fast rate of cosmic expansion during the radiation-dominated era suppressed the gravitational collapse of small-scale structures. To capture this physics, we multiply the primordial spectrum by a **Cosmological Transfer Function**, $T(k)$. A standard analytical approximation for this is the **BBKS Transfer Function** (Bardeen, Bond, Kaiser, Szalay, 1986)—explained later. The final shaped power spectrum is evaluated as:

$$P(k_{\text{phys}}) = A \cdot k_{\text{phys}}^{n_s} T(k_{\text{phys}})^2$$

Here, A is a master normalization constant that scales the overall strength of the fluctuations. Each random mode is scaled by the square root of this power spectrum: $\delta_\rho(\mathbf{k}) = \delta(\mathbf{k})\sqrt{P(k_{\text{phys}})}$.

3. **Compute the displacement field.** The random field $\delta_\rho(\mathbf{k})$ we just generated is a scalar map of density. However, our goal is the displacement field $\mathbf{\Psi}_0(\mathbf{x})$, which is a vector map telling particles which way to move. To bridge this gap, we must use the physics of gravity. In the Zel'dovich approximation, particles are displaced by falling down the gradient of the gravitational potential created by the density fluctuations. First, we use Poisson's equation to convert the density field into a gravitational potential $\hat{\Phi}(\mathbf{k})$, which requires an inverse Laplacian operation (dividing by $-k^2$). Second, we take the gradient of this potential to find the displacement vector, which in Fourier space corresponds to multiplying by $i\mathbf{k}$. Because this derivative is taking place on our discrete grid, we use the dimensionless internal grid-unit wavevectors ($\mathbf{k}_{\text{grid}}$):

$$\hat{\mathbf{\Psi}}_0(\mathbf{k}) \propto i\mathbf{k}_{\text{grid}} \frac{\delta_\rho(\mathbf{k})}{k_{\text{grid}}^2}$$

4. **Transform back to real space.** Finally, we apply the inverse Fourier transform to recover the displacement pattern in real space:

$$\mathbf{\Psi}_0(\mathbf{x}) = \mathcal{F}^{-1}\{\hat{\mathbf{\Psi}}_0(\mathbf{k})\}$$

Normalizing the Power Spectrum (σ_8) In the previous step, we left the overall amplitude multiplier, A, undefined. To anchor our initial conditions to observational reality, this amplitude is pinned to a standard measured value known as σ_8 (Sigma-8).

σ_8 represents the root-mean-square (RMS, $\sigma_8 = \sqrt{\langle \delta_R(\mathbf{x})^2 \rangle}$) of mass density fluctuations within a sphere of radius 8 Mpc/h in the present-day universe. If we were to drop spheres of this size randomly throughout the cosmos, the mass inside them would vary depending on whether they landed in an empty void or a dense supercluster. σ_8 quantifies this variance—but with a caveat. The real universe today at this scale is mildly non-linear. However, the $\sigma_8 \approx 0.81$ value provided by CMB surveys (like Planck) is a linearly extrapolated parameter. It represents what the variance would be today if structures had grown purely linearly since the Big Bang. Because our initial conditions generator (using the Zel'dovich approximation) relies entirely on linear theory, this standard convention is exactly what we need.

To enforce this in our simulation, we must normalize our theoretical power spectrum so that its mathematical variance at $R = 8$ Mpc/h equals σ_8^2. The variance σ_R^2 of a field smoothed over a physical scale R is found by integrating the power spectrum multiplied by a "window function," $\tilde{W}(kR)$, in Fourier space:

$$\sigma_R^2 = \frac{1}{2\pi^2} \int_0^\infty P_{\text{unnorm}}(k) \tilde{W}^2(kR) k^2 dk$$

For a spherical volume, the appropriate filter is the **spherical top-hat window function**, whose Fourier transform is:

$$\tilde{W}(kR) = \frac{3\left(\sin(kR) - kR\cos(kR)\right)}{(kR)^3}$$

By numerically integrating our unnormalized BBKS power spectrum (where $A = 1$) using this window function at $R = 8h^{-1}$Mpc, we calculate the unnormalized variance. The master normalization constant, A, is then the ratio of the target observational variance to this theoretical variance:

$$A = \frac{\sigma_8^2}{\sigma_{R=8,\text{unnorm}}^2}$$

Applying this constant A ensures our theoretical power spectrum matches the baseline amplitude expected by linear theory for the present day.

The Physics of the Transfer Function: BBKS If the universe contained only dark matter, the primordial power spectrum ($P(k) \propto k^{n_s}$) would remain nearly a straight line across all scales. However, the early universe was dominated by intense radiation. The radiation era altered how structures of different sizes were allowed to grow, a process we capture using the **Cosmological Transfer Function**, $T(k)$.

The shape of $T(k)$ is driven by a cosmic race between gravity and the expansion of space, dictated by the **horizon** (the maximum distance light, and therefore any causal interaction, could have traveled since the Big Bang).

1. **Large Scales (Small k):** These density fluctuations were larger than the cosmic horizon in the early universe. Because they were larger than the distance light could travel, local gravity could not pull them together. However, because overdense regions on these massive scales contained more energy than the cosmic average, their local space expanded slightly slower than the background universe. As the background diluted, these denser regions diluted more slowly, differentiating them more from the background. By the time the cosmic horizon grew large enough to encompass them, the universe had already transitioned into the **matter-dominated era**. Because they grew steadily the entire time, they never lost any growth potential. We therefore define them as our unsuppressed baseline: $T \approx 1$.
2. **Small Scales (Large k):** These tiny fluctuations were small enough to enter the horizon very early on. During this epoch, the energy density of the universe was dominated by radiation, which acted as a brake to the still fast expansion. The dark matter particles were trying to collapse under their own gravity, but because the uniform radiation provided no localized gravitational help, they were entirely on their own. The rapid expansion acted like a cosmic treadmill, pulling the particles apart faster than they could fall together. Consequently, the gravitational collapse of these small-scale density ripples was stalled—a phenomenon known as the **Mészáros effect**. They could only begin to collapse later, once the radiation diluted, the universe transitioned into the matter-dominated era, and the expansion slowed down enough for gravity to finally win.

Calculating the shape of this suppression requires solving complex differential equations tracking dark matter, baryons, photons, and neutrinos. In 1986, Bardeen, Bond, Kaiser, and Szalay (BBKS) published an analytical fitting formula that approximates the result of these calculations.

The BBKS transfer function depends on a scaled wavenumber, q, which adjusts the physical wavenumber k (measured in Mpc^{-1}) based on the density of the universe. For a standard model, it is approximated as:

$$q = \frac{k}{\Omega_m h^2 \exp(-\Omega_b - \sqrt{2h}\Omega_b/\Omega_m)}$$

The BBKS formula itself is:

$$T(q) = \frac{\ln(1 + 2.34q)}{2.34q} \left[1 + 3.89q + (16.1q)^2 + (5.46q)^3 + (6.71q)^4\right]^{-0.25}$$

This equation captures the physics of the early universe:

- As $q \to 0$ (huge cosmic scales), the function approaches 1, preserving the primordial k^{n_s} shape.
- As $q \to \infty$ (tiny cosmic scales), the function falls off proportionally to q^{-2}.

Because the final power spectrum is proportional to $T(k)^2$, the small-scale power drops off by a factor of k^{-4}. This creates a distinct "turnover" peak in the total power spectrum. The location of this peak is a signature of the transition from the radiation era to the matter era (representing the size of the horizon at matter-radiation equality), while the steep drop-off that follows is the signature of the radiation epoch itself.

Applying the Displacements and Velocities

With the spatial pattern $\mathbf{\Psi}_0(\mathbf{x})$ calculated and normalized to the present-day universe ($a = 1$), we can set the initial state of the simulation. Because the normalization is baked into the field, applying it to the early universe relies entirely on cosmological scaling.

The final initial position of each particle is its grid position plus the displacement field, scaled back in time to the starting epoch. In the very early universe (e.g., $a = 0.02$), matter dominates over Dark Energy. Because of this, we can safely approximate that the early growth factor scales directly with the expansion of space, $D(a) \approx a$.

However, because our field $\mathbf{\Psi}_0(\mathbf{x})$ is normalized to the present day ($a = 1$), we cannot simply multiply by a_{initial}. In a ΛCDM universe, late-time Dark Energy stalls structure formation, meaning the present-day growth factor is actually suppressed to a value $g_1 = D(1) < 1$. To correctly unwind this late-time suppression and recover the true early-universe amplitude, we must divide by g_1 (typically evaluated using the Carroll, Press, and Turner 1992 fitting formula):

$$\mathbf{x}_{\text{final}} = \mathbf{x}_{\text{grid}} + \frac{a_{\text{initial}}}{g_1}\mathbf{\Psi}_0(\mathbf{x}_{\text{grid}})$$

where g_1 is defined as:

$$g_1 = \frac{2.5\Omega_m}{\Omega_m^{4/7} - \Omega_\Lambda + \left(1 + \frac{\Omega_m}{2}\right)\left(1 + \frac{\Omega_\Lambda}{70}\right)}$$

Calculating the initial "peculiar" velocity (a particle's motion on top of the Hubble flow) requires more care. The velocity is the time derivative of the comoving displacement, meaning it depends on the rate of change of the growth factor:

$$\mathbf{v}_{\text{pec}} = \left.\frac{dD(t)}{dt}\right|_{t_{\text{initial}}} \mathbf{\Psi}_0(\mathbf{x}_{\text{grid}})$$

In a pure Einstein-de Sitter universe, this derivative simplifies neatly to $\frac{dD}{dt} = H(t)D(t)$. But in a full ΛCDM universe, we must account for the fact that Dark Energy is actively suppressing the rate at which these structures grow. To express this physically, cosmologists define the **Logarithmic Growth Rate**, f:

$$f = \frac{d\ln D}{d\ln a}$$

The foundational approximation for this rate was first derived by P.J.E. Peebles in 1980 as $f \approx \Omega_m^{0.6}$ for a purely matter-dominated universe. However, in a modern flat ΛCDM universe, Dark Energy alters this suppression slightly. Today, the approximation used in cosmological codes is:

$$f \approx \Omega_m(a)^{0.55}$$

By substituting this growth rate into our derivative, we arrive at the generalized cosmological equation for the initial peculiar velocities. It is proportional to the displacement field, scaled by the Hubble parameter, the critical suppression factor f, and the unwound growth factor:

$$\mathbf{v}_{\text{pec}} = H_{\text{initial}} \cdot f \cdot \frac{a_{\text{initial}}}{g_1}\mathbf{\Psi}_0(\mathbf{x}_{\text{grid}})$$

This method produces a self-consistent set of initial conditions for both position and velocity, tailored to the chosen cosmology.

Key Literature & Further Reading

Hahn, O. (2024). Bridging perturbation theory and simulations: initial conditions and fast integrators for cosmological simulations. *SciPost Physics Lecture Notes*. Available at https://scipost.org/preprints/scipost_20 2507_00057v2/

Bardeen, J. M., Bond, J. R., Kaiser, N., & Szalay, A. S. (1986). The statistics of peaks of Gaussian random fields. *The Astrophysical Journal*, 304, 15-61. Available at: https://articles.adsabs.harvard.edu/pdf/1986ApJ...304...15B See Appedix G.

Linder, E. V. (2005). Cosmic growth history and expansion history. *Physical Review D*, 72(4), 043529. Available at: https://arxiv.org/pdf/astro-ph/0507263

Carroll, S. M., Press, W. H., & Turner, E. L. (1992). The cosmological constant. *Annual Review of Astronomy and Astrophysics*, 30, 499-542. Available at: https://www.physics.rutgers.edu/~saurabh/physics690.spring08/Carroll-Press-Turner-1992.pdf

Gravitational softening

In a previous chapter, we introduced gravitational softening as a technique to avoid numerical singularities. According to Newton's law of gravity, $F \propto 1/r^2$, as the distance r between two point masses approaches zero, the force approaches infinity. Softening prevents these forces to cause numerical problems:

$$F = \frac{G m_1 m_2 r}{(r^2 + \epsilon^2)^{3/2}}$$

However, gravitational softening also serves a physical purpose: **enforcing the collisionless limit of Dark Matter.**

Real Dark Matter particles move smoothly through a collective gravitational potential well, behaving as a continuous, collisionless fluid.

However, with unmodified $1/r^2$ gravity, the massive simulation particles (which might represent milions of solar masses each) can violently slingshot off one another in close encounters, transferring kinetic energy in a numerical artifact known as **artificial two-body relaxation**. Over time, this artificial scattering evaporates the cores of simulated Dark Matter halos.

The softening length (ϵ) smears the mass of these particles over a finite spherical volume, simulating soft, overlapping clouds of density. This results in a smooth, collisionless fluid behavior.

Determining the Optimal Softening Length

The softening length must balance two requirements:

- **The Lower Bound (Preventing Relaxation):** If ϵ is too small, the point-mass nature of the macro-particles dominates. Artificial two-body relaxation will transfer kinetic energy into the halo cores, causing them to artificially heat up, and dissolve over time.
- **The Upper Bound (Resolving Forces):** If ϵ is too large, physical forces are artificially smoothed out. The simulation will fail to resolve the gravitational potential wells necessary for matter to collapse.

Power et al. (2003) demonstrated that to prevent strong discreteness effects and minimize integration timesteps, the optimal softening length must scale inversely with the square root of the particle number inside the halo:

$$\epsilon_{opt} = \frac{4 r_{200}}{\sqrt{N_{200}}}$$

Here, r_{200} is the virial radius of the halo, and N_{200} is the number of particles enclosed within it. The *200* suffix refers to the density contrast ($\delta(\mathbf{x}) = \frac{\rho(\mathbf{x}) - \bar{\rho}}{\bar{\rho}}$) of the halo in this study.

However, this formula can't be applied when the sizes and masses of the halos to be formed are not known beforehand. For practical reasons, our code exposes a configurable parameter that multiplies the mean inter-particle distance (d) to define the softening length.

The Physical Softening Cap

Cosmological simulations are typically computed in comoving coordinates to factor out the expansion of the universe. If the softening length is defined as a fixed comoving distance ($\epsilon_{comoving}$), a physical conflict arises when a Dark Matter halo collapses and reaches **Virial Equilibrium**. In the real universe, a virialized halo decouples from the Hubble flow; its *physical* size remains constant over time.

If the simulation maintains a fixed *comoving* softening length, the physical size of that softening length is forced to grow alongside the scale factor (a):

$$\epsilon_{physical} = a \times \epsilon_{comoving}$$

Because $\epsilon_{physical}$ is artificially expanding with the universe, the physical size of the Dark Matter core is dragged outward with it.

Since the mass of the core remains roughly constant, but its physical volume expands proportional to a^3, the physical density of the halo decreases over time:

$$\rho_{physical} = \frac{M_{core}}{a^3 \times \epsilon^3_{comoving}} \propto \frac{1}{a^3}$$

The halo is losing density as it's being artificially inflated by the comoving coordinate system. To allow halos to maintain virial equilibrium, we can implement a **Maximum Physical Softening Cap**. Once the simulation reaches a predefined epoch (for example around $z \approx 2$ or $a \approx 0.33$), the physical softening length is not allowed to grow any further.

To achieve a constant physical softening length, the comoving softening length must be dynamically shrunk proportional to $1/a$. If a_{cap} is the scale factor at which the cap engages, the active comoving softening becomes:

$$\epsilon_{comoving}(a) = \epsilon_{comoving,base} \times \left(\frac{a_{cap}}{a}\right)$$

With this dynamic cap, the halo holds its structural shape against the expanding background universe.

Gravity Validation and Accuracy

Conservation of Energy and Momentum

A meaningful simulation must obey the same laws as the universe it models. A closed system governed by gravity must obey the laws of conservation of energy and momentum.

Note that the rules of **conservation of energy and momentum** are only valid for a closed, non-expanding system. The total energy of a system of particles is *not* a conserved quantity in an expanding universe. The expansion of space itself does work on the system. The peculiar velocities of particles decrease due to Hubble drag, and the potential energy changes as the physical distances between all particles grow.

To use conservation as a test of a simulation's accuracy, we must first disable the cosmic expansion. This is done by setting the scale factor to a constant value, $a(t) = 1$. In this "static box", the simulation becomes a pure gravitational N-body problem.

Conservation of Momentum

In a closed system with no external forces, the total momentum must remain constant. The conservation of momentum is a direct consequence of Newton's third law: for every force $\mathbf{F}_{ij}$ that particle j exerts on particle i, there is an equal and opposite force $\mathbf{F}_{ji} = -\mathbf{F}_{ij}$. When we sum the forces over the entire system, all these internal forces cancel out. If the code correctly implements this symmetry, total momentum will be conserved.

The total momentum $\mathbf{P} = (P_x, P_y, P_z)$ can be computed as:

$$P_x = \sum_i m_i v_{ix}$$

$$P_y = \sum_i m_i v_{iy}$$

$$P_z = \sum_i m_i v_{iz}$$

Where m_i is the mass of particle i, and v_{ix}, v_{iy} and v_{iz} are its velocity components. The values of P_x, P_y and P_z should remain unchanged along the simulation to within machine precision.

The total **angular momentum** must also be conserved. For a system of particles, the total angular momentum is the vector sum of each particle's individual angular momentum, $\mathbf{L} = \mathbf{r} \times \mathbf{p}$.

We can calculate the total angular momentum vector $\mathbf{L} = (L_x, L_y, L_z)$ using the formula:

$$L_x = \sum_i m_i (y_i v_{iz} - z_i v_{iy})$$

$$L_y = \sum_i m_i (z_i v_{ix} - x_i v_{iz})$$

$$L_z = \sum_i m_i (x_i v_{iy} - y_i v_{ix})$$

The values of L_x, L_y, and L_z should each remain unchanged.

Conservation of Energy

For a conservative system like gravity, the total energy—the sum of the **Kinetic Energy (KE)** from motion and the **Potential Energy (PE)** from gravitational attraction—must remain constant.

This validates the entire simulation loop, especially the accuracy of the **time integrator**. While momentum checks the symmetry of the forces at a single instant, energy conservation checks how well the simulation evolves the system from one state to the next.

Because the simulation moves in discrete time steps, we don't expect the energy to be conserved perfectly. Instead, the *behavior* of the error tells us if the integrator is working correctly:

- **A good (symplectic) integrator** will produce an energy error that **oscillates** around the initial value. The energy will wobble, but it will not show a long-term trend of increasing or decreasing.
- **A bad or inconsistent integrator** will cause the energy to **drift** steadily over time, indicating a systematic error that is continuously adding or removing energy from the system.

To calculate the total energy, $E(0) = KE + PE$ we just add the kinetic and potential energies as described below.

- **Kinetic Energy (KE):** The total kinetic energy is the sum of the kinetic energy of every particle in the system.

$$KE = \sum_{i=1}^{N} \frac{1}{2} m_i v_i^2$$

  Where m_i is the mass of particle i and v_i is its speed ($v_i^2 = v_{ix}^2 + v_{iy}^2 + v_{iz}^2$).

- **Potential Energy (PE):** The total potential energy is the sum of the potential energy for every *unique pair* of particles. We can measure the simulation's accuracy by checking how well it conserves the total energy of the **ideal, unsoftened system**.

$$PE = \sum_{i=1}^{N} \sum_{j>i}^{N} \frac{-G m_i m_j}{r_{ij}}$$

  Where r_{ij} is the distance between particles i and j.

We can periodically recalculate this total energy, $E(t)$, as the simulation runs and monitor the relative error over time: $\frac{E(t)-E(0)}{E(0)}$.

A small, bounded oscillation in this value is the signature of a healthy simulation. A consistent drift points to an issue that needs to be fixed.

The Two-Body Problem and Kepler's Laws

To know if the trajectories of our particles are correct, we need a problem with a known solution that we can compare our simulation against. A good example is the **two-body problem**, whose solution is described by **Kepler's Laws of Planetary Motion**. The setup requires two bodies:

1. **The "Star":** Create one particle with a very large mass and place it near the center of the simulation box. Give it zero initial velocity to keep it mostly stationary.
2. **The "Planet":** Create a second particle with a much smaller mass and place it some distance away from the star, for example, along the x-axis. Give the planet an initial velocity in the y-direction. A specific value for the magnitude of this velocity will produce a circular orbit, while slightly different values will produce stable elliptical orbits.

After running the simulation, we can check the planet's trajectory against Kepler's predictions:

1. Is the Orbit a Closed Ellipse? (Kepler's First Law) The planet should trace a stable, closed ellipse with the star at one of the foci. Common failure modes are:

- **Energy Drift:** The orbit spirals inwards or outwards, indicating a non-symplectic integrator or a bug.
- **Unphysical Precession:** The ellipse itself rotates over time. A large, rapid precession is a sign of numerical inaccuracy. A stable, non-precessing ellipse is a sign of a healthy integrator.

2. Does it Speed Up and Slow Down Correctly? (Kepler's Second Law) This law states that a planet sweeps out equal areas in equal times, which is a consequence of angular momentum conservation. This means

the planet must move **fastest** when it is closest to the star (perihelion) and **slowest** when it is farthest away (aphelion).

3. Does it Obey the Law of Periods? (Kepler's Third Law) The law states that the square of the orbital period (P) is proportional to the cube of the orbit's semi-major axis (a), or $P^2 \propto a^3$. The time it takes the simulated planet to complete one full orbit and the size of its orbit must satisfy this relationship.

Sources of Error

A perfect simulation is impossible. The goal is not to eliminate all errors but to understand where they come from, control them, and ensure they are small enough that the results are physically meaningful. In a P^{3}M simulation, the errors arise from the approximations we make to turn the continuous problem of gravity into a discrete problem a computer can solve.

Time Discretization Error

Physics happens continuously, but a simulation must leap forward in discrete chunks of time, Δt. The time integrator works by assuming that the acceleration changes in a simple, predictable way during that small leap.

However, the true acceleration, especially during a close encounter, can be more complex. The difference between the integrator's assumed path and the true physical path is called **truncation error**. With a second-oder integration method, this error scales with the square of the time step ($O(\Delta t^2)$) and is a primary contributor to the oscillations in the total energy.

Softening Bias

To prevent infinite forces when particles get too close, we modify the force law with a softening parameter, ϵ.

$$F = \frac{Gm_1m_2}{r^2 + \epsilon^2}$$

This is not a numerical error but a **physical modeling error**. We are knowingly simulating a slightly different, "softer" version of gravity. This error is most significant at very short distances ($r \lesssim \epsilon$) and prevents the formation of very "hard," dense structures. We accept this trade-off because it grants us numerical stability, but the simulation becomes blind to any physics occurring at scales smaller than the softening length.

Spatial Discretization Error

The Particle-Mesh method gains its speed by calculating long-range forces on a grid. This introduces several errors related to the grid's finite resolution.

- **Aliasing:** The grid can only represent waves with a wavelength larger than about two cell sizes. When two particles get closer than this, their sharp, high-frequency density spike is misinterpreted by the grid as a smooth, low-frequency wave. This is **aliasing**, and it is the primary source of inaccuracy in the PM force, making it "blurry" and even repulsive at short distances.
- **Interpolation Error:** The schemes used to assign mass to the grid and interpolate forces back (like NGP or CIC) are approximations that contribute to the total error.
- **Finite Difference Error:** The force on the grid is calculated using a finite difference approximation of the gradient. This is not a perfect derivative and adds a small amount of error.

Key Literature & Further Reading
Dehnen, W., & Read, J. I. (2011). *N-body simulations of gravitational dynamics.* arXiv:1105.1082. Available at https://arxiv.org/abs/1105.1082.

Hydrodynamics

To simulate the formation of the luminous structures—stars, galaxies, and galaxy clusters—a simulation must include the physics of **baryonic matter**. In cosmology, "baryons" is the term for all normal matter, which primarily exists as a vast, diffuse **gas**.

Unlike dark matter, gas is not collisionless. Its particles (atoms and ions) interact with each other, giving rise to familiar fluid properties like **pressure** and **temperature**.

It is natural to wonder how a medium so empty—often containing just a single atom in a volume the size of a small room—can behave like a fluid and push back against gravity. The answer lies in the sheer scale of the universe and the physics of plasmas.

In the cosmic voids, the primordial gas is very cold and diffuse. However, because cosmological structures are millions of light-years across, the distance an atom travels before colliding with another is still microscopic compared to the size of the system. On these vast scales, even a near-vacuum experiences enough microscopic collisions to possess macroscopic fluid properties like pressure and density.

The physics changes when this cold gas falls into the gravity of a dark matter halo. As the gas crashes into the halo at hypersonic speeds, shockwaves heat it to millions of degrees, turning it into a highly energetic plasma. In this state, the charged ions and electrons do not need to collide like billiard balls to interact. Instead, they repel each other across vast distances via electromagnetic forces and are threaded together by large-scale magnetic fields. These long-range interactions ensure that even inside the hottest, most violent galaxy clusters, the diffuse matter continues to act as a cohesive fluid.

The Hybrid Simulation Approach

To model this continuous behavior a simulation must solve the laws of **hydrodynamics**.

In a **hybrid code**, the universe is modeled using two distinctly different computational perspectives. The dark matter is treated as a collection of **Lagrangian** N-body particles (tracking individual masses as they move freely through space). Conversely, the gas is treated using an **Eulerian** approach (tracking the continuous properties of a fluid as it flows past stationary points).

To seamlessly link these two components, the Eulerian fluid dynamics are solved on the same grid used by the Particle-Mesh gravity solver. The two components communicate via the force of gravity, which is sourced by their combined density on this shared grid.

The Euler Equations

The motion of a simple, non-viscous gas (a good approximation for most cosmic fluids), is governed by a set of conservation laws known as the **Euler Equations**. These equations describe how the density, momentum, and energy of the gas change over time.

The full set of cosmological hydrodynamic equations couples the conservation laws of fluid dynamics with the source terms from gravity in an expanding universe. The equation for the vector of conserved quantities, $\mathbf{U}$, can be written as:

$$\frac{\partial \mathbf{U}}{\partial t} + \nabla \cdot \mathbf{F}(\mathbf{U}) = \mathbf{S}(\mathbf{U})$$

Where:

- $\mathbf{U}$ is the vector of conserved state variables.
- $\nabla \cdot \mathbf{F}(\mathbf{U})$ is the "flux" term, which describes how quantities move due to pressure and advection (the fluid flowing).
- $\mathbf{S}(\mathbf{U})$ is the "source" term, which describes changes due to external forces, like gravity ($\rho\mathbf{g}$).

Standard fluid dynamics relies only on three fundamental variables (mass, momentum, and total energy). However, in cosmological flows, gas moves at supersonic speeds. In these conditions, a grid cell's kinetic energy becomes vastly larger than its thermal energy. Trying to numerically calculate the tiny internal temperature by subtracting a massive kinetic energy from a massive total energy leads to catastrophic floating-point errors.

To solve this, cosmological codes use the **Dual Energy Formalism** (which will be explored in detail in a later section), explicitly tracking a fourth variable: **internal energy density (ie)**. Therefore, our expanded state vector becomes a four-element array:

$$\mathbf{U} = [\rho, \rho\mathbf{v}, E, ie]$$

Expanding this into the continuous differential equation yields the complete model:

$$\frac{\partial}{\partial t}\begin{bmatrix}\rho\\ \rho\mathbf{v}\\ E\\ ie\end{bmatrix}+\nabla\cdot\begin{bmatrix}\rho\mathbf{v}\\ \rho\mathbf{v}\otimes\mathbf{v}+P\mathbf{I}\\ (E+P)\mathbf{v}\\ ie\mathbf{v}\end{bmatrix}=\begin{bmatrix}0\\ \rho\mathbf{g}\\ \rho\mathbf{v}\cdot\mathbf{g}\\ -P(\nabla\cdot\mathbf{v})\end{bmatrix}$$

Here is the breakdown of what each row represents:

- **Row 1 (Conservation of Mass):**
  - **Flux:** $\rho\mathbf{v}$ is the advection (flow) of mass.
  - **Source:** 0, because the universe does not spontaneously create or destroy mass.
- **Row 2 (Conservation of Momentum):**
  - **Flux:** $\rho\mathbf{v}\otimes\mathbf{v}+P\mathbf{I}$. This combines the momentum advection tensor ($\rho\mathbf{v}\otimes\mathbf{v}$) with the thermal pressure (P). The advection tensor is a 3x3 matrix that tracks how each directional component of momentum is transported along every spatial axis. The $\mathbf{I}$ is the identity matrix, ensuring the pressure gradient acts perpendicular to the cell faces—this is the force that allows the gas to resist gravitational collapse.
  - **Source:** $\rho\mathbf{g}$, the external acceleration due to gravity pulling on the gas. This links the fluid to the Particle-Mesh gravity solver.
- **Row 3 (Conservation of Total Energy):**
  - **Flux:** $(E+P)\mathbf{v}$. This is the total energy sliding along with the gas ($E\mathbf{v}$), plus the mechanical PdV work being done by the fluid as it compresses and expands against its neighbors ($P\mathbf{v}$).
  - **Source:** $\rho\mathbf{v}\cdot\mathbf{g}$. This is the mechanical work done *by gravity*. As gas falls into a dark matter halo (velocity aligns with gravity), gravity does work on the gas, adding to its total energy.
- **Row 4 (Internal Energy / Dual Energy Formalism):**
  - **Flux:** $ie\mathbf{v}$. This is the pure advection of thermal energy (hot gas flowing from one place to another).
  - **Source:** $-P(\nabla\cdot\mathbf{v})$. This term represents the macroscopic PdV **work** (Pressure × change in Volume). In Eulerian fluid dynamics, the divergence of velocity ($\nabla\cdot\mathbf{v}$) is a direct measurement of the fractional rate of change of a fluid parcel's volume (which is physically equivalent to a change in its density). In the standard Total Energy equation (Row 3), thermodynamic heating and cooling are handled by the pressure flux ($P\mathbf{v}$). However, because we isolated the internal energy from the main equation to avoid floating-point errors, we must explicitly add the PdV work back in as a manual source term:
    - **Adiabatic Compression:** As gas falls into a dark matter halo and converges ($\nabla\cdot\mathbf{v}<0$), its specific volume shrinks and its density spikes. The math yields a positive source term, meaning the environment does work *on* the gas, heating it.
    - **Adiabatic Expansion:** As gas flows outward into a cosmic void ($\nabla\cdot\mathbf{v}>0$), its volume expands and its density drops. The math yields a negative source term, meaning the gas spends its own internal energy to push outward, causing it to cool.

Finally, to calculate the pressure (P) required for these equations, we relate it to the density and internal energy using an **equation of state**. For a simple, ideal gas, this equation is:

$$P=(\gamma-1)ie$$

Here, γ is the adiabatic index, a constant which is typically 5/3 for a monatomic gas like the hydrogen and helium that fill the cosmos. It describes how much the pressure of a gas responds to a change in volume during an adiabatic process—when the gas is compressed or expands so quickly that it doesn't have time to exchange heat with its surroundings. A higher γ means the pressure rises more sharply for the same amount of compression.

An Operator-Split Hydro-Solver

Solving this equation all at once is difficult. In a real cosmological fluid, multiple physical processes are happening simultaneously: gas is flowing through space, pressure waves are expanding, gravity is accelerating the mass, and thermal energy is radiating away. Mathematically, these distinct physical phenomena are represented by different "operators" (the flux and source terms in the differential equation) that are tangled together. Solving them concurrently in a single, massive calculation is difficult.

A common and effective technique called **operator splitting** breaks the problem into simpler, sequential steps. Instead of attempting to solve everything at once, we temporarily decouple the physics.

For a tiny slice of time (Δt), we allow the gas to *only* flow across the grid and respond to its own internal pressure, ignoring external forces. Once we calculate the new state of the fluid, we freeze its motion. Then, assuming that same time slice (Δt) has passed, we apply *only* the pull of gravity to this updated state.

By taking turns applying each physical process sequentially, the combined result approximates the true physics, provided the time step Δt is small enough. This "divide and conquer" approach also allows us to use the best, specialized mathematical solver for each individual physical process without them interfering with one another.

Here is the step-by-step process to advance the gas grid from time t to $t + \Delta t$ following the **Kick-Drift-Kick (KDK)** structure.

Step 1: Convert to Primitive Variables

(A quick note on notation: In the generalized Euler equations above, we used $\mathbf{S}(\mathbf{U})$ *to represent the full "Source Vector." However, from this point forward, we will adopt the standard simulation nomenclature where the vector* $\mathbf{S}$ *(and its components* S_x, S_y, S_z*) refers specifically to the* ***momentum density****,* $\rho\mathbf{v}$*.)*

The solver begins with the grid of **conserved variables** (ρ, $S_x = \rho v_x$, $S_y = \rho v_y$, E) from the previous step. To calculate fluxes and forces, it first computes the **primitive variables** (velocity $\mathbf{v}$ and pressure P):

$$v_x = \frac{S_x}{\rho} \quad , \quad v_y = \frac{S_y}{\rho}$$

$$P = (\gamma - 1)\left(E - \frac{1}{2}\rho|\mathbf{v}|^2\right)$$

Step 2: Gravity Half-Step (Kick)

First, the external forces from gravity are applied for half a time step, $\Delta t/2$. The gravitational acceleration field, $\mathbf{g}$, is provided by the Particle-Mesh solver. The acceleration field must be derived from the **total matter density**—the sum of the dark matter density (from the N-body particles) and the gas density (from the hydro grid). This step updates the momentum and total energy of the gas:

$$\mathbf{S}(t + \tfrac{1}{2}\Delta t) = \mathbf{S}(t) + (\rho\mathbf{g})\frac{\Delta t}{2}$$

$$E(t + \tfrac{1}{2}\Delta t) = E(t) + (\mathbf{v} \cdot \rho\mathbf{g})\frac{\Delta t}{2}$$

The energy is updated by the **power density** (Force Density · Velocity, or $\mathbf{v} \cdot \rho\mathbf{g}$), which is the rate at which the gravitational field does work on the gas.

Step 3: Hydrodynamic Full-Step (Drift)

With the gravitational source terms applied, we can solve the pure hydrodynamic equations (the "flux" part) for a full time step, Δt. To calculate how much gas flows from one cell to the next, we must first estimate the fluid's properties at the boundary between them—a process known as **spatial reconstruction**.

If we use a second-order spatial reconstruction scheme (like MUSCL, detailed in the next section), we must match the spatial accuracy with **second-order time integration**. A robust method for finite-volume hydrodynamics is the **Strong Stability Preserving Runge-Kutta (SSP-RK2)** scheme.

Because operator splitting temporarily isolates the fluid's motion (the "Drift" stage) into a purely hydrodynamic problem, we are free to solve it using its own specialized mathematical engine. Therefore, nested inside the global KDK loop, this step utilizes the SSP-RK2 scheme. KDK orchestrates the overall physics, while SSP-RK2 acts as the high-precision sub-engine that actually pushes the gas across the grid.

In multi-stage Runge-Kutta methods, it is standard to denote the fluid state at the current time t using the step index n (written as $\mathbf{U}^n$), and the final state at $t + \Delta t$ as $\mathbf{U}^{n+1}$. The SSP-RK2 method achieves second-order accuracy by breaking the fluid advection into two intermediate mathematical stages (denoted by parenthetical superscripts) and averaging the results:

1. The Predictor Stage: The solver takes a full Euler step using the current fluid state ($\mathbf{U}^n$) to calculate the fluxes. This generates an intermediate, predicted state:

$$\mathbf{U}^{(1)} = \mathbf{U}^n + \Delta t \cdot \mathbf{L}(\mathbf{U}^n)$$

Where $\mathbf{L}(\mathbf{U})$ acts as the **discrete spatial operator**. Earlier in the chapter, the fluid's motion was described by the continuous calculus term $\nabla \cdot \mathbf{F}(\mathbf{U})$. Because our simulation cannot perform infinite calculus, we must approximate this physics on a discrete grid. $\mathbf{L}(\mathbf{U})$ serves as the numerical approximation of that physics on that grid. Mathematically, it is defined as:

$$\mathbf{L}(\mathbf{U}) \approx -\nabla \cdot \mathbf{F}(\mathbf{U}) + \mathbf{S}_{\text{internal}}$$

Expanded:

$$\mathbf{L}(\mathbf{U}^n) = \underbrace{\begin{bmatrix} -\nabla \cdot (\rho \mathbf{v})^n \\ -\nabla \cdot (\rho \mathbf{v} \otimes \mathbf{v} + P\mathbf{I})^n \\ -\nabla \cdot ((E+P)\mathbf{v})^n \\ -\nabla \cdot (ie\mathbf{v})^n \end{bmatrix}}_{-\nabla \cdot \mathbf{F}(\mathbf{U}^n)} + \underbrace{\begin{bmatrix} 0 \\ \mathbf{0} \\ 0 \\ -P^n(\nabla \cdot \mathbf{v}^n) \end{bmatrix}}_{\mathbf{S}_{\text{internal}}}$$

Here, the negative sign appears because divergence ($\nabla \cdot \mathbf{F}$) measures the net outflow, whereas $\mathbf{L}(\mathbf{U})$ calculates the net influx across the cell boundaries. Instead of evaluating a continuous derivative, this discrete divergence for a given cell i is calculated as the algebraic difference between the fluxes crossing its right and left boundaries, divided by the cell width (Δx):

$$\nabla \cdot \mathbf{F}(\mathbf{U}) \approx \frac{\mathbf{F}_{i+1/2} - \mathbf{F}_{i-1/2}}{\Delta x}$$

These specific boundary fluxes ($\mathbf{F}_{i+1/2}$ and $\mathbf{F}_{i-1/2}$) are exactly what the HLLC Riemann solver (explained later) will compute.

The $\mathbf{S}_{\text{internal}}$ term accounts for internal thermodynamics, specifically the PdV work required by the Dual Energy Formalism. Because the Dual Energy Formalism tracks internal energy separately from total energy, we must explicitly add this PdV source term so that the gas physically heats up when compressed by gravity and cools down when expanding into voids.

Noticeably absent is the external gravity source term ($\rho\mathbf{g}$); because we are using an operator-split Kick-Drift-Kick architecture, gravity is handled entirely separately during the half-steps.

To see exactly how this discrete spatial operator updates the fluid, we can expand the equation $\mathbf{U}^{(1)} = \mathbf{U}^n + \Delta t \cdot \mathbf{L}(\mathbf{U}^n)$ into its full column-vector form:

$$\begin{bmatrix} \rho^{(1)} \\ (\rho\mathbf{v})^{(1)} \\ E^{(1)} \\ ie^{(1)} \end{bmatrix} = \begin{bmatrix} \rho^n \\ (\rho\mathbf{v})^n \\ E^n \\ ie^n \end{bmatrix} + \Delta t \cdot \begin{bmatrix} -\nabla \cdot (\rho\mathbf{v})^n \\ -\nabla \cdot (\rho\mathbf{v} \otimes \mathbf{v} + P\mathbf{I})^n \\ -\nabla \cdot ((E+P)\mathbf{v})^n \\ -\nabla \cdot (ie\mathbf{v})^n - P^n(\nabla \cdot \mathbf{v}^n) \end{bmatrix}$$

2. The Corrector Stage: The solver calculates an entirely new set of fluxes based on the *intermediate* state ($\mathbf{U}^{(1)}$). It takes another full Euler step from this intermediate state to generate a second state:

$$\mathbf{U}^{(2)} = \mathbf{U}^{(1)} + \Delta t \cdot \mathbf{L}(\mathbf{U}^{(1)})$$

3. Final Averaging: Finally, the true state at the next time step ($\mathbf{U}^{n+1}$) is found by averaging the original state and the twice-stepped state:

$$\mathbf{U}^{n+1} = \frac{1}{2}\mathbf{U}^n + \frac{1}{2}\mathbf{U}^{(2)}$$

This staggered, averaging approach allows the gas to flow across the grid with true second-order accuracy in both space and time. While calculating the fluxes twice per cycle essentially doubles the computational cost of the hydrodynamics, the dramatic improvement in shock resolution and overall stability makes it a strict requirement for modern cosmological simulations.

Step 4: Second Gravity Half-Step (Kick)

Finally, the gravitational source terms are applied again for the second half of the time step, $\Delta t/2$, using the updated values from the "Drift" step:

$$\mathbf{S}(t+\Delta t) = \mathbf{S}(t+\tfrac{1}{2}\Delta t) + (\rho\mathbf{g})\frac{\Delta t}{2}$$

$$E(t+\Delta t) = E(t+\tfrac{1}{2}\Delta t) + (\mathbf{v}\cdot\rho\mathbf{g})\frac{\Delta t}{2}$$

At the end of this sequence, the conserved variables of the gas grid have been fully advanced to time $t+\Delta t$, accounting for both the internal fluid dynamics and the external force of gravity.

Spatial Reconstruction and the MUSCL Scheme

In a basic finite-volume grid, the simplest way to determine the state of the fluid at the interface between two cells is to assume the fluid properties (density, velocity, pressure, calso called **primitive variables**) are constant across the entire cell. This is known as **Piecewise Constant** reconstruction.

While computationally cheap, piecewise constant reconstruction is highly diffusive. It acts like a blur filter, smearing out sharp shock waves across many grid cells. To capture the sharp, violent shocks of a cosmological simulation, we need a higher-order approach.

The **MUSCL (Monotonic Upstream-centered Scheme for Conservation Laws)** scheme upgrades our spatial resolution to second-order by replacing the flat constant states with **Piecewise Linear** slopes. Instead of assuming a fluid variable q (which represents $\rho, \mathbf{v}$, or P) is flat inside cell i, we calculate a gradient (slope) based on its neighbors, $i-1$ and $i+1$. We then use this slope, Δq_i, to linearly extrapolate the fluid's properties from the cell center to the left and right interfaces of the cell.

Mathematically, to find the fluid states on the exact left (L) and right (R) sides of the interface located between cell i and cell $i+1$, we extrapolate outwards from the cell centers:

$$q_{L,i+1/2} = q_i + \frac{1}{2}\Delta q_i$$

$$q_{R,i+1/2} = q_{i+1} - \frac{1}{2}\Delta q_{i+1}$$

Where:

$$\Delta q_i = \frac{q_{i+1} - q_{i-1}}{2}$$

However, a naive linear extrapolation creates a disastrous numerical artifact at shock fronts. When a steep slope tries to extrapolate across a discontinuous shock, it inevitably overshoots the true value, causing wild, unphysical oscillations (ringing) known as the Gibbs phenomenon.

To safely use linear reconstruction, we must introduce a **Slope Limiter**. A slope limiter acts as a localized safety switch. We first calculate two separate slopes for cell i: the backward difference (looking left) and the forward difference (looking right):

$$\Delta q_{\text{backward}} = q_i - q_{i-1}$$

$$\Delta q_{\text{forward}} = q_{i+1} - q_i$$

- If the fluid is smooth, these two slopes will be similar, and the limiter allows the second-order linear slope.
- If a shock is present, the slopes will violently disagree or change signs. The limiter detects this extremum and immediately forces the slope Δq_i to zero, temporarily dropping the local reconstruction back to a safe, flat first-order state just for that specific shock front.

A common and highly robust choice is the **Minmod limiter**. It compares the forward and backward slopes and selects the one with the smallest magnitude if they point in the same direction, returning zero if they point in opposite directions:

$$\text{minmod}(a,b) = \begin{cases} a & \text{if } |a| < |b| \text{ and } ab > 0 \\ b & \text{if } |b| < |a| \text{ and } ab > 0 \\ 0 & \text{if } ab \leq 0 \end{cases}$$

By setting our cell slope to $\Delta q_i = \text{minmod}(\Delta q_{\text{backward}}, \Delta q_{\text{forward}})$, we guarantee that the reconstructed values at the interfaces will never create new, artificial extrema. By applying this MUSCL reconstruction to the primitive variables $(\rho, \mathbf{v}, P)$, the simulation can resolve sharp shock fronts without sacrificing stability.

The HLLC Riemann Solver

Once the fluid states have been extrapolated to the left ($\mathbf{U}_L$) and right ($\mathbf{U}_R$) sides of an interface, the code must determine the flux of mass, momentum, and energy passing through it ($\mathbf{F}(\mathbf{U})$). This is done using an **approximate Riemann solver**.

When two different fluid states collide at an interface, they spawn a complex fan of waves propagating outwards. The Harten-Lax-van Leer (HLL) solver is a classic approximation that assumes this complex interaction can be simplified into just two waves: the fastest wave moving left and the fastest wave moving right. These wave speeds are denoted by the scalar variables S_L and S_R (where S stands for "Signal velocity", which should not be confused with the momentum density vector $\mathbf{S}$ used earlier in the macroscopic solver).

While incredibly stable, the standard HLL solver has a fatal flaw: it ignores the middle of the wave structure. Specifically, it completely misses the **Contact Discontinuity**—a distinct middle boundary where fluid density and temperature jump abruptly. Across this boundary, pressure and velocity remain perfectly continuous, as the faster acoustic waves in the fan have already propagated outward and equalized the mechanical forces. Because HLL averages over this middle region, it heavily smears out contact discontinuities and shear flows, which are vital for resolving the intricate gas filaments of the cosmic web.

To fix this, modern cosmological codes use the **HLLC** solver (where "C" stands for Contact). The HLLC solver restores the missing middle physics by introducing a third wave speed, S_*, which represents the speed of the contact discontinuity. As the left and right waves move outward from the interface over the time step, they sweep out an expanding region of interacting gas. The S_* wave divides this middle region into two distinct "star" states: $\mathbf{U}_L^*$ and $\mathbf{U}_R^*$.

The calculation of the HLLC flux at the interface ($\mathbf{F}_{HLLC}$) proceeds as follows:

1. Estimate the Signal Velocities First, the local sound speeds for the left ($c_{s,L}$) and right ($c_{s,R}$) states are calculated using the adiabatic index γ, pressure P, and density ρ:

$$c_s = \sqrt{\frac{\gamma P}{\rho}}$$

These sound speeds, along with the normal velocities of the fluid ($v_{n,L}$ and $v_{n,R}$), are used to estimate the outermost wave speeds bounding the Riemann problem. To guarantee numerical stability, the solver must encompass the fastest possible waves traveling in either direction. A robust and standard estimate achieves this by taking the minimum and maximum possible signal velocities across the interface:

$$S_L = \min(v_{n,L} - c_{s,L}, v_{n,R} - c_{s,R})$$

$$S_R = \max(v_{n,L} + c_{s,L}, v_{n,R} + c_{s,R})$$

2. Calculate the Contact Wave Speed (S_*) The speed of the middle contact wave is derived directly from the conservation of momentum across the interface:

$$S_* = \frac{P_R - P_L + \rho_L v_{n,L}(S_L - v_{n,L}) - \rho_R v_{n,R}(S_R - v_{n,R})}{\rho_L(S_L - v_{n,L}) - \rho_R(S_R - v_{n,R})}$$

3. Compute the Star States Using the contact wave speed S_*, the solver can calculate the exact conserved variables for the intermediate "star" regions ($\mathbf{U}_L^*$ and $\mathbf{U}_R^*$) wedged between the shock fronts and the contact discontinuity.

For either the Left or Right state (denoted by the subscript K, where $K \in \{L, R\}$), the density of the star state is determined by mass conservation across the outermost wave:

$$\rho_K^* = \rho_K \left(\frac{S_K - v_{n,K}}{S_K - S_*} \right)$$

Next, we must determine the velocities and energy of these star states by looking at the physics of the contact discontinuity itself. This middle wave (S_*) is the literal boundary where the two original gas masses meet.

Because they are pushing against each other with matching pressure and velocity, they do not mix; the interface acts as an impermeable wall that moves with the fluid.

Since no mass actually flows across this boundary, any property tied strictly to the fluid mass—specifically the transverse velocities (v_{t1}, v_{t2}) and the specific internal energy (ie/ρ)—remains constant and simply slides along with the flow. Meanwhile, the normal velocity of the fluid, by definition of moving perfectly with the boundary, becomes exactly S_*.

Therefore, the full vector of conserved variables for the star state $\mathbf{U}_K^* = [\rho^*, (\rho v_n)^*, (\rho v_t)^*, E^*, ie^*]_K$ is given by:

$$\mathbf{U}_K^* = \rho_K^* \begin{bmatrix} 1 \\ S_* \\ v_{t,K} \\ \frac{E_K}{\rho_K} + (S_* - v_{n,K}) \left(S_* + \frac{P_K}{\rho_K (S_K - v_{n,K})} \right) \\ \frac{ie_K}{\rho_K} \end{bmatrix}$$

4. Calculate the Final Flux The solver determines the flux at the interface based on the direction of these three wave speeds:

- If $S_L \geq 0$, the entire wave structure is moving right. The flux is simply the original Left flux: $\mathbf{F}_L$.
- If $S_L < 0 \leq S_*$, the interface is inside the left star region. The flux is: $\mathbf{F}_L^* = \mathbf{F}_L + S_L(\mathbf{U}_L^* - \mathbf{U}_L)$.
- If $S_* < 0 \leq S_R$, the interface is inside the right star region. The flux is: $\mathbf{F}_R^* = \mathbf{F}_R + S_R(\mathbf{U}_R^* - \mathbf{U}_R)$.
- If $S_R \leq 0$, the entire wave structure is moving left. The flux is the original Right flux: $\mathbf{F}_R$.

By properly resolving the contact wave, the HLLC solver accurately tracks the boundaries between hot and cold gas, allowing the simulation to capture the complex, multi-phase thermodynamics of galaxy formation.

The Dual Energy Formalism

In cosmology, gas falling into a dark matter halo routinely hits hypersonic speeds, reaching Mach 10 to Mach 100+ (because the primordial intergalactic gas is very cold, its local speed of sound is exceptionally low, allowing infalling gas to achieve massive Mach numbers). At these extreme velocities, the macroscopic kinetic energy (E_{kin}) of the gas completely dominates its internal thermal energy (E_{int}). Note that the ratio of kinetic to internal energy scales with the square of the Mach number.

This creates a numerical problem. In a standard finite-volume solver, we only track the total energy ($E_{tot} = E_{kin} + E_{int}$). To find the gas pressure, we must extract the internal energy by subtracting the kinetic energy:

$$E_{int} = E_{tot} - E_{kin}$$

When E_{kin} is 99.999% of E_{tot}, the limits of floating-point arithmetic cause a catastrophic cancellation. The subtraction destroys the precision of the thermal energy, often resulting in exactly zero or even negative internal energies, which instantly crashes the simulation with unphysical negative pressures.

To solve this, modern cosmological codes employ the **Dual Energy Formalism**. Alongside the standard conserved variables, the simulation tracks the internal energy density as an independent, actively advected fluid field. This independent field is updated by its own fluxes and the physical work done by compression or expansion $(-P\nabla \cdot \mathbf{v})$.

The code then employs a dynamic "switch" during the calculation of the primitive variables:

- If the local flow is subsonic or undergoing a strong shock (e.g., where thermal energy is $> 0.1\%$ of the total energy), the code relies on the standard Total Energy equation. This guarantees perfect energy conservation across shock fronts:
$$P = (\gamma - 1) \left(E - \frac{1}{2} \rho |\mathbf{v}|^2 \right)$$
- If the local flow is hypersonic ($E_{int}/E_{tot} < 0.001$), the total energy calculation is deemed mathematically polluted. The code "switches" to calculating pressure strictly from the independently tracked internal energy:
$$P = (\gamma - 1) ie$$

This dual-tracking approach guarantees that the gas temperature remains physically valid even when plummeting into the deepest gravitational wells of the cosmic web.

Coupling Hydrodynamics to the Expanding Universe

In a static box, the hydrodynamic equations are only sourced by gravity and their own internal pressure. However, in a cosmological simulation, the gas, just like the dark matter, is defined in **comoving coordinates** and is therefore subject to the physics of the expanding universe.

To correctly model this, the Euler equations must be modified with new "source terms" that account for the expansion.

The cosmological effects are applied as "source terms" in the main integrator "Kick" steps, alongside the gravity. During each "Kick" step, an **external source acceleration** is applied to the gas in every grid cell, which is the sum of two components:

1. **Modified Gravity:** The comoving gravitational acceleration, $\mathbf{g}_{\text{comoving}}$, is calculated from the total density of *both* dark matter and gas. This acceleration is then scaled by $1/a^3$ to account for the dilution of physical density as the universe's volume ($V \propto a^3$) increases.
2. **Hubble Drag:** A velocity-dependent "friction" term, $-2H\mathbf{v}$, is added. This term, where H is the Hubble parameter and $\mathbf{v}$ is the gas's peculiar velocity, correctly damps the gas's peculiar momentum as space stretches.

The **external source acceleration** applied to the gas in each cell is therefore:

$$\mathbf{a}_{\text{source}} = \frac{\mathbf{g}_{\text{comoving}}}{a^3} - 2H\mathbf{v}$$

This acceleration, which is a force per unit mass, is then used to update the gas grid's conserved quantities over the half-time-step ($\Delta t/2$):

- **Momentum Update:** The momentum density $\mathbf{S} = \rho\mathbf{v}$ is updated by the force density ($\rho\mathbf{a}_{\text{source}}$):

$$\Delta\mathbf{S} = (\rho\mathbf{a}_{\text{source}})\frac{\Delta t}{2}$$

- **Energy Update:** The total energy density E is updated by the power density (Force $\cdot$ Velocity) delivered by these forces:

$$\Delta E = (\mathbf{v} \cdot \rho\mathbf{a}_{\text{source}})\frac{\Delta t}{2}$$

Sub-Grid Coupling

In a **hybrid** simulation code—where dark matter is treated as Lagrangian particles and gas is treated on an Eulerian grid—dark matter particles have sub-grid resolution; they use the grid for long distances, but the PP step allows them to interact even when they occupy the same grid cell. Gas, however, is limited to the grid. It only feels the gravitational acceleration vector ($\mathbf{g}_{\text{comoving}}$) calculated from the grid's potential.

Since we rely on the Particle-Mesh (PM) gravity solver to act as the bridge between the collisionless dark matter particles and the baryonic gas, in the scale of a single grid cell **the gas and the dark matter are blind to each other**. The gravitational acceleration at the center of cell i is calculated using a symmetric central difference stencil, comparing the potential of its neighbors:

$$a_i = -\frac{\Phi_{i+1} - \Phi_{i-1}}{2\Delta x}$$

Notice that the potential of cell i itself is absent from this equation. The grid calculates the gradient *across* the cell, effectively ignoring the mass *inside* the cell.

Consequently:

1. **Dark matter ignores local gas:** When a dark matter particle interpolates its long-range force from the grid, it feels the pull of distant gas, but it doesn't feel the pull from the gas in the same cell. The short-range Particle-Particle (PP) solver only iterates over other dark matter particles, leaving the gas out of the sub-grid physics.
2. **Gas ignores local dark matter:** The Eulerian gas updates its momentum by reading this same central-difference grid. It feels no gravitational pull toward the dark matter particles moving inside its own boundaries.

This lack of sub-grid resolution causes that at distances smaller than one grid cell (Δx), there is no gravitational coupling between the two fluids.

The Pseudo-Particle Solution

To bridge this sub-grid gap, we can compress the fluid mass of a cell into a **pseudo-particle** located at the center of the cell. Its mass is the local comoving density multiplied by the cell volume ($m_{\text{gas}} = \rho_{\text{cell}} \Delta x^3$).

During the PP step, as a dark matter particle searches its neighboring cells for other dark matter, it also calculates the $1/r^2$ Newtonian pull exerted by these stationary gas pseudo-particles.

For the gas to feel the dark matter, we can use a **momentum-conserving back-reaction**. When the dark matter particle calculates the gravitational force it feels *from* the gas, it applies an equal and opposite force *back* to the gas grid's momentum array. This force-matching guarantees that the sub-grid interactions conserve total momentum.

The Shell Theorem

Dark matter in our simulation represents a cold, collisionless fluid. We discretize this fluid into particles, and use small softening lengths (ϵ) so they can orbit each other tightly. Gas, however, cannot collapse into a singularity smaller than a grid cell.

The dark matter particles are flying through smooth, continuous clouds of gas. According to Newton's **Shell Theorem**, as an object moves deeper inside a uniform sphere of mass, the net gravitational pull it experiences smoothly decreases, reaching zero at the center.

To model this fluid physics, we can use a **Cubic Spline Softening Kernel**. We multiply the Newtonian force between a DM particle and a gas pseudo-particle by a weighting function, $W(r/h)$, where the boundary h is set to the cell size (Δx). This mimics the Shell Theorem for a uniform cloud of gas:

- **Outside the cell ($r \geq \Delta x$)**: The weight is $W = 1.0$. The dark matter particle feels the unmodified $1/r^2$ Newtonian pull of the gas mass.
- **Inside the cell ($r < \Delta x$)**: The weight smoothly drops as a cubic polynomial, reaching $W = 0$ at the center. This ramps down the force, representing the particle passing through a continuous fluid medium without experiencing a singularity.

The polynomial we'll use to calculate this weight is derived from the standard M_4 **Cubic Spline Kernel**, used by Monaghan (1992) for Smoothed Particle Hydrodynamics (SPH). In mathematical approximation theory, the M_n family of splines is generated by convolving a uniform boxcar function with itself n times. The M_4 cubic spline is the lowest-order spline that guarantees C^2 continuity—meaning the force and its derivative are smooth—which is required to prevent energy leaks in a symplectic KDK integrator.

The cubic spline models the gas pseudo-particle as a cloud whose density profile $\rho(q)$ cloud is defined as:

$$\rho(q) = \frac{8}{\pi h^3} M_{\text{gas}} \begin{cases} 1 - 6q^2 + 6q^3 & \text{if } 0 \leq q < 0.5 \\ 2(1-q)^3 & \text{if } 0.5 \leq q < 1.0 \\ 0 & \text{if } q \geq 1.0 \end{cases}$$

According to Newton's Shell Theorem (a particle moving through this spherically symmetric cloud only feels the gravitational pull of the mass interior to its current radius), the true sub-grid force is scaled by the **fraction of enclosed mass**:

$$F = \frac{G M_{\text{gas}} m_{\text{dm}}}{r^2} \times \left(\frac{M(< r)}{M_{\text{gas}}} \right)$$

To find this enclosed mass fraction, $W(q) = M(< r)/M_{\text{gas}}$, we must integrate the density profile over a spherical volume from the center out to the particle's radius r:

$$W(q) = \frac{1}{M_{\text{gas}}} \int_0^r 4\pi r'^2 \rho(r') dr'$$

By substituting the piecewise M_4 density profile into this integral and solving it, we get the polynomial weighting functions used to scale the force. For a dark matter particle deep inside the cell core ($0 \leq q < 0.5$), the integrated weight is:

$$W(q) = \frac{32}{3} q^3 - \frac{192}{5} q^5 + 32 q^6$$

For a dark matter particle in the outer envelope of the cell ($0.5 \leq q < 1.0$), the integrated weight is:

$$W(q) = -\frac{1}{15} + \frac{64}{3}q^3 - 48q^4 + \frac{192}{5}q^5 - \frac{32}{3}q^6$$

When $q \geq 1.0$, the integral evaluates to 1.0, as the particle is outside the gas cloud and feels all of its mass.

The Computational Reality

The pseudo-particle technique is rarely implemented in modern cosmological codes. The reason lies in the computational cost and the philosophy of grid-based solvers.

Instead of performing expensive approximations to compute the forces inside a coarse cell, Eulerian codes solve the resolution problem by using **Adaptive Mesh Refinement (AMR)**. These codes dynamically subdivide the grid whenever a region becomes dense, so that the grid itself provides the required spatial resolution.

Initial Conditions for the Gaseous Component

In cosmological simulations, the baryonic gas and the dark matter must be initialized to resemble the universe long after the epoch of recombination. By the time a typical simulation begins, the gas has already spent millions of years falling into the gravitational potential wells created by the dark matter.

Therefore, the gaseous component must be initialized in lockstep with the dark matter, sharing its density fluctuations and bulk velocity flows, governed by the Zel'dovich approximation.

1. Initial Density The gas density, ρ_{gas}, must trace the initial density perturbations of the dark matter. Once the dark matter particles are displaced to their starting positions, their mass is mapped to the Eulerian grid (via schemes like Cloud-in-Cell) to establish the primordial dark matter density field. The gas density in each cell is then set directly proportional to this field, scaled by the cosmic baryon fraction—the ratio of gas mass to dark matter mass in the simulation.

2. Initial Peculiar Velocity Because the gas has been gravitationally influenced by the dark matter prior to the start of the simulation, it shares the same large-scale velocity flows. To achieve this synchronization, the primordial velocity field is calculated using the Zel'dovich approximation. The gas grid is populated with these velocity vectors, $\mathbf{v}_{\text{pec}}$. The dark matter particles sample their initial velocities from this field based on their starting coordinates.

3. Initial Energy Because we employ the **Dual Energy Formalism**, we must initialize two separate energy fields. First, the explicitly tracked *internal* energy is initialized to a very low, uniform baseline. Second, the *total* energy of the gas grid is initialized as the sum of this internal thermal energy and the macroscopic kinetic energy ($E_k = \frac{1}{2}\rho v^2$) dictated by its initial primordial momentum.

To set the internal energy baseline, we link the internal energy density to the physical temperature (T) of the early universe using the ideal gas law:

$$ie = \frac{\rho k_B T}{(\gamma - 1)\mu m_p}$$

Here, k_B is the Boltzmann constant, m_p is the mass of a proton, and μ is the mean molecular weight (which is approximately 1.22 for the primordial, neutral mix of hydrogen and helium). In a typical cosmological simulation starting at a redshift of $z = 49$, the universe has expanded and cooled significantly since the Big Bang. At this epoch, the background gas temperature is roughly 50 K. By plugging this baseline temperature and the local cell density (ρ) into the equation above, we set the initial ie for every cell.

Key Literature & Further Reading

Teyssier, R. (2002). *Cosmological hydrodynamics with adaptive mesh refinement. A new high-resolution code called RAMSES.* arXiv:astro-ph/0111367. Available at https://arxiv.org/abs/astro-ph/0111367

Riemann Solvers & MUSCL Schemes:

Toro, E. F. (2009). *Riemann Solvers and Numerical Methods for Fluid Dynamics: A Practical Introduction* (3rd ed.). Springer.

Gas Physics

The engine of galaxy formation is **thermodynamics**—the physics of how gas heats up and cools down. The balance between these two processes determines whether a gas cloud has enough pressure to resist gravity or whether it will collapse to form stars.

Temperature

Temperature is defined as a measure of the **average random kinetic energy** of the gas particles. It is a statement about **how fast the particles are moving** relative to the bulk flow of the fluid.

- A **"hot"** cosmic gas is one where the individual atoms and ions are zipping around at high random speeds.
- A **"cold"** cosmic gas is one where the particles are moving relatively slowly.

In the hydrodynamics solver, we explicitly track the specific internal energy of the gas, u (which is the total internal energy density divided by the mass density, ie/ρ). To translate this bulk macroscopic energy into a physical temperature, T, we can use the ideal gas law:

$$u = \frac{k_B T}{(\gamma - 1)\mu m_p}$$

By rearranging this equation, we can calculate the temperature of any gas cell:

$$T = \frac{u(\gamma - 1)\mu m_p}{k_B}$$

Here, k_B is the Boltzmann constant and m_p is the mass of a proton. The two remaining parameters describe the fluid:

- γ is the adiabatic index (5/3 for a monatomic ideal gas).
- μ is the **mean molecular weight**, which represents the average mass of a particle in the gas in units of proton masses. For a primordial, fully ionized mix of 76% hydrogen and 24% helium, $\mu \approx 1.22$.

Gravitational Compression and Shocks

Cosmic gas increases its temperature when energy is added to it from large-scale astrophysical processes. The primary heating mechanism is **gravitational compression**.

As gas is pulled into the gravitational well of a dark matter halo, it accelerates to high speeds. When this rapidly falling gas meets the gas that has already accumulated, it collides violently, creating a **shock wave**. This shock wave is an almost instantaneous conversion of the gas's ordered, in-falling kinetic energy into disordered, random motion—in other words, heat. This process, known as **virial heating**, can raise the gas temperature to millions of degrees.

Other significant heating sources include:

- **Supernova Feedback:** The explosive death of massive stars creates powerful blast waves that rip through the surrounding medium, shocking and heating the gas.
- **Radiation:** High-energy photons from stars and active galactic nuclei can ionize atoms, transferring their energy to the gas and heating it.

At this stage in our simulation, the primary heating mechanism—virial heating—is already implemented. Gravity accelerates the gas into the dark matter halos, generating kinetic energy. The HLLC solver resolves the shock fronts (dissipating the macroscopic kinetic energy into internal thermal energy), while the explicit $-P(\nabla \cdot \mathbf{v})$ source term in our Dual Energy Formalism captures the steady heating of adiabatic compression. With these operators in place, the cosmic gas will shock and heat on its own.

Radiative Cooling

The *only* way a gas cloud in the vacuum of space can lose thermal energy is by radiating it away in the form of photons (light). This process, known as **radiative cooling**, is crucial for galaxy formation. If a gas cloud cannot cool, its internal thermal pressure will resist the pull of gravity, preventing it from condensing into stars.

Just as we used operator splitting to separate the fluid dynamics from gravity and cosmic expansion, we can integrate radiative cooling as one more independent operator in the sequence. After the gas flows and responds to gravitational forces, we calculate how much the gas cools over the timestep Δt in every cell of the grid.

The Cooling Function (Λ)

We must determine how fast the gas is radiating energy away. Gas particles turn their kinetic energy into escaping photons through several quantum mechanisms (such as line emission from atomic electron transitions or Compton scattering).

For the hot plasmas found inside collapsed dark matter halos, the dominant cooling mechanism is **Bremsstrahlung** (German for "braking radiation"), also known as free-free emission. In a fully ionized plasma, a fast-moving, negatively charged free electron will occasionally fly close to a positively charged ion. The ion's electric field deflects the electron, causing it to decelerate. As the electron is deflected by the ion, it emits a high-energy photon (typically an X-ray). Because this photon escapes into space carrying energy with it, the electron must lose an equivalent amount of its own kinetic energy, thereby cooling the gas.

Astrophysicists encapsulate all of these complex light-emitting processes into the **Cooling Function**, denoted by $\Lambda(T, \rho)$.

The cooling function outputs the **volumetric energy loss rate**—the total amount of energy radiated away per unit volume, per second (typically expressed in erg cm^{-3} s^{-1}). For pure Bremsstrahlung, the cooling function can be approximated as:

$$\Lambda_{\text{brem}} \approx 1.4 \times 10^{-27} \sqrt{T} n_e n_i$$

Notice that:

- $\sqrt{T}$ **(The Velocity Limit):** To emit a photon, an electron must fly past an ion. The rate at which these encounters happen depends on the electron's speed. Because kinetic energy is proportional to temperature ($v^2 \propto T$), the velocity of the electron scales with the square root of the temperature ($v \propto \sqrt{T}$).
- $n_e n_i$ **(The Collision Probability):** The **number density** (n) counts the *number of particles* in a given volume. Here, n_e is the count of electrons and n_i is the count of ions. For a flash of Bremsstrahlung radiation to occur, one electron and one ion must cross paths. The probability of this happening is the product of their populations ($n_e \times n_i$). Because both the number of electrons and the number of ions rise directly with the mass density of the gas (ρ), the resulting cooling rate scales with the **density squared** ($\Lambda \propto \rho^2$). This means gas violently compressed in the center of a dark matter halo will cool drastically faster than the diffuse gas on the outskirts.

The proportionallity above ($n_e \times n_i \propto \rho^2$) assumes the gas is a fully ionized plasma. However, around 10,000 K, primordial gas undergoes recombination. Protons capture the free electrons, turning the plasma into a gas of cold, neutral hydrogen and helium atoms. Because there are practically no free electrons or ions left, Bremsstrahlung cooling shuts down and the physical radiative cooling rate (Λ) plummets to zero below 10,000 K. *(Note: In the real universe, as the first stars and quasars ignite, they flood the cosmos with Ultraviolet radiation. This UV background acts as a universal heating source (Γ) that balances against this atomic cooling limit, creating a cosmic thermostat that holds the diffuse intergalactic gas steady at around 10,000 K.)*

The Differential Equation

In our Eulerian grid, we explicitly track the internal energy **density**, ie (energy per unit volume). However, when calculating how the gas in a specific cell cools over time, it is cleaner to use the **specific internal energy**, u (energy per unit mass). We extract this for each cell by dividing the energy density by the mass density ($u = ie/\rho$). Consequently, to balance the equation, we must also convert the volumetric cooling function Λ (energy lost per unit volume) into a specific rate by dividing it by the gas density ρ.

This leaves us with the ordinary differential equation (ODE) that governs radiative cooling:

$$\frac{du}{dt} = -\frac{\Lambda(T, \rho)}{\rho}$$

Attempting to solve this equation numerically inside a simulation is challenging. We'll see why in the following section.

Stiff Equations

Up to this point, our simulation has relied on **explicit integration**. In an explicit method (such as the KDK leapfrog scheme), the rate of change is calculated using the current state of the system. This rate is then assumed to remain constant over a small forward timestep, Δt.

Applying a simple Forward Euler explicit integration to the cooling equation ($\frac{du}{dt} = -\frac{\Lambda}{\rho}$), the expression to advance the specific internal energy from its current state (u^n) to its new state (u^{n+1}) is written as:

$$u^{n+1} = u^n - \Delta t \cdot \frac{\Lambda(T^n, \rho^n)}{\rho^n}$$

However, this approach introduces a computational problem. To understand why, we must introduce the concept of **timescales**.

The Cooling Timescale (t_{cool})

Every physical process in the universe happens at a certain speed. In computational physics, this is quantified using a characteristic timescale—roughly, the amount of time it takes for a process to significantly change the state of the system.

For radiative cooling, the **cooling timescale** (t_{cool}) is defined as the time it would take for a gas cloud to radiate away 100% of its current internal thermal energy if it continued cooling at its current rate:

$$t_{\text{cool}} = \frac{u}{|du/dt|} = \frac{\rho u}{\Lambda}$$

This timescale is highly volatile. In the vast, empty voids of the cosmic web, the gas is so diffuse that Λ is practically zero, making t_{cool} billions of years. But when that gas falls into a dark matter halo and compresses, the density ρ increases dramatically, and the cooling timescale shortens accordingly.

In the dense center of a collapsing halo, t_{cool} can drop to just a few thousand years. This creates a conflict with our global timestep Δt, which might be on the order of a few million years:

$$\Delta t_{\text{hydro}} \gg t_{\text{cool}}$$

Limitations of Explicit Integration

When a differential equation contains a timescale that is drastically shorter than the simulation's timestep, we refer to the equation as **stiff**. Explicit integrators become unstable when applied to stiff equations over standard simulation timesteps.

We can see why by looking back at our explicit Forward Euler equation. Let's imagine a dense gas cell where t_{cool} is 10,000 years, but our hydro solver dictates we must take a step of $\Delta t = 1,000,000$ years.

$$u^{n+1} = u^n - (1,000,000) \cdot \frac{\Lambda}{\rho}$$

Because $t_{\text{cool}} = \frac{\rho u}{\Lambda} = 10,000$, we know mathematically that the cooling term ($\frac{\Lambda}{\rho}$) is exactly equal to $\frac{u}{10,000}$. Substituting this in:

$$u^{n+1} = u^n - (1,000,000) \cdot \left(\frac{u^n}{10,000}\right)$$

$$u^{n+1} = u^n - 100 \cdot u^n = -99u^n$$

The explicit solver assumes the gas continues to cool at its initial rate for the entire timestep. It overshoots the physical reality—where the gas would cool down, the cooling rate would drop, and the temperature would gently settle. Instead, the solver extracts 100 times more energy than the cell actually contains. The result is a **negative internal energy**, an unphysical outcome.

At first glance, the obvious solution to this overshoot is to shrink the global timestep Δt to match the shortest cooling timescale. If the code took tinier steps, the explicit solver would remain stable. However, this is

computationally unpractical. Forcing the hydrodynamics solver to run hundreds of extra times just to cool a small fraction of dense gas cells isn't a good solution.

Implicit Integration

The Backward Euler Method

As we saw earlier, explicit integration fails because it blindly projects the current cooling rate forward, resulting in massive overshoots. To deal with the timescale mismatch between hydrodynamics and cooling, we can use an **Implicit Integration** method within the cooling operator, like the **Backward Euler** scheme.

Instead of asking, *"How fast is the gas cooling right now?"*, the Backward Euler method asks, *"What future temperature would justify the energy lost to get there?"*

We evaluate the cooling function Λ not at the old state (u^n), but at the **unknown future state** (u^{n+1}):

$$u^{n+1} = u^n - \Delta t \cdot \frac{\Lambda(T^{n+1}, \rho)}{\rho}$$

Conveniently, because the cooling rate Λ drops rapidly as the temperature drops ($\Lambda \propto \sqrt{T}$), evaluating it at the colder, future state creates an automatic, self-regulating feedback loop.

If we take a massive timestep (Δt), the equation assumes the gas rapidly cools down early in the step and spends the rest of the time radiating at a slow rate. This guarantees **unconditional stability**, meaning the numerical solution will never diverge or 'blow up', regardless of the size of the timestep Δt.

Newton-Raphson Root-Finding

The Backward Euler equation cannot be solved directly. The unknown future state, u^{n+1}, is embedded within the non-linear cooling function $\Lambda(u^{n+1})$.

To solve for it, we must rewrite the equation as a root-finding problem. We define a function $f(u^{n+1})$ such that the correct physical answer occurs when the function equals zero:

$$f(u^{n+1}) = u^{n+1} - u^n + \Delta t \cdot \frac{\Lambda(u^{n+1}, \rho)}{\rho} = 0$$

To find this root, we can employ the **Newton-Raphson method**. The process begins with an initial guess (usually assuming the future energy will remain equal to the current energy, $u_{\text{guess}} = u^n$). The slope (the derivative) of the function is then calculated at that initial guess and used to extrapolate a new, more accurate guess closer to zero.

The iterative update formula is:

$$u_{\text{next}} = u_{\text{guess}} - \frac{f(u_{\text{guess}})}{f'(u_{\text{guess}})}$$

Here, f' is the derivative of the root-finding function with respect to the internal energy. In practice we can to compute this derivative numerically—by evaluating Λ at u_{guess} and at a slightly offset value to determine the local slope. This decouples the root-finding algorithm from the specific physics being modeled, allowing the cooling function to be upgraded with complex, tabulated atomic chemistry data without requiring a hard-coded analytical derivative.

The solver repeats this process, converging toward the true future energy. Once the guess stops changing (reaching a strict tolerance), the solver exits. Furthermore, we can inject a hard **temperature floor** (e.g., the temperature of the Cosmic Microwave Background, or the 10,000 K **atomic cooling limit** established earlier) into this solver; if a Newton-Raphson guess ever drops below this floor, the solver immediately overrides the answer to the floor value and exits.

Coupling Cooling to the Simulation

The implicit solver calculates how much thermal energy the gas radiates away during a timestep. Now this must be integrated in the simulation.

In the hydrodynamics solver, we established the **Dual Energy Formalism** to survive hypersonic flows. This means the grid tracks two different energy variables for every cell: the total energy density ($E = E_{\text{kin}} + E_{\text{int}}$) and the internal thermal energy density ($ie = E_{\text{int}}$).

Radiative cooling removes *thermal* energy. The photons escaping into space carry away heat, but they do not slow down the macroscopic bulk flow of the gas. The kinetic energy ($\frac{1}{2}\rho v^2$) must be preserved.

Since we are using the Dual Energy Formalism, to consistently extract the radiated light while preserving the kinetic velocity of the gas, the lost thermal energy must be substracted from **both**, internal energy and total energy arrays:

$$ie_{\text{new}} = ie_{\text{old}} - \Delta E_{\text{vol}}$$
$$E_{\text{new}} = E_{\text{old}} - \Delta E_{\text{vol}}$$

Because $E_{\text{new}} - ie_{\text{new}} = (E_{\text{old}} - \Delta E_{\text{vol}}) - (ie_{\text{old}} - \Delta E_{\text{vol}}) = E_{\text{old}} - ie_{\text{old}}$, the kinetic energy remains untouched. The gas cools down, the pressure drops, but the fluid continues to flow at the same speed.

Thermodynamics in Comoving Coordinates

In our cosmological simulation, the Eulerian grid is fixed in comoving space. While the comoving volume of a cell remains constant, the true physical volume it represents stretches with the scale factor, a.

Since the comoving velocity is defined as the time derivative of the comoving position: $v_{\text{code}} = \dot{x}$, the physical peculiar velocity of the gas is $v_{\text{phys}} = a \cdot v_{\text{code}}$. Because kinetic energy scales with the square of the velocity, the physical kinetic energy natively carries an a^2 dependence ($E_{\text{kin, phys}} \propto a^2 v_{\text{code}}^2$).

To maintain consistency within the Dual Energy Formalism, the internal energy must be scaled identically before it can be summed with the kinetic energy. Therefore, the internal energy density tracked by the solver (ie_{code}) relates to the physical internal energy density (ie_{phys}) via the scale factor:

$$ie_{\text{phys}} = a^2 \cdot ie_{\text{code}}$$

This transformation dictates every interaction between the cooling module and the hydrodynamic grid:

1. Temperature and Cooling Conversions When computing the physical temperature of the gas or querying tabulated cooling functions (Λ), the solver must first recover the physical specific internal energy by multiplying the code variables by a^2. Conversely, once the physical cooling rate is calculated, the required energy deduction (ΔE_{vol}) must be mapped back to the grid by dividing the physical value by a^2 before applying the subtraction.

2. Adiabatic Expansion (PdV Work) This coordinate transformation alters the integration of adiabatic expansion. In a physical universe, a gas cools adiabatically as its volume expands, governed by its specific adiabatic index (γ). The temperature of an expanding gas scales with its density as $T \propto \rho^{\gamma-1}$. Because the physical density drops proportionally to the expanding volume ($\rho \propto a^{-3}$), the temperature—and therefore the physical internal energy for the constant mass of gas within a comoving cell—scales as:

$$ie_{\text{phys}} \propto (a^{-3})^{\gamma-1} = a^{-3(\gamma-1)}$$

Taking the time derivative of this relationship we get the rate of energy loss. Applying the chain rule to the scale factor dependence yields:

$$\frac{d}{dt}(ie_{\text{phys}}) \propto -3(\gamma-1)a^{-3(\gamma-1)-1}\dot{a}$$

By factoring out the original $a^{-3(\gamma-1)}$ scaling and substituting the definition of the Hubble parameter ($H = \frac{\dot{a}}{a}$), we see that the physical internal energy decays at a rate of $-3(\gamma-1)H$:

$$\frac{d}{dt}(ie_{\text{phys}}) = -3(\gamma-1)\left(\frac{\dot{a}}{a}\right)a^{-3(\gamma-1)} = -3(\gamma-1)H \cdot ie_{\text{phys}}$$

However, because the simulation arrays must store the scaled ie_{code} variable, the rate of change for the grid is governed by the chain rule:

$$\frac{d}{dt}(a^2 \cdot ie_{\text{code}}) = -3(\gamma - 1)H(a^2 \cdot ie_{\text{code}})$$

Evaluating the derivative on the left side (and noting that $\dot{a}/a = H$) yields:

$$a^2 \frac{d}{dt}(ie_{\text{code}}) + 2Ha^2 \cdot ie_{\text{code}} = -3(\gamma - 1)Ha^2 \cdot ie_{\text{code}}$$

Subtracting the expansion term to isolate the update rate for the simulation array results in:

$$a^2 \frac{d}{dt}(ie_{\text{code}}) = [-3(\gamma - 1) - 2]\, Ha^2 \cdot ie_{\text{code}}$$

By factoring out the a^2 scaling on both sides and simplifying the mathematical coefficient (since $-3\gamma + 3 - 2 = -3\gamma + 1 = -(3\gamma - 1)$) we get the generalized code update rate:

$$\frac{d}{dt}(ie_{\text{code}}) = -(3\gamma - 1)H \cdot ie_{\text{code}}$$

The physical thermodynamic cooling ($-3(\gamma - 1)H$) combines with the coordinate system's stretching ($-2H$) to produce a $-(3\gamma - 1)H$ decay. To consistently simulate the cosmological PdV work and prevent the expansion of the universe from artificially heating the gas, the integrator must drain the internal and total energy arrays at this rate.

The Optically Thin Approximation

In our implementation of radiative cooling, when a gas cell cools down, we subtract that thermal energy from the grid and permanently delete it from the simulation. From a conservation standpoint, this means our simulated universe is an open system leaking energy.

In the real universe, a photon emitted by a decelerating electron could indeed travel across space, strike another atom, and transfer its energy back into heat. To simulate this accurately, we would need to upgrade our hydrodynamics engine to **Radiation Hydrodynamics (RHD)** by solving the Radiative Transfer equation.

However, doing so is computationally prohibitive. To avoid this, cosmological codes rely on the **Optically Thin Approximation**. We assume that the gas is "optically thin," meaning it is transparent to its own radiation.

For the hot plasmas found inside collapsed dark matter halos, this is a good physical assumption. Even though the gas is compressed by gravity, cosmological densities are still a hard vacuum. When a high-energy X-ray photon is emitted via Bremsstrahlung cooling, its mean free path is so large that it will almost certainly sail entirely out of the galaxy, out of the dark matter halo, and out of the simulation box without ever striking another atom. Therefore, assuming the photon escapes into the void and permanently deleting its energy from the computational domain is a reliable representation of the physics.

There are also environments where this approximation breaks down. When gas condenses into star-forming molecular clouds, it becomes "optically thick" and opaque, trapping heat inside. Furthermore, during the Epoch of Reionization, dense neutral hydrogen violently absorbed incoming Ultraviolet light from the first stars. To account for these complex radiation effects without actually running a Radiative Transfer solver, simulations employ sub-grid approximations—rules that reproduce physics occurring on scales smaller than a single grid cell. A common technique is to assume the universe is bathed in a uniform glow of UV light, which we mimic by enforcing an artificial temperature floor—such as the 10,000 K atomic cooling limit—preventing the diffuse gas from cooling below the thermodynamic baseline set by this universal radiation background.

The Subgrid Clumping Factor

In an ideal, infinite-resolution simulation, the Eulerian equations of hydrodynamics capture the collapse of gas into Dark Matter halos. However, when modeling large cosmological volumes, the physical size of a single grid cell may span tens or hundreds of kiloparsecs. This discretization introduces a thermodynamic problem: the artificial suppression of radiative cooling.

The Problem of Eulerian Smoothing

In reality, gas is not a uniform fog; supersonic turbulence (chaotic, swirling macroscopic gas motions driven by colliding inflows) shreds it into a multiphase medium consisting of dense, cold knots surrounded by hot, sparse bubbles.

Radiative cooling mechanisms, such as Bremsstrahlung (free-free emission), are two-body collisional processes. Consequently, the cooling rate (Λ) is proportional to the square of the gas density:

$$\Lambda \propto \rho^2$$

Because an Eulerian solver assigns a single, smoothed average density ($\langle\rho\rangle$) to an entire cell, the grid calculates the cooling rate based on the square of that average. However, due to a mathematical principle known as **Jensen's Inequality**, the square of an average is smaller than the average of the squares:

$$\langle\rho\rangle^2 < \langle\rho^2\rangle$$

This means the grid underestimates the true cooling rate. The thermal energy becomes trapped inside the cell, causing artificial pressure to build up. This numerical artifact results in a gas that is blasted out of the gravitational well rather than condensing into a galactic core.

The Subgrid Clumping Model

To solve this without requiring computationally expensive grid resolutions, we employ a **Subgrid Clumping Factor** (often denoted as C). This model injects the missing, small-scale physics back into the macroscopic grid cells.

The clumping factor is defined as the ratio of the average of the squared densities to the square of the average density:

$$C = \frac{\langle\rho^2\rangle}{\langle\rho\rangle^2}$$

To implement this dynamically, we tie the subgrid variance to the macroscopic environment. As a cosmological structure collapses, supersonic turbulence increases, leading to more severe clumping. Therefore, the clumping factor can be modeled as a function of the local overdensity (Δ), which is the ratio of the local cell density to the mean density of the universe:

$$\Delta = \frac{\rho_{local}}{\bar{\rho}}$$

In our model, we adopt a generalized power-law scaling for collapsing regions ($\Delta > 1$) to account for unresolved gas structures. The modified cooling factor is computed as:

$$C = 1 + A\Delta^B$$

While the conceptual foundation for scaling subgrid clumping with local overdensity and turbulence is well-established in studies (e.g., Mao et al. 2020; Federrath et al. 2008), our formulation is a custom fit where the parameters A and B are empirically calibrated based on the physics and resolution of the simulation:

- **The Exponent (B):** This parameter captures how fast the subgrid gas fragments into dense knots as the halo collapses. As the gas compresses, its rising temperature provides thermal pressure support against further fragmentation. Because this thermal resistance grows stronger as the density increases, the efficiency of subgrid fragmentation gradually diminishes, preventing the clumping from scaling proportionally with the macroscopic overdensity. Thus B is set to be sublinear (< 1.0).
- **The Amplitude (A):** This parameter serves as a correction factor specific to the simulation's grid resolution. It represents the degree to which the mesh fails to resolve the small-scale clumping. This amplitude is calibrated empirically so that the simulation matches macroscopic physical observables, such as the cosmic star formation rate or the global cold gas fraction.

The subgrid clumping factor acts as a correction to the squared average density of the grid cell:

$$\Lambda_{true} = (C \times \langle\rho\rangle^2) f(T)$$

where $f(T)$ represents the temperature-dependent physics of the cooling function.

To calibrate the subgrid clumping amplitude (A), we require the simulation to reproduce the macroscopic condensed baryon fraction at $z = 0$. Because our hydrodynamics model does not include a subgrid recipe for star formation, gas that cools below $T < 10^4$ K and reaches halo overdensities of $\Delta > 100$ remains trapped in the fluid phase. Therefore, the simulated cold dense gas fraction acts as a proxy for the total collapsed baryon budget—comprising both the interstellar medium (ISM) and stellar mass.

Observational baryon censuses in the local universe indicate that stars and stellar remnants account for roughly 5% to 7% of the total cosmic baryon density, while cold neutral gas (H I, He I and H_2) accounts for an additional 1.5% to 2% (Shull, Smith, & Danforth 2012). Consequently, we tune the parameter A such that the final simulated cold dense gas mass fraction at $z = 0$ settles within the interval 7% to 10%.

This subgrid model acknowledges that overdense cells contain unresolved, dense knots of gas, where thermal energy is radiated away, allowing the pressure to drop and the gas to undergo collapse.

Key Literature & Further Reading

Radiative Physics:

Rybicki, G. B., & Lightman, A. P. (1979). *Radiative Processes in Astrophysics*. Wiley-VCH. (See Chapter 5 for the derivation of the Thermal Bremsstrahlung cooling rates).

Mo, H., van den Bosch, F. C., & White, S. (2010). *Galaxy Formation and Evolution*. Cambridge University Press. (See Chapter 8 for the application of cooling functions in cosmological structure formation).

Numerical Implementation of Stiff Cooling:

Anninos, P., Zhang, Y., Abel, T., & Norman, M. L. (1997). *Cosmological Hydrodynamics with Multi-Species Chemistry and Nonequilibrium Ionization*. New Astronomy, 2(3), 209-224. Available at https://arxiv.org/abs/astro-ph/9608041. (Details the standard use of operator-split, implicit backward-Euler integration with Newton-Raphson iterations for cosmological cooling).

Subgrid clumping factor:

Daisuke Nagai, Erwin Lau. (2011). *Gas Clumping in the Outskirts of Lambda-CDM Clusters*. The Astrophysical Journal. Available at https://arxiv.org/abs/1103.0280.

Metallicity and the Cosmic Ultraviolet Background

Metallicity (Z)

In astrophysics, the elemental composition of the universe is historically divided into three categories: Hydrogen (mass fraction X), Helium (mass fraction Y), and everything else. **Metallicity**, denoted by the letter Z, is the mass fraction of all elements heavier than helium (Pagel, 2009). The relationship defining the composition of any gas parcel is $X + Y + Z = 1$. The present-day metallicity in the Sun is approximately $Z \approx 0.014$ to 0.02 (Asplund et al., 2009).

These heavy elements were absent in the early cosmos. After the Big Bang, primordial nucleosynthesis produced almost exclusively Hydrogen and Helium, alongside small traces of Lithium. Every element heavier than Helium had to be forged much later via stellar nucleosynthesis of the very first stars (Population III). When these massive, short-lived stars exhausted their fuel, they exploded as supernovae, seeding the surrounding gas with the first heavy elements and permanently altering the chemical and thermodynamic evolution of the universe.

To form low-mass stars like the Sun, the gas temperature must drop to ~ 10 K (Bodenheimer, 2011). In a primordial universe ($Z = 0$), gas cools very inefficiently. Below 10^4 K, atomic hydrogen can no longer radiate efficiently, leaving only trace amounts of molecular hydrogen (H_2) as a coolant. H_2 is a poor radiator, meaning the gas remains relatively warm, which results in a high Jeans mass (the minimum mass a cloud must have for its gravity to overcome its internal thermal pressure, $M_J \propto \frac{T^{3/2}}{\rho^{1/2}}$) and the formation of massive, short-lived Population III stars (Bromm, Coppi, & Larson, 1999).

Metals ameliorate this thermodynamic bottleneck. Heavy elements like Carbon, Oxygen, and Iron possess complex electron structures with numerous atomic transitions and fine-structure lines that can be excited even at low temperatures. As a result, once a gas cloud is enriched with metals beyond a certain "critical metallicity"

(roughly $Z_{crit} \approx 5 \times 10^{-4} Z_{\odot}$, depending on halo mass and redshift), metal-line cooling overcomes the heat generated by gravitational compression (Bromm, Ferrara, Coppi, & Larson, 2001). The gas can then cool to a few tens of Kelvin, drastically lowering the Jeans mass and allowing the cloud to fragment into the smaller Population II and Population I stars we see today (Schneider et al., 2006).

Ultraviolet Background (UVB)

The cosmic ultraviolet background (UVB) is a uniform radiation field originating from the cumulated radiation of ionizing sources, almost entirely quasars (extremely luminous active galactic nucleus) and young, massive stars (Haardt & Madau, 2001). This background radiation impacts the gas by modifying its ionization and excitation states. Specifically, the UVB drives the photoionization (the physical process in which an ion is formed from the interaction of a photon with an atom or molecule) and photoheating of the intergalactic medium to temperatures of $T \sim 10^4$ K. This photoheating process raises the cosmological Jeans mass, providing sufficient pressure support to prevent baryonic gas from collapsing into the gravitational potential wells of low-mass halos. Consequently, the UVB acts as a feedback mechanism that reduces gas infall and suppresses star formation in small galaxies (Benítez-Llambay & Frenk, 2020).

Numerical methods

Passive Scalar Advection

To handle metallicity within our second-order Godunov-type scheme (a finite-volume method that computes fluid flow by solving for the wave interactions at cell boundaries, in this case using HLLC + MUSCL + SSP-RK2), the grid tracks the conserved comoving metal mass density, ρ_Z. This metal density is treated as a "passive scalar" within the Eulerian hydrodynamics solver, meaning that it does not directly influence the pressure or sound speed of the fluid during the resolution of shockwaves or contact discontinuities. Instead, metals flow passively alongside the gas mass.

However, during the spatial reconstruction step of the Riemann solver, we extrapolate the primitive metal mass fraction to the cell interfaces rather than the conserved density:

$$Z = \frac{\rho_Z}{\rho}$$

Reconstructing the dimensionless fraction Z rather than the metal density ρ_Z ensures consistency and prevents unphysical numerical artifacts—such as the spontaneous generation of metals. This artifact occurs because non-linear slope limiters clip the gradients of ρ and ρ_Z to different degrees, allowing their reconstructed ratio at the cell boundary to artificially overshoot the actual metallicity of the surrounding fluid.

To resolve this, the flux of the metal density, F_{ρ_Z}, crossing a cell boundary is calculated as the mass flux, F_ρ, multiplied by the reconstructed metal fraction at the upwind cell side (the cell from which the fluid flows) (Toro, 2009):

$$F_{\rho_Z} = F_\rho Z_{\text{upwind}}$$

Reconstructing directly Z and linking its flux to the mass flux, guarantees that the advected fluid maintains its chemical proportions.

Thermodynamics

Z becomes active in the energy equation. During the thermodynamic step, the solver calculates the net rate of change of specific internal energy, $\frac{du}{dt}$. This evolution is driven by the balance between the volumetric photo-heating rate (Γ) and the volumetric radiative cooling rate (Λ), expressed as:

$$\frac{du}{dt} = \frac{\Gamma(T, \rho, z, Z) - \Lambda(T, \rho, z, Z)}{\rho}$$

Because computing the quantum mechanical emission, ionization balance (the equilibrium where the rate of atoms being ionized equals the rate of ions reverting to neutral), and excitation states of every element on the fly is computationally expensive, some codes rely on pre-computed tables. These tables include the metal-dependent cooling rates and the heating rates generated by the photo-ionizing effect of the cosmic ultraviolet background (Haardt & Madau, 2001; Wiersma, Schaye, & Smith, 2009).

Our code uses data from Grackle, an open-source chemistry and radiative cooling library designed for astrophysical simulations of the Intergalactic Medium (IGM) (Smith et al., 2017). The specific data file implemented in this model, `HM2012.h5`, contains pre-computed, tabulated cooling and heating rates assuming ionization equilibrium generated using the photoionization code Cloudy. Cloudy numerically solves the quantum mechanical equations governing ionization balance, bound-bound emission lines (emission by excited bound electrons), and radiative recombination (radiation after a free electron is captured) for an array of atomic species. For the data table we are using, the Cloudy calculations were driven by the time-dependent ultraviolet background (UVB) model developed by Haardt & Madau (2012).

The resulting file provides the volumetric energy loss and gain rates structured across a three-dimensional grid. The axes of this grid correspond to cosmological redshift, gas density, and temperature. The thermodynamic outputs are separated into primordial (Hydrogen and Helium) and metal-dependent contributions. This separation allows the solver to scale the metal-line cooling rates by the metallicity fraction within each cell.

The pre-computed tables are dependent on redshift, as the intensity of the cosmic ultraviolet background and its corresponding photoheating rate evolve over time. The redshift-dependent background radiation is derived relying on input parameters for the chosen cosmology, including the Hubble constant, the matter density (Ω_M), and the cosmological constant (Ω_Λ). Consequently, the cooling tables are cosmology-dependent. Modifying a simulation's cosmological framework requires the generation of a new set of tables to ensure consistency.

Moreover, the dependency of the model on redshift decouples the gas thermodynamics from the actual emission of ionizing sources, such as star-forming galaxies and active galactic nuclei, present within the simulation volume. With this abstraction, the photoionization and heating rates of the intergalactic medium are uniformly driven by the average global history of the real universe rather than the radiation dynamically generated by the simulation's own structures. The reason for allowing this violation of causality is that performing on-the-fly radiative transfer to track local photons, or solving the massive network of non-equilibrium chemical reactions would be too expensive computationally.

To extract continuous rates from the pre-computed tables, we employ trilinear interpolation across the three dimensions of the data grid: redshift, gas density, and temperature. Because radiative cooling is a stiff equation, to update the specific internal energy of the gas we use an implicit integration scheme, which requires finding the future internal energy, u_{new}:

$$f(u_{new}) = u_{new} - u_{old} - \Delta t \frac{du}{dt}(u_{new}) = 0$$

Solving this implicit equation requires a numerical root-finding algorithm. The Newton-Raphson method relies on the evaluation of smooth, continuous derivatives, $f'(u)$. However, tabulated metal cooling curves are inherently jagged due to the sudden onset of quantum line-emission transitions at specific temperatures.

A more appropriate method for this scenario is the bisection method. The bisection method is an unconditionally stable bracketing algorithm that doesn't depend on derivative calculations. It operates by identifying a lower bound (u_{low}) and an upper bound (u_{high}) that enclose the true root, ensuring that $f(u_{low})$ and $f(u_{high})$ have opposite signs. The algorithm calculates the midpoint of the bracket and evaluates the function. Depending on the sign of the result, the midpoint replaces either the upper or lower bound, halving the search interval with each step. The bisection method guarantees a physical solution regardless of how jagged the underlying function may be.

Key Literature & Further Reading

Haardt, F., & Madau, P. (2012). *Radiative Transfer in a Clumpy Universe. IV. New Synthesis Models of the Cosmic UV/X-Ray Background*. The Astrophysical Journal, 746(2), 125.

Smith, B. D., Bryan, G. L., Glover, S. C. O., et al. (2017). *GRACKLE: a chemistry and cooling library for astrophysics.* Monthly Notices of the Royal Astronomical Society, 466(2), 2217-2234.

Asplund, M., Grevesse, N., Sauval, A. J., & Scott, P. (2009). *The Chemical Composition of the Sun*. Annual Review of Astronomy and Astrophysics, 47, 481-522. Available at https://arxiv.org/abs/0909.0948.

Bodenheimer, P. (2011). *Principles of Star Formation*. Springer Berlin Heidelberg.

Bromm, V., Coppi, P. S., & Larson, R. B. (1999). *Forming the First Stars in the Universe: The Fragmentation of Primordial Gas*. The Astrophysical Journal, 527(1), L5-L8. Available at https://arxiv.org/abs/astro-ph/9910224.

Bromm, V., Ferrara A., Coppi, P. S., & Larson (2001). *The Fragmentation of Pre-enriched Primordial Objects*. Monthly Notices of the Royal Astronomical Society, 328(3), 969-976. Available at https://arxiv.org/abs/astro-ph/0104271.

Jeans, J. H. (1902). *The Stability of a Spherical Nebula.* Philosophical Transactions of the Royal Society of London. Series A, Containing Papers of a Mathematical or Physical Character, 199, 312-320.

Pagel, B. E. J. (2009). *Nucleosynthesis and Chemical Evolution of Galaxies* (2nd ed.). Cambridge University Press.

Schneider, R., Salvaterra, R., Ferrara, A., & Ciardi, B. (2006). *Fragmentation of gas clouds enriched by first stars.* Monthly Notices of the Royal Astronomical Society, 369(2), 825-834. Available at https://arxiv.org/abs/astro-ph/0603766.

Sutherland, R. S., & Dopita, M. A. (1993). *Cooling functions for low-density astrophysical plasmas.* The Astrophysical Journal Supplement Series, 88, 253-327.

Toro, E. F. (2009). *Riemann Solvers and Numerical Methods for Fluid Dynamics: A Practical Introduction* (3rd ed.). Springer.

Wiersma, R. P. C., Schaye, J., & Smith, B. D. (2009). *The effect of photoionization on the cooling rates of enriched, astrophysical plasmas.* Monthly Notices of the Royal Astronomical Society, 393(1), 99-107. Available at https://arxiv.org/abs/0807.3748.

Haardt, F., & Madau, P. (2001). *Modelling the UV/X-ray cosmic background with CUBA.*

Benítez-Llambay, A., & Frenk, C. S. (2020). *The detailed structure and the onset of galaxy formation in low-mass gaseous dark matter haloes.* Monthly Notices of the Royal Astronomical Society, 498(4), 4887–4900.

Meshless Finite Mass (MFM) Hydrodynamics

Historically, cosmological hydrodynamics codes have relied on two philosophies: Lagrangian (particle-based) and Eulerian (grid-based) methods.

- **Lagrangian methods**, most prominently smoothed particle hydrodynamics (SPH), discretize the fluid into particles that move with the flow, guaranteeing Galilean invariance and automatic spatial adaptivity. However, SPH has well-documented deficiencies: an intrinsic zeroth-order error at contact discontinuities (the "E0 error"), an artificial surface tension that suppresses fluid instabilities, and a dependence on artificial viscosity that degrades accuracy in subsonic turbulent flows.
- **Eulerian methods**, exemplified by adaptive mesh refinement (AMR), are mesh-based approaches where the volume is discretized into points or cells, and the fluid equations are solved across these elements. These methods employ Godunov-type finite-volume schemes and achieve high-order convergence on smooth flows with sharp shock capturing, but they suffer from advection errors that scale with the bulk velocity of the fluid relative to the grid and break Galilean invariance.

The **Meshless Finite Mass (MFM)** method was developed to capture the advantages of both Lagrangian and Eulerian schemes. Implemented in the cosmological code GIZMO (built upon the framework of GADGET-3), MFM utilizes a Riemann solver to calculate fluxes (traditionally reserved for grid-based Eulerian codes), but applies it over a meshless, Lagrangian distribution of fixed mass particles that move through space under the influence of gravity, much like the collisionless particles used to model Dark Matter.

The MFM method is defined by three interdependent components: a kernel-based volume partition with a continuous density estimate, a least-squares matrix gradient estimator for second-order spatial reconstruction, and a Riemann solver that computes Godunov-type fluxes across the effective faces between particles.

Volume Partitioning and Density

In MFM the simulation volume is partitioned using a continuous, spherical "kernel" function centered on each particle, which smoothly overlaps with its neighbors.

A differential ν-dimensional volume $d^\nu x$ at any arbitrary coordinate x is fractionally distributed among the nearest particles using a continuous weighting function. The fraction of the volume associated with particle i is defined as $\psi_i(x)$:

$$\psi_i(x) \equiv \frac{1}{\omega(x)} W(x - x_i, h(x))$$

Here, W is the kernel function, and $h(x)$ is the kernel size (often referred to as the smoothing length, and evaluated at the particle center as h_i). We can think of $h(x)$ as defining the particle's sphere of influence—the

radial distance beyond which the kernel evaluates to zero and the particle no longer "sees" or interacts with its surroundings.

To ensure that the sum of all fractional weights equals unity at every point, the weights are normalized by the function $\omega(x)$, which is the sum of the kernel contributions from all neighboring particles j:

$$\omega(x) \equiv \sum_j W(x - x_j, h(x))$$

To achieve second-order accuracy, the conservation of linear and angular momentum, and the locality of the hydrodynamic operations, W must be continuous, have compact support (meaning $W = 0$ when $|x - x_i| \gg h(x)$), and be symmetric. We'll use the 3D cubic spline kernel, defined as a function of the normalized distance $q \equiv |x - x_i|/h_i$:

$$W(q, h_i) = \frac{8}{\pi h_i^3} \begin{cases} 1 + 6q^2(q-1) & (0 \le q < \frac{1}{2}) \\ 2(1-q)^3 & (\frac{1}{2} \le q < 1) \\ 0 & (q \ge 1) \end{cases}$$

$h(x)$ is determined dynamically by constraining it against the smoothed particle number density, n_i, which makes the resolution adaptive. Note that n_i is not a discrete integer count of particles, but rather the continuous sum of the fractional kernel weights from all overlapping neighbors. The code solves for h_i iteratively to maintain a constant "effective" neighbor number, N_{NGB}. The formula used to define and solve for h_i in ν dimensions is:

$$N_{NGB} = C_\nu h_i^\nu n_i = C_\nu h_i^\nu \sum_j W(x_i - x_j, h_i)$$

Where:

- C_ν is the volume normalization constant for a ν-dimensional sphere (1, π, and $4\pi/3$ for 1D, 2D, and 3D simulations, respectively).
- n_i is the smoothed particle number density, calculated by summing the kernel weights of the interacting particles.
- N_{NGB} is the target effective neighbor number chosen for the simulation (e.g., typically set to 32 when using the 3D cubic spline kernel in the MFM method).

Because this formulation relies on the particle positions to estimate the number density rather than the fluid mass, it avoids dependencies on local fluid properties and guarantees that the kernel length remains continuous across the flow.

The **effective volume** V_i of particle i is found by integrating its volume partition over all of space:

$$V_i = \int \psi_i(x) d^\nu x$$

For a general function where h varies with position, it may not be possible to solve this integral analytically. However, by Taylor-expanding the terms, the volume can be approximated to second-order accuracy as:

$$V_i = \omega(x_i)^{-1}(1 + \mathcal{O}(h^2))$$

This expression becomes exact if the kernel length h remains locally constant across the kernel domain. For this reason, we use a constant smoothing length h_i for each particle, making $h(x) = h_i$ over its entire local domain.

The density of the particle is then its fixed mass divided by this effective volume:

$$\rho_i = \frac{m_i}{V_i}$$

Gradient Estimation and Spatial Reconstruction

To achieve second-order spatial accuracy, the MFM method must reconstruct the fluid variables at the effective face from the particle-centered values. The MFM method uses a matrix gradient estimator. The gradient of a quantity q at particle i is obtained by a least-squares fit over all neighbors within the kernel support:

$$(\nabla q)_i = \mathbf{B}_i \sum_j (q_j - q_i)\mathbf{x}_{ij} V_j W(|\mathbf{x}_{ij}|, h_i)$$

where $\mathbf{x}_{ij} = \mathbf{x}_j - \mathbf{x}_i$ and $\mathbf{B}_i$ is the inverse of the moment matrix:

$$\mathbf{B}_i = \mathbf{E}_i^{-1}$$

$$\mathbf{E}_i = \sum_j (\mathbf{x}_{ij} \otimes \mathbf{x}_{ij}) V_j W(|\mathbf{x}_{ij}|, h_i)$$

This estimator is exact for linear functions — it returns the correct gradient for any field of the form $q(\mathbf{x}) = a + \mathbf{b}\cdot\mathbf{x}$ — regardless of the particle distribution.

To determine whether the moment matrix $\mathbf{E}_i$ can be reliably inverted, we evaluate its condition number, defined as:

$$N_{\text{cond}} = \frac{1}{d}\sqrt{\|\mathbf{E}_i\|^2 \|\mathbf{B}_i\|^2}$$

where $d = 3$ is the number of spatial dimensions.

If specific pathological particle configurations appear (for example, if all particles in the kernel align along a single axis), the condition number will exceed a safe tolerance. When N_{cond} surpasses a critical threshold (typically between 100 and 1000), the matrix is considered ill-conditioned. To handle these situations, we fall back to using standard SPH gradient estimators for that particle and timestep. For a generic quantity q, this SPH gradient estimator is defined as:

$$(\nabla q)_i^{\text{SPH}} = \sum_j \frac{1}{\omega_j} q_j \nabla_i W_{ij}(h_i)$$

where ω_j represents the continuous normalizing weight (effectively the number density) evaluated at particle j, and $\nabla_i W_{ij}(h_i)$ is the gradient of the smoothing kernel evaluated with respect to the coordinates of particle i. In the numerical implementation, utilizing the discrete effective volume V_j associated with the neighbor and subtracting the central value to ensure the gradient of a constant field vanishes, this is evaluated as:

$$(\nabla q)_i^{\text{SPH}} = \sum_j V_j (q_j - q_i) \nabla_i W(|\mathbf{x}_{ij}|, h_i)$$

With the gradient in hand, the primitive variables (density, velocity, pressure) are extrapolated to the effective face using a MUSCL-type reconstruction:

$$q_L = q_i + \phi_i (\nabla q)_i \cdot (\mathbf{x}_{\text{face}} - \mathbf{x}_i)$$

where q_L is the left state at the face, ϕ_i is a slope limiter that prevents spurious oscillations near discontinuities, and $\mathbf{x}_{\text{face}}$ is the position of the effective face. The right state q_R is obtained analogously from particle j.

The original theoretical formulation of MFM outlines a slope-limiter procedure employing a two-step limiting process:

- **Gradient Limiter:** First, a scalar slope limiter restricts the gradients to ensure that linearly reconstructed values at the faces do not exceed the maximum or minimum values found among a particle's interacting neighbors. The "true" gradient $(\nabla\phi)^i_{true}$ of a quantity ϕ is replaced by an effective limited gradient $(\nabla\phi)^i_{lim}$:

$$(\nabla\phi)^i_{lim} = \alpha_i (\nabla\phi)^i_{true}$$

where the scaling factor α_i is defined as:

$$\alpha_i \equiv \min\left[1, \beta_i \min\left(\frac{\phi^{max}_{ij,ngb} - \phi_i}{\phi^{max}_{ij,mid} - \phi_i}, \frac{\phi_i - \phi^{min}_{ij,ngb}}{\phi_i - \phi^{min}_{ij,mid}}\right)\right]$$

Here, $\phi^{max}_{ij,ngb}$ and $\phi^{min}_{ij,ngb}$ are the maximum and minimum values among all neighbors j of particle i, while $\phi^{max}_{ij,mid}$ and $\phi^{min}_{ij,mid}$ are the maximum and minimum values reconstructed at the interfaces. The parameter β_i (typically between 1 and 2) determines how aggressively the limiter acts.

- **Pairwise Limiter:** Second, an additional pairwise limiter is applied to the reconstructed states at the interface between interacting particles i and j. This step guarantees stability in extreme situations, such as very strong shocks, by enforcing strict monotonicity bounds (ensuring that the reconstructed interface values never overshoot or undershoot the actual values of the two interacting particles), parameterized by tunable constants. The initial interface estimate $\phi^0_{ij,mid}$ is replaced by a limited value $\phi'_{ij,mid}$ based on the bounds of the interacting pair:

$$\phi'_{ij,mid} = \begin{cases} \phi_i & (\phi_i = \phi_j) \\ \max(\phi_-, \min[\overline{\phi}_{ij} + \delta_2, \phi^0_{ij,mid}]) & (\phi_i < \phi_j) \\ \min(\phi_+, \max[\overline{\phi}_{ij} - \delta_2, \phi^0_{ij,mid}]) & (\phi_i > \phi_j) \end{cases}$$

where $\overline{\phi}_{ij}$ represents the linearly interpolated value between the particles:

$$\overline{\phi}_{ij} \equiv \phi_i + \frac{|x_{ij} - x_i|}{|x_j - x_i|}(\phi_j - \phi_i)$$

The parameters $\delta_1 \equiv \psi_1|\phi_i - \phi_j|$ and $\delta_2 \equiv \psi_2|\phi_i - \phi_j|$ scale the allowed deviations using the tunable constants ψ_1 and ψ_2, while ϕ_- and ϕ_+ bound the allowable overshoots based on δ_1.

While the theoretical formulation of the MFM method describes the two-step pairwise slope limiting procedure above (Hopkins 2015, Appendix B), in the actual GIZMO implementation, gradient limiting is performed using a single-step vector magnitude limiter. Rather than evaluating reconstructed interface states individually with pairwise strictly monotonic bounds, the gradient magnitude $\|\nabla q\|_i$ is scaled directly to ensure that the maximum variation over a characteristic distance $\alpha_{\rm lim} \cdot r_{\rm max}$ does not exceed the minimum absolute deviation to any neighbor:

$$(\nabla q)_{{\rm lim},i} = (\nabla q)_i \cdot \min\left(1, \frac{\min(|\Delta q_{\rm min}|, |\Delta q_{\rm max}|)}{\alpha_{\rm lim} \cdot r_{\rm max} \cdot \|(\nabla q)_i\|}\right)$$

where $\Delta q_{\rm min} = \min_j(q_j - q_i)$, $\Delta q_{\rm max} = \max_j(q_j - q_i)$, $r_{\rm max}$ is the maximum neighbor distance, and $\alpha_{\rm lim} \in [0.25, 0.5]$ is a parameter controlling limiter aggressiveness (with 0.5 providing strict monotonicity). This approach avoids an additional loop over all neighbors.

Effective Faces

The effective face area $\mathbf{A}_{ij}$ between particles i and j, wich enables a Godunov-type scheme, is defined as:

$$\mathbf{A}_{ij} = V_i \tilde{\psi}_j(\mathbf{x}_i) - V_j \tilde{\psi}_i(\mathbf{x}_j)$$

Here, $\tilde{\psi}_j(\mathbf{x}_i) \equiv \mathbf{B}_i(\mathbf{x}_j - \mathbf{x}_i)\psi_j(\mathbf{x}_i)$ represents a matrix-conditioned spatial weight (conditioning a vector means applying a transformation to correct for biases, skewness, or scaling in the raw data), where $\psi_j(\mathbf{x}_i)$ is the scalar volume fraction particle j contributes at $\mathbf{x}_i$. The effective face area vector $\mathbf{A}_{ij}$ is directed from particle i to particle j, and its magnitude $|\mathbf{A}_{ij}|$ plays the role of the face area in the flux computation.

To evaluate the fluxes across this interface, the Riemann problem must be solved at a specific spatial location $\mathbf{x}_{\rm face}$ along the axis connecting the two particles. The second-order accurate quadrature point (the chosen point to approximate the value of an integral) is the location where the volume partition between the two particles is

equal. Because the kernel function is spherically symmetric, this point is where the fractional distance relative to each particle's smoothing length is equal, given by:

$$\mathbf{x}_{\text{face}} = \mathbf{x}_i + \frac{h_i}{h_i + h_j}(\mathbf{x}_j - \mathbf{x}_i)$$

In practice, however, using the first-order geometric midpoint $\mathbf{x}_{\text{face}} = (\mathbf{x}_i + \mathbf{x}_j)/2$ yields nearly identical results in standard test problems and can enhance numerical stability.

The Riemann Solver and Flux Computation

The reconstructed left and right states at the effective face form a Riemann problem: two constant states separated by a discontinuity at $t = 0$. The MFM method evaluates this Riemann problem in a frame moving at the speed of the contact wave. Because the effective face moves with the fluid, there is no mass flux across the face.

The Riemann solver provides the contact pressure (P^*) and the velocity of the face (S^*). Particles interact through mechanical forces. The contact pressure acts over the effective face area to exert a force, transferring momentum between the particles (a push). Simultaneously, because this pressure force is applied to a moving face, it computes the exact PdV mechanical work being done as the particles compress or expand each other's volumes, which updates their internal energies.

The method uses an HLLC approximate Riemann solver. The flux vector across the face is computed as:

$$\mathbf{F}_{ij} = \tilde{\mathbf{F}}_{\text{HLLC}}(q_L, q_R) \cdot \mathbf{A}_{ij}$$

where $\mathbf{A}_{ij}$ is the effective area vector of the face between particles i and j and $\sim$ is used to denote a numerical flux. The fluxes being exchanged are momentum and internal energy.

The semi-discrete conservation equations for a particle are then:

$$\frac{d\mathbf{U}_i}{dt} = -\sum_j \mathbf{F}_{ij}$$

where $\mathbf{U}_i = (\mathbf{P}_i, U_i)$ is the vector containing the particle's momentum and internal energy. Because the fluxes are antisymmetric ($\mathbf{F}_{ij} = -\mathbf{F}_{ji}$), the scheme manifestly conserves momentum to machine precision.

Dual Energy Formalism

Because the total energy in highly supersonic flows is dominated by kinetic energy, deriving the smaller internal (thermal) energy by subtracting the kinetic energy from the total energy leads to massive truncation errors.

To resolve this, the MFM implementation employs a dual energy formalism. Rather than relying on the total energy to compute thermodynamics, the internal energy U is evolved alongside the other conserved quantities. Using the momentum and energy fluxes provided by the Riemann solver, the time rate of change for the internal energy is integrated as:

$$\frac{dU}{dt} = \frac{dE}{dt} - \mathbf{v} \cdot \frac{d\mathbf{P}}{dt}$$

where E is the total energy and $\mathbf{P}$ is the momentum. Since MFM doesn't allow inter-particle mass fluxes, this is equivalent to accumulating the PdV work done by the fluxes at the effective faces.

When trusting this integrated internal energy to determine the local pressure, the method sacrifices total energy conservation in favor of maintaining accurate temperatures in hypersonic and gravity-dominated flows. For these cases, an entropy-based fallback switch is implemented.

At each timestep, the expected thermal energy of a particle is compared against the maximum kinetic energy of its interacting neighbors and the work done by local gravitational forces. If the thermal energy falls below a conservative threshold (typically $\approx 0.1\%$) of these kinetic or gravitational energies, the Riemann solver's internal energy update is bypassed. Instead, the thermal energy is calculated as if the flow were undergoing purely adiabatic expansion, utilizing a passively tracked entropy variable (S).

The entropic function, S, dictates the relationship between pressure and density along an adiabatic fluid streamline via $P = S\rho^\gamma$. It is computed from the specific internal energy u and density ρ using the relation

$S = (\gamma - 1)u/\rho^{\gamma-1}$. When the fallback is triggered, the internal energy is overridden and recalculated from the current density using this stored entropy. Under normal shock conditions, this switch remains inactive, and the passive entropy array simply resynchronizes to the new, shock-heated state calculated by the Riemann solver.

Adaptive Gravitational Softening

Particle-based methods couple naturally to N-body gravitational solvers. However, to maintain consistency with the second-order spatial reconstruction of the fluid solver, the gravitational solver must also account for the distributed mass of the gas at second order. For an unstructured particle distribution (where mass elements move freely rather than sitting on a fixed, regular grid), this means the gravitational softening must be adaptive and set equal to the smoothing length used for the hydrodynamic computations. This ensures that both solvers share an identical resolution limit and operate on a unified definition of the fluid volume.

Continuous Mass Distribution

In MFM, the differential mass at a point x associated with a given particle i is defined by the volume partition:

$$dm_i = d^\nu x \rho(x) \frac{W(x - x_i, h(x))}{\omega(x)}$$

Expanding this to leading order (i.e., zeroth order, treating the fluid properties as constant across the volume of a particle) in the gradients of the density ρ and the smoothing length h, the density distribution associated with a particle takes the same form as the kernel centered on that particle:

$$dm_i \approx m_i W(x - x_i, h_i) d^\nu x$$

This way, we treat the fluid cells in the N-body solver as standard particles "softened" by the same kernel function used for the hydrodynamics, matching the kernel length h_i. On scales larger than h_i, the potential and the force match that of a Newtonian point mass. Inside the kernel radius, the gravitational potential $\Phi_i \equiv G m_i \phi_i$ (where ϕ_i represents the gravitationally softened potential) is computed by integrating Poisson's equation over the kernel mass distribution.

Conservative Force Law

The kernel lengths h_i change as particles move closer together or further apart. We must maintain momentum and energy conservation when dealing with variable softening lengths. If we define the gravitational self-energy of the gas (the gravitational potential energy of the entire system of gas particles) as $E_{grav} = \frac{1}{2} \sum_{i,j} G m_i m_j \phi(r_{ij}, h_j)$, we can derive the resulting forces.

The conservative gravitational acceleration for particle i interacting with particle j is given by:

$$m_i \frac{dv_i}{dt}\bigg|_{grav} = -\nabla_i E_{grav} = \mathbf{F}_{\text{sym}} + \mathbf{F}_{\text{corr}}$$

This splits the force into two components: the symmetrized Newtonian force ($\mathbf{F}_{\text{sym}}$) and the softening variation correction ($\mathbf{F}_{\text{corr}}$). The full expression is:

$$m_i \frac{dv_i}{dt}\bigg|_{grav} = -\sum_j \frac{G m_i m_j}{2} \left(\frac{\partial \phi(r, h_i)}{\partial r}\bigg|_{r_{ij}} + \frac{\partial \phi(r, h_j)}{\partial r}\bigg|_{r_{ij}} \right) \frac{\mathbf{r}_{ij}}{r_{ij}} - \sum_j \frac{G}{2} \left(\zeta_i \frac{\partial W(r, h_i)}{\partial r}\bigg|_{r_{ij}} + \zeta_j \frac{\partial W(r, h_j)}{\partial r}\bigg|_{r_{ij}} \right) \frac{\mathbf{r}_{ij}}{r_{ij}}$$

Where $\mathbf{r}_{ij} = \mathbf{x}_i - \mathbf{x}_j$.

- **The Symmetrized Newtonian Force:** The first term containing $\frac{\partial \phi}{\partial r}$ is the modified $1/r^2$ gravitational force. Because interacting particles i and j often have different smoothing lengths, the force is symmetrized by averaging the potential derivatives, guaranteeing that Newton's third law is obeyed.
- **The ζ Correction Term:** The second term containing $\frac{\partial W}{\partial r}$ accounts for the temporal and spatial derivatives of the kernel lengths. By moving the particles, the simulation changes the local density, which modifies the gravitational potential. This means additional work is done by the expanding or contracting potentials. Note that the derivative of the kernel $\frac{\partial W}{\partial r}$ conveniently evaluates to zero for distant particles outside the kernel radius.

Computing the ζ (Zeta) Coefficients

The correction factors ζ_a are evaluated for each particle using the dimensionless term Ω_a, which tracks how the density changes relative to the smoothing length. The formulation is defined as:

$$\zeta_a \equiv m_a \frac{h_a}{n_a \nu} \frac{1}{\Omega_a} \sum_b m_b \frac{\partial \phi(r_{ab}, h_a)}{\partial h_a}\bigg|_{h_a}$$

$$\Omega_a \equiv 1 + \frac{h_a}{n_a \nu} \frac{\partial n_a}{\partial h_a} = 1 - \frac{h_a}{n_a \nu} \sum_b \left(\frac{r_{ab}}{h_a} \frac{\partial W(r, h_a)}{\partial r}\bigg|_{r_{ab}} + \frac{\nu}{h_a} W(r_{ab}, h_a) \right)$$

where ν is the number of spatial dimensions.

In code implementations, evaluating these terms fits into the existing MFM density iteration loop. During the solver that determines the smoothing length h_i, the term $\frac{\partial n_i}{\partial h_i}$ is already computed to find the root. Once h_i converges, the code performs one final pass over the particle's neighbors to sum $m_b \frac{\partial \phi}{\partial h_a}$, finalizing ζ_i.

During the subsequent gravity loop, these pre-computed ζ values are utilized alongside the analytical derivatives of the specific kernel (e.g., the cubic spline) to resolve the conservative gravitational forces between all gas elements. For interactions between gas and collisionless components (like Dark Matter), the collisionless particles act as point masses that sample the adaptive potential of the gas without generating their own ζ variations.

Initializing the Meshless Finite Mass (MFM) Gas

The initial conditions for the MFM gas particles are generated using the same Zel'dovich Approximation displacement field used for the Dark Matter. However, there are some differences.

The Interleaved Lattice

Before applying the Zel'dovich perturbations, particles must be arranged on a uniform grid. If both the Dark Matter and MFM gas particles were placed on the same lattice points, their centers of mass would overlap. This would result in immediate numerical instability.

To solve this, we interleave the two particle distributions:

- **Dark Matter:** Placed at half-integer grid coordinates, e.g., $q_x = (i + 0.5)\Delta x$.
- **MFM Gas:** Placed at integer grid coordinates, e.g., $q_x = i\Delta x$.

This shifts the gas lattice relative to the Dark Matter, ensuring physical separation at $t = 0$.

Displacements and Peculiar Velocities

Once the unperturbed coordinates are established, the MFM particles are displaced like the Dark Matter.

For a gas particle at an initial lattice position $\mathbf{q}$, the continuous 3D Zel'dovich displacement field ($\mathbf{d}$) is sampled and the particle's properties are updated:

- **Position:** The Zel'dovich displacement is added to the unperturbed coordinate, wrapped by the periodic boundaries of the domain:

$$\mathbf{x} = (\mathbf{q} + \mathbf{d}) \ (\mod L_{box})$$

- **Velocity:** The peculiar velocity is assigned by scaling the displacement vector by the Hubble parameter (H) and the dimensionless linear growth rate (f):

$$\mathbf{v} = H\mathbf{d}f$$

Thermodynamic and MFM Initialization

The MFM particles represent a pressure-bearing fluid. After their spatial coordinates and velocities are set, they are initialized with the following properties:

- **Internal Energy:** A user-defined initial background temperature (T_{init}) is assigned uniformly to all gas particles.

- **Metallicity:** The mass fraction of heavy elements is initialized uniformly to a predefined seed value.
- **Smoothing Length:** Because the MFM density solver relies on finding a number of neighbors within a kernel radius (h), an initial guess must be provided. We set this initial smoothing length to $h = 1.2\Delta x$. Making the kernel slightly larger than the initial particle spacing guarantees sufficient overlap between neighboring particles to allow the adaptive root-finding algorithm to converge during the first density calculation.

Key Literature & Further Reading
Hopkins, P. F. (2014). *A New Class of Accurate, Mesh-Free Hydrodynamic Simulation Methods.* MNRAS, 450, 53, (2015). Available at https://doi.org/10.1093/mnras/stv195

Groth, F., Steinwandel, U. P., Valentini, M. & Dolag, K. (2023). *The cosmological simulation code OpenGadget3 – implementation of meshless finite mass.* MNRAS, (2023). Available at https://doi.org/10.1093/mnras/stad2717

Validation of the Hydrodynamic Solver

A hydrodynamic solver is typically validated by its ability to reproduce the known analytical solutions to a set of classic test problems. For all the tests mentioned in this section, the gravitational solver must be turned off.

Conservation in a Closed Box

This test ensures the solver conserves all quantities in the absence of external forces.

- **The Setup:** A periodic, non-expanding box is initialized with a random distribution of gas densities, pressures, and velocities.
- **The Validation:** The simulation is run for many time steps. At each step, the following total quantities, summed over all grid cells, must remain constant to machine precision:
  1. **Total Mass:** $M_{total} = \sum_i \rho_i L^3$
  2. **Total Momentum:** $\mathbf{P}_{total} = \sum_i (\rho \mathbf{v})_i L^3$
  3. **Total Energy:** $E_{total} = \sum_i E_i L^3$
- **What it Proves:** This test confirms that the flux calculations are balanced—that any mass, momentum, or energy that leaves one cell correctly enters its neighbor, with no numerical "leaks" or "sources."

The Linear Soundwave Test

This test is well suited to perform a convergence analysis for a hydrodynamic solver.

- **The Setup:** A sinusoidal soundwave is initialized with a background density of 1 and a small perturbation amplitude of 10^{-4}. The test utilizes a wavenumber of $k = 2\pi$ and a speed of sound of $c_s = 2/3$. The simulation domain is configured as a box with a length of 1 in the x-direction and 0.75 in both the y and z-directions. The particles are arranged in a perturbed hexagonal close packed (hpc) grid using varying resolutions.
- **The Expected Result:** The wave propagates through the periodic domain.
- **The Validation:** The overall accuracy of the solver is validated by measuring the L1 error norm, calculated as

$$\frac{1}{N_{tot}} \sum_{i}^{N_{tot}} |\rho_i - \rho(x,t)|$$

.

Analytical Solution

Because the perturbation is small, an analytical solution exists for all fluid variables. For a left-propagating wave, the phase is determined by $k(x + c_s t)$.

- **Density:** The fluid density evolves according to the equation $\rho(x,t) = \rho_0 + \Delta\rho \sin(k(x + c_s t))$.
- **Velocity:** The velocity perturbation for the left-moving wave is defined as $v_x = -c_s(\Delta\rho/\rho_0)\sin(k(x+c_s t))$.
- **Pressure and Energy:** The pressure perturbation follows $P = P_0 + c_s^2 \Delta\rho \sin(k(x+c_s t))$, and the specific internal energy is defined as $u = P/((\gamma - 1)\rho)$.

Convergence and Error Sources

Evaluating the L1 error across different resolutions reveals the order of convergence for the numerical scheme.

- **Convergence Rate:** While a solver might theoretically be designed for second-order accuracy, the application of a slope-limiter reduces the actual order of convergence. For Meshless Finite Mass (MFM) implementations, this convergence typically falls between first and second order.
- **Dispersion Errors:** Deviations from the expected sound speed are related to dispersion errors, which manifest as a spatial offset compared to the analytical solution.
- **Diffusion Errors:** Errors in the measured amplitude of the soundwave are related to the numerical diffusion introduced by the solver.
- **Scatter:** Scatter errors will also be present in the final density distribution.

The 1D Shock-Tube

This test has a known analytical solution (the **Sod Shock Tube**, named after Gary A. Sod, is the most famous variant) that validates the code's ability to handle all three fundamental wave structures.

- **The Setup:** A 1D tube of gas is initialized with a "diaphragm" at its center. The gas on the "Left" state has a high density and pressure, while the gas on the "Right" state has a low density and pressure. At $t = 0$, the diaphragm is removed.
- **The Expected Result:** The collision of the two states generates a self-similar wave structure (meaning its shape remains constant as it stretches over time). This structure is complex, splitting into several distinct features (see below).
- **The Validation:** After evolving the system to a time t, a snapshot of the simulation's density, pressure, and velocity along the 1D line is plotted. This must be compared against the known analytical solution (see below).

Physical Assumptions of the Analytical Model

The model enforces the following assumptions:

- **Ideal, Adiabatic Gas:** The fluid obeys the ideal gas equation of state without any radiative cooling, heating, or thermal conduction.
- **Pure Hydrodynamics:** The system is isolated. No external body forces, such as gravity or cosmological expansion, are present.
- **Initial States:** The fluid begins at rest ($v_L = v_R = 0$). The left state is initialized with $P_L = 1.0$ and $\rho_L = 1.0$, while the right state is set to $P_R = 0.1$ and $\rho_R = 0.125$.

The Analytical Solution and Wave Regions

The removal of the diaphragm divides the 1D space into five regions, separated by moving boundaries. Let $c_L = \sqrt{\gamma P_L/\rho_L}$ represent the initial speed of sound in the unperturbed left state.

- **Region 1: Unperturbed Left State:** This is the gas that the rarefaction wave has not yet reached ($x \leq x_0 - c_L t$). The fluid remains at its initial state ρ_L, P_L, and v_L.
- **Region 2: The Rarefaction Fan:** An isentropic (meanning that a process occurs at constant entropy, so pressure and density become locked by the polytropic relation: $P \propto \rho^\gamma$), self-similar expansion wave. The fluid variables change continuously. The local velocity expands according to $v = \frac{2}{\gamma+1}\left(c_L + \frac{x-x_0}{t}\right)$. As the gas accelerates and expands, the local sound speed drops to $c_{fan} = c_L - 0.5(\gamma - 1)v$. Density and pressure follow smooth isentropic relations: $\rho = \rho_L(c_{fan}/c_L)^{\frac{2}{\gamma-1}}$ and $P = P_L(c_{fan}/c_L)^{\frac{2\gamma}{\gamma-1}}$.
- **Region 3: Post-Fan Plateau (Star State Left):** The fully expanded gas behind the contact discontinuity. The state sits at constant intermediate values ρ_3, P_*, and u_*.
- **Region 4: Post-Shock Plateau (Star State Right):** The abruptly compressed gas immediately ahead of the contact discontinuity. Across a contact discontinuity, pressure and velocity must balance to avoid infinite accelerations, so $P = P_*$ and $v = u_*$. However, the density jumps to a new value, ρ_4.
- **Region 5: Unperturbed Right State:** The gas ahead of the primary shock wave ($x > x_0 + V_{shock}t$). The state remains at ρ_R, P_R, and v_R.

Deriving the Intermediate "Star" States

The analytical solution relies in finding the thermodynamic properties of the intermediate "Star" region (Regions 3 and 4).

1. The Intermediate Pressure (P_*) Across the contact discontinuity, pressure and velocity are continuous ($P_3 = P_4 = P_*$ and $u_3 = u_4 = u_*$). The pressure P_* is found by equating the velocity change across the left rarefaction wave to the velocity change across the right shock wave. For initial rest states, this yields a non-linear equation:

$$f(P_*) = f_L(P_*, W_L) + f_R(P_*, W_R) = 0$$

Where f_L represents the isentropic expansion and f_R represents the shock jump conditions. This is determined by comparing our guess for P_* against the initial pressure P_K:

- **Rarefaction Wave (Isentropic Expansion, $P_* \leq P_K$)**: If the intermediate pressure is lower than the initial pressure, the gas expands smoothly. The change in velocity is governed by the conservation of Riemann invariants along characteristics and the isentropic relation ($P/\rho^\gamma = \text{const}$), yielding:

$$f_K(P_*, W_K) = \frac{2c_K}{\gamma - 1}\left(\left(\frac{P_*}{P_K}\right)^{\frac{\gamma-1}{2\gamma}} - 1\right)$$

  where $c_K = \sqrt{\gamma P_K/\rho_K}$ is the sound speed of the unperturbed gas.

- **Shock Wave ($P_* > P_K$)**: If the intermediate pressure is higher, a shock compresses the gas abruptly. The velocity change is governed by the Rankine-Hugoniot jump conditions, which enforce mass, momentum, and energy conservation across the discontinuous shock front:

$$f_K(P_*, W_K) = (P_* - P_K)\sqrt{\frac{A_K}{P_* + B_K}}$$

  where the data-dependent constants are defined as $A_K = \frac{2}{(\gamma+1)\rho_K}$ and $B_K = \frac{\gamma-1}{\gamma+1}P_K$.

  Because this piecewise function $f(P_*)$ must be found using an iterative numerical root-finding method, such as the Newton-Raphson method.

2. The Intermediate Velocity (u_*) With P_* known, the post-shock velocity is derived from the momentum jump conditions across the shock front:

$$u_* = (P_* - P_R)\left(\frac{1-\mu^2}{\rho_R(P_* + \mu^2 P_R)}\right)^{1/2}$$

where $\mu^2 = (\gamma - 1)/(\gamma + 1)$.

3. The Densities (ρ_3 and ρ_4) The two densities in the star region are calculated using different thermodynamic laws, reflecting their distinct physical histories:

- **Region 3 (ρ_3):** This gas expanded smoothly, undergoing a reversible, isentropic process. Using the relationship $P/\rho^\gamma = \text{constant}$, the density evaluates to $\rho_3 = \rho_L(P_*/P_L)^{1/\gamma}$.
- **Region 4 (ρ_4):** This gas was abruptly compressed by a shock, which is an entropy-generating process. It is governed by the Rankine-Hugoniot jump conditions: $\rho_4 = \rho_R \frac{P_* + \mu^2 P_R}{P_R + \mu^2 P_*}$.

4. The Shock Velocity (V_{shock}) The propagation speed of the shock wave into the unperturbed right state is dictated by the mass and momentum fluxes across the shock front. Given the sound speed of the unperturbed right gas ($c_R = \sqrt{\gamma P_R/\rho_R}$), the Rankine-Hugoniot relations yield:

$$V_{shock} = c_R\sqrt{\frac{\gamma+1}{2\gamma}\frac{P_*}{P_R} + \frac{\gamma-1}{2\gamma}}$$

The Sedov-Taylor Blast Wave (Point Explosion)

This is a multi-dimensional test for how a code handles a powerful, symmetric explosion.

- **The Setup:** A uniform, low-density gas fills the grid, initially at rest. At $t = 0$, a very large amount of thermal energy is deposited into a central cell.
- **The Expected Result:** A spherical shock wave propagates outwards from the center, sweeping the surrounding gas into a dense, hot shell. Because this explosion creates extreme hypersonic velocities (high Mach numbers), this is a stress-test for the **Dual Energy Formalism** and the **MUSCL slope limiters**, proving the code can handle violent energy conversions.
- **The Validation:** This test has a known self-similar solution and is validated in two ways:
  1. **Symmetry:** The shock front must remain spherical. Any "boxy" or distorted shape indicates that the solver is introducing errors.
  2. **Propagation Speed:** The radius of the shock front, R, must grow with time, t, according to a specific power law. For an explosion in a uniform medium, this is $R(t) \propto t^{2/5}$.

The analytical solution for the Sedov-Taylor blast wave relies on a self-similar scaling that depends on the total injected energy (E), the background gas density (ρ_{bg}), and the time (t). For a 3D expansion, the position of the shock front R_s is given by:

$$R_s(t) = \alpha \left(\frac{E t^2}{\rho_{bg}} \right)^{1/5}$$

where α is a dimensionless constant of order unity that depends on the adiabatic index γ (for $\gamma = 5/3$, $\alpha \approx 1.15$).

Because the background pressure is effectively zero compared to the blast, the expanding boundary is considered an "infinitely strong" shock. The state of the gas immediately behind the shock front (the "peak" values) can be calculated using the strong shock limit of the Rankine-Hugoniot jump conditions:

- **Shock Velocity (D):** Taking the derivative of the radius yields the speed of the shock front itself: $D = \frac{2}{5}\frac{R_s}{t}$.
- **Peak Density:** The density jumps to a maximum compression ratio determined by the thermodynamics: $\rho_{peak} = \rho_{bg}\frac{\gamma+1}{\gamma-1}$. For a monatomic ideal gas ($\gamma = 5/3$), the shell is compressed to 4 times the background density.
- **Peak Gas Velocity:** The physical velocity of the gas particles being swept up is $v_{peak} = \frac{2}{\gamma+1}D$.
- **Peak Pressure:** The maximum thermal pressure occurs at the shock front: $P_{peak} = \frac{2}{\gamma+1}\rho_{bg}D^2$.

Inside the blast wave ($r < R_s$), the fluid variables follow smooth, self-similar curves: the density plunges rapidly toward zero near the origin, while the specific internal energy spikes, creating a nearly empty, hot cavity surrounded by a cold, dense, and fast-moving shell.

The Kelvin-Helmholtz Instability

This test measures the code's ability to model fluid mixing and the growth of instabilities.

- **The Setup:** A 2D box is initialized with two layers of fluid sliding past each other in opposite directions. For example, the top half has a velocity $v_x = +v$ and the bottom half has $v_x = -v$. A tiny, sinusoidal perturbation is introduced at the interface.
- **The Expected Result:** The shear at the interface is unstable. The small initial perturbation should grow exponentially, causing the interface to roll up into a characteristic series of vortices.
- **The Validation:** This is a test of the solver's numerical diffusion. Resolving these vortices correctly requires second-order spatial reconstruction and an adequate solver (like HLLC) to avoid introducing too much artificial viscosity.

The Evrard Spherical Collapse

This is a multi-dimensional test for how a code couples hydrodynamics with self-gravity during extreme compression and shock heating.

- **The Setup:** A cold sphere of gas of mass M and radius R is initialized at rest in a vacuum. The gas follows a specific $1/r$ density profile: $\rho(r) = \frac{M}{2\pi R^2 r}$ for $r < R$. The initial thermal energy is set artificially low ($u \ll GM/R$) so that gravity dominates the initial dynamics.

- **The Expected Result:** The gas sphere free-falls inward under its own self-gravity, converting gravitational potential energy into kinetic energy. As the core reaches maximum compression, the central pressure spikes, halting the collapse and driving an outward-propagating shock wave (the "bounce"). This tests the **Gravity-Hydro Coupling** and **Total Energy Conservation**, as the code must accurately track the massive conversion of potential to kinetic, and finally to thermal energy without artificial dissipation.
- **The Validation:** This test lacks an analytical solution for the transient phase, so it is validated against high-resolution 1D reference simulations and theoretical equilibrium limits:

1. **Profile Evolution:** At specific times during the bounce (e.g., $t = 0.8$), the radial profiles of density, velocity, and entropy must align with established 1D high-order benchmarks. The outward-moving shock front must be sharp and correctly positioned.
2. **Global Energy Conservation:** The total energy of the fluid must remain conserved.
3. **Hydrostatic Equilibrium:** The system must eventually settle into a permanent, stable state where the outward thermal pressure balances the inward gravitational pull: $\nabla P = -\rho \nabla \Phi$.

Sources of Error

The Temperature Floor

As the simulation progresses, the expansion of the universe applies adiabatic cooling (PdV work) to the gas. When the gas temperature drops below a configured minimum (like the Cosmic Microwave Background), codes often artificially clamp it. When the temperature is clamped to a floor, thermal energy is injected into the particle. Diagnostic energy computations may perceive this as a steady leak of positive energy.

However, it is possible to track the energy injected by the temperature floor, accounting whenever possible for every drop of artificial energy injected or removed from the system.

The temperature floor in cosmological simulations can have a physical interpretation as photoheating. It is often set as a proxy for the Cosmic Microwave Background (CMB) or the Ultraviolet Background (UVB) reionization. In this case, it is consistent to log this energy injection as a heating source.

The Energy-Entropy Switch (Dual Energy Formalism)

When using the Energy-Entropy Switch (dual energy formalism), as noted in the AREPO paper by Springel (2010), we "temporarily give up manifest conservation of total energy, as the thermal energy is now not defined as a difference between total energy and kinetic energy, but rather based on the value expected for isentropic evolution of the gas" (where *isentropic* means evolving at a constant entropy). By explicitly overwriting the internal energy with the entropy-derived value, the code sacrifices machine-precision energy conservation to ensure the gas temperatures remain physically accurate.

Unlike in the case of the temperature floor, the entropy switch represents a numerical error without a physical interpretation. Therefore, it is not usual to balance out the energy lost or gained by the entropy switch. However, it can be tracked to determine how much it is contributing to the total error and to detect other possible sources.

Adaptive Timestep

Cosmological simulations are filled with different components. Dark matter particles interact only through gravity, a long-range force that can be relatively slow. Baryonic gas, however, interacts through hydrodynamic pressure, leading to shock waves that propagate at very high speeds, and it radiates thermal energy, which can cause temperatures to drop very fast.

This creates a challenge: the simulation evolves on many different timescales simultaneously. If we were to use a single, "fixed" timestep (Δt) for the entire simulation, we would be forced to choose the *smallest* possible timescale. This would make the simulation to waste resources where bigger steps could be safely taken.

A solution to this problem is the **adaptive timestep**. At every cycle, the shortest timescale required to maintain stability for every physical component is computed. The final timestep used to advance the simulation is the minimum of all of these, ensuring both physical accuracy and computational efficiency.

The Courant-Friedrichs-Lewy (CFL) Condition for hydrodynamics

For the grid-based hydrodynamic solver, the most restrictive limit is the **Courant-Friedrichs-Lewy (CFL) condition**. This condition is based on the principle that information (i.e., a sound wave or the fluid itself) must

not be allowed to travel more than one grid cell (Δx) in a single timestep (Δt).

If a parcel of gas moves two cells in one step, the numerical solver for the adjacent cell never "sees" it, leading to a breakdown of the solution.

To enforce this, we must first find the maximum "signal velocity" (v_{signal}) anywhere in the simulation. This is the sum of the bulk fluid velocity (v) and the local sound speed (c_s). The sound speed itself depends on the gas pressure (P) and density (ρ) through the adiabatic index (γ):

$$c_s = \sqrt{\frac{\gamma P}{\rho}}$$

$$v_{\text{signal}} = v + c_s$$

The simulation must find the maximum signal velocity across all N cells, $v_{\max} = \max(v_{\text{signal},i} \text{ for } i \in [1, N])$. The CFL timestep limit is then the time it would take this fastest signal to cross one cell, scaled by a "safety factor" (C_{CFL}, typically 0.1 to 0.5) to ensure stability:

$$\Delta t_{\text{CFL}} = C_{\text{CFL}} \cdot \frac{\Delta x}{v_{\max}}$$

The Cooling Timestep Limiter

The implicit Backward Euler method guarantees that our thermodynamics solver will never crash, even if the timestep is absurdly large. It is unconditionally stable. However, "stable" does not mean "accurate".

If t_{cool} in the center of a dense halo is 10,000 years, and we take a single massive timestep of 10 million years, the implicit solver will safely drop the temperature to the cosmic floor. But in reality, over those 10 million years, that gas might have been hit by a shockwave, compressed further by gravity, or pushed out of the halo entirely. Taking a massive leap forward blinds the simulation to the complex, shifting dynamics of galaxy formation.

To maintain physical accuracy, we must force the simulation to respect the speed of the thermodynamics. We do this by adding a new constraint to our architecture: the **cooling timestep limiter**.

During every cycle, the simulation evaluates the cooling timescale ($t_{\text{cool}} = \frac{\rho u}{\Lambda}$) for every single gas cell on the grid. We then mandate that the global simulation timestep cannot exceed a small safety fraction (typically 10%) of the shortest cooling time found anywhere in the universe:

$$\Delta t_{\text{cool}} = 0.1 \times \min\left(\frac{\rho u}{\Lambda}\right)$$

This forces the simulation to take smaller, more frequent steps whenever a gas cloud enters a phase of violent, runaway cooling. By restricting the clock, we ensure that the implicit solver only ever has to predict cooling over a modest, physically reasonable interval.

The Gravitational Timestep

For the N-body (particle) integrator, a different stability criterion applies. Particles in a strong gravitational field (e.g., near a dense halo or during a close encounter) experience high acceleration. If the timestep is too large, the integrator will "overshoot" the correct trajectory, artificially adding energy to the system and making it unstable.

The principle here is that the timestep must be a small fraction of the local dynamical time, which can be defined as the time it takes a particle to cross the gravitational softening length (ϵ) given its current acceleration.

This is a kinematic constraint. From basic kinematics ($x = \frac{1}{2}at^2$), the time to travel a distance ϵ under acceleration a is $t \sim \sqrt{\epsilon/|a|}$.

To ensure stability for all particles, the simulation must find the maximum acceleration experienced by any particle, $a_{\max} = \max(|a_i|)$. The gravitational timestep limit is then:

$$\Delta t_{\text{grav}} = C_{\text{grav}} \cdot \sqrt{\frac{\epsilon}{a_{\max}}}$$

Where C_{grav} is a dimensionless safety factor (e.g., 0.1 to 0.3) that controls the accuracy of the particle integration.

The Global Timestep

At each cycle, the simulation computes all relevant timestep limits. The **global timestep**, Δt, which will be used to advance *all* components (particles, gas, and thermodynamics) from time t to $t + \Delta t$, must be the single most restrictive (smallest) value.

Furthermore, it is common practice to introduce a user-defined maximum timestep, Δt_{max}. This acts as a "ceiling," preventing the timestep from becoming excessively large in quiet regions of the simulation (which could reduce accuracy) and ensuring that data snapshots are saved at reasonably regular intervals.

The final global timestep is therefore the minimum of all constraints:

$$\Delta t = \min(\Delta t_{\text{CFL}}, \Delta t_{\text{grav}}, \Delta t_{\text{cool}}, \Delta t_{\text{max}})$$

This ensures that the simulation proceeds as fast as possible while respecting the physical constraints present anywhere in the computational domain, guaranteeing that no part of the simulation evolves into an unstable, non-physical state.

Key Literature & Further Reading
Bryan, G. L., Norman, M. L., O'Shea, B. W., et al. (2014). *ENZO: An Adaptive Mesh Refinement Code for Astrophysics*. The Astrophysical Journal Supplement Series, 211(2), 19. arXiv:1307.2265. Available at https://arxiv.org/abs/1307.2265

Subcycled Operator Splitting

In the previous chapters, we proposed the use of an adaptive timestep. To maintain numerical stability, the global simulation clock must tick forward at a rate dictated by the fastest physical process occurring anywhere in the simulation.

However, this creates a computational bottleneck. A simulation consists of multiple physical operators—gravity, hydrodynamics, and thermodynamics. These processes evolve on different timescales. At a given instant, particle-particle (PP) gravitational interactions might require a timestep of just 10,000 years to remain stable. Meanwhile, the hydrodynamics solver might allow a step of 2 million years.

If we force the entire simulation to step together using a single global timestep (the minimum of all constraints), we still waste computational power. We are forcing the hydrodynamics solver to run hundreds of times simply because gravity is acting quickly.

The **Subcycled Operator splitting** is an architectural solution to this timescale mismatch. Instead of locking all physics to the tightest constraint, we decouple the operators. The simulation advances using a large global timestep (the "macro-step") dictated by the *slowest* physics, while the faster, more restrictive physical processes run in a localized loop (the "micro-steps") to catch up.

Adaptive Multirate Operator Splitting

The standard Kick-Drift-Kick (KDK) integrator separates the physics sequentially: it applies a gravitational "Kick" for half a step, a "Drift" for a full step, and then a second "Kick" for a half step. This symmetric $A/2 \to B \to A/2$ structure is mathematically known as **Strang Splitting**, and it guarantees second-order accuracy. Operator subcycling takes this splitting a step further by nesting loops inside the operators without breaking the outer symmetry. A robust way to handle this in a cosmological context is through an **Adaptive Multirate Operator Splitting scheme**.

This dynamic subcycling engine evaluates both the gravitational timestep limit (Δt_{grav}) and the hydrodynamic CFL limit (Δt_{hydro}) at the start of every cycle. It then designates the *larger* of the two as the global **macro-step**, and the *smaller* to be subcycled.

$$\Delta t_{\text{macro}} = \max(\Delta t_{\text{hydro}}, \Delta t_{\text{grav}})$$

This macro-step is then capped by the cosmological expansion limiter (e.g., ensuring the universe expands by no more than 1% per step) and any user-defined maximums to ensure the simulation doesn't skip past critical output snapshots.

- **If Gravity is Slow ($\Delta t_{\mathbf{grav}} > \Delta t_{\mathbf{hydro}}$):** The overall cycle advances by the gravity timestep. Because the gas flows faster than the dark matter, it must take multiple sub-steps. However, if the gas were to drift without feeling gravity for the entire macro-step, it would artificially expand out of dark matter halos. To solve this, we employ a predictor-corrector approach: we compute an approximation of the future gravitational field to guide the gas, and linearly interpolate it, allowing the gas to undergo a full sequence of micro-KDK steps against a smoothly evolving background potential.

    1. **Compute Macro-Forces & Macro-Kick 1:** Calculate (or retrieve) the current gravitational field and save it as our "old" state. Apply the gravity and expansion kicks to the dark matter for $\Delta t_{\text{macro}}/2$.
    2. **Macro-Drift (Dark Matter):** Advance the collisionless dark matter positions by Δt_{macro}.
    3. **Predictor Force Solve:** Advance the global scale factor to the end of the macro-step and solve Poisson's equation. Because the dark matter has moved but the gas has not, this yields an accurate *predictor* of the future gravitational field.
    4. **Interpolated Gas Subcycling:** Enter an iterative subcycling loop. Dynamically re-evaluate Δt_{hydro} based on the shifting gas state, and run the hydrodynamics solver repeatedly until the gas time catches up to Δt_{macro}. Inside this loop, the gas undergoes its own KDK sequence:
        - **Interpolation:** Calculate a blending factor (α) based on the current sub-step time. Linearly interpolate the "old" and "predictor" scale factors, Hubble parameters, and gravitational forces to the midpoint of the sub-step.
        - **Micro-Kick 1 (Gas):** Apply the interpolated gravity and expansion kicks to the gas for $\Delta t_{\text{hydro}}/2$.
        - **Hydro Drift:** Advance the fluid solver (and apply radiative cooling) for Δt_{hydro}.
        - **Micro-Kick 2 (Gas):** Apply the interpolated gravity and expansion kicks to the gas for the remaining $\Delta t_{\text{hydro}}/2$.
    5. **Corrector Force Solve:** With the gas and dark matter now fully synchronized at the end of the macro-step, solve Poisson's equation a second time to obtain the true, final gravitational field.
    6. **Macro-Kick 2 (Dark Matter):** Apply the true, synchronized gravity kick to the dark matter for the remaining $\Delta t_{\text{macro}}/2$.

- **If Gravity is Fast ($\Delta t_{\mathbf{grav}} < \Delta t_{\mathbf{hydro}}$):** The overall cycle advances by the hydrodynamics timestep. To maintain the Strang splitting symmetry, we must split the gravity subcycles into two halves wrapping the single hydro step.

    1. **First Gravity Half-Cycle:** Enter a `while` loop to advance gravity by $\Delta t_{\text{macro}}/2$. Inside this loop, the particles undergo smaller KDK steps (Kick, Drift, Recompute Forces, Kick). Because the universe is expanding, the scale factor (a) and Hubble parameter (H) used for these micro-kicks must be linearly interpolated to the sub-step time. During this phase, the gas density is "frozen", providing a static background potential for the dark matter.
    2. **Macro-Drift (Gas):** The hydrodynamics solver takes a single, large step to advance the gas grid by Δt_{macro}.
    3. **Second Gravity Half-Cycle:** Enter a second `while` loop to advance the gravity by the remaining $\Delta t_{\text{macro}}/2$, using the newly updated, post-hydro gas density to source the Poisson solver.

Decoupling the Thermodynamics

Noticeably absent from the macro-step logic is the cooling timestep (Δt_{cool}). Because radiative cooling is a stiff equation, we utilize an implicit integration scheme (the Newton-Raphson Backward Euler method). This way, cooling is unconditionally stable over large leaps in time. We handle it through **local cell subcycling**.

During the Hydrodynamics "Drift" stage, whether it is taking a micro-step or a macro-step, the fluid solver iterates over the grid. If a specific gas cell has a cooling timescale shorter than the current hydro step, that individual cell enters its own private `while` loop.

To determine the size of these micro-steps, we evaluate the instantaneous cooling timescale ($t_{\text{cool}} = \frac{\rho u}{\Lambda}$) for the cell. To ensure the implicit solver remains accurate, the cell restricts its thermodynamic micro-step to a safe fraction—typically 10%—of this timescale:

$$\Delta t_{\text{cell}} = 0.1 \times \frac{\rho u}{\Lambda}$$

Breaking the hydro step down into these thermodynamic micro-steps allows the gas to resolve its energy loss without slowing down the simulation.

Accuracy and Symplecticity in a Subcycled Architecture

For a cosmological simulation running for millions of steps, second-order accuracy is a requirement. As introduced earlier, our architecture achieves this through the time-centering of the **Strang Splitting**. Because we apply half the gravity, flow the gas for a full step, and then apply the second half of the gravity, the sequence of operations is perfectly symmetric.

Strang splitting does not dictate the internal complexity of the middle operator; it strictly requires temporal symmetry. In our subcycled architecture, this central operator might consist of a single hydrodynamic advance, or it may encompass an iterative subcycling loop that evolves the gas against an interpolated background potential. From a mathematical perspective, the number of internal micro-steps is irrelevant. Provided the outer operator is advanced for half a timestep before the inner sequence begins, and half a timestep after it concludes, the time-reversibility of the algorithm is preserved, guaranteeing that the global simulation remains second-order accurate.

The Symplectic Nature of Subcycled KDK

As established in earlier chapters, a symplectic integrator (like KDK leapfrog) is designed to preserve the volume of phase space, preventing the energy drift of non-symplectic methods. We will now consider if a subcycled KDK scheme retains this property.

A powerful property of symplectic integrators is that **the composition of any number of symplectic maps is itself a symplectic map.** If we look at the regime where gravity is subcycling, the dark matter takes dozens of tiny micro-KDK steps while the gas grid is temporarily frozen. A single micro-KDK step is symplectic. Therefore, stacking fifty micro-KDK steps in a row is mathematically symplectic. Evolving the gas, and then doing another fifty micro-KDK steps, is still symplectic. Because the subcycling loops are built out of symplectic building blocks, the dark matter integration retains its strict phase-space preservation.

On the other hand, symplectic integration strictly requires the physics to be governed by a **Hamiltonian system**—a framework where processes are fundamentally reversible and the volume of phase space is perfectly conserved. The moment we introduce hydrodynamics, our simulation breaks these rules. Gas physics is inherently dissipative and irreversible. When gas flows collide, supersonic shocks irreversibly generate entropy. Furthermore, with radiative cooling active, the gas actively deletes its own thermal energy by emitting photons into the void. Because these processes permanently destroy phase-space information and leak energy out of the system, the total combined simulation physically cannot be symplectic.

Ultimately, the non-symplectic nature of the combined system is not a consequence of operator subcycling; it is an unavoidable—and necessary—reality of simulating baryonic matter. By marrying these operators in a Strang-split architecture, we maintain the baseline physics of the simulation: the collisionless dark matter remains symplectic, while the gas models the dissipative, irreversible thermodynamics required to forge galaxies—all without sacrificing the global second-order accuracy of the solver.

Key Literature & Further Reading

Springel, V. (2005). *The cosmological simulation code GADGET-2. Monthly Notices of the Royal Astronomical Society*, 364(4), 1105-1134. arXiv:astro-ph/0505010. Available at https://arxiv.org/abs/astro-ph/0505010

Almgren, A. S., Bell, J. B., Lijewski, M. J., Lukić, Z., & Van Andel, E. (2013). *Nyx: A massively parallel AMR code for computational cosmology.* The Astrophysical Journal, 765(1), 39. Available at https://arxiv.org/pdf/1301.4498

Strang, G. (1968). *On the construction and comparison of difference schemes.* SIAM Journal on Numerical Analysis, 5(3), 506-517. Available at https://www.pas.rochester.edu/astrobear/raw-attachment/blog/shuleli08202013/Strang1968.pdf

Spatial Partitioning and the LBVH

While the Particle-Mesh (PM) method solves the long-range forces, the P^3M algorithm still requires direct Particle-Particle (PP) calculations for close neighbors. Furthermore, the Meshless Finite Mass (MFM) hydrodynamics solver requires gas particles to find their nearest neighbors to partition volume, estimate spatial gradients, and compute Riemann fluxes across effective faces.

To make these local searches computationally viable, we must subdivide the simulation space. The spatial partitioning must dynamically adapt to the clustering of matter, providing high resolution in dense zones and coarse resolution in empty voids.

The Oct-Tree Concept

An **Oct-Tree** is a hierarchical data structure that divides 3D space recursively. The process begins by treating the entire simulation box as a single cube (the "root" node). If this root cube contains more than one particle, it is sliced in half along the X, Y, and Z axes, creating 8 smaller sub-cubes (octants).

The algorithm then looks at each of these 8 new octants. If an octant is empty, it is ignored. If it contains only one particle, it becomes a "leaf" node. If it contains multiple particles, it is subdivided into 8 even smaller octants. This recursive slicing continues until every particle sits isolated in its own custom-sized bounding box.

This hierarchy drops the computational cost of spatial searches. When an MFM gas particle searches for its neighbors, it checks the Axis-Aligned Bounding Box (AABB) of the tree nodes. If the particle's search sphere does not intersect a node's bounding box, the algorithm ignores that node and all the particles inside it, saving unnecessary distance calculations.

The Barnes-Hut Algorithm

While hydrodynamics uses the tree to find nearby neighbors, the gravity solver uses the tree to handle the vast distances of the cosmos.

In 1986, Josh Barnes and Piet Hut introduced an algorithm that leveraged this hierarchical tree structure to solve the $O(N^2)$ gravitational bottleneck. Their method, known as the **Barnes-Hut algorithm**, allows us to group distant particles together and approximate their collective gravitational pull.

A Barnes-Hut tree calculates the total mass and the Center of Mass for every internal node (bounding box) in the hierarchy. When calculating the gravitational pull on a specific particle, the algorithm traverses the tree starting from the root.

For each node it encounters, it evaluates a geometric rule known as the **Multipole Acceptance Criterion (MAC)**. This criterion is defined by the ratio between the physical size of the node (L) and the distance from the particle to the node's center of mass (d). The algorithm compares this ratio against a dimensionless accuracy threshold, θ (typically set around $\theta \approx 0.5$):

- **If the threshold is not exceeded ($L/d < \theta$):** The node is considered sufficiently distant for an approximation. The algorithm halts its traversal down that branch and treats the entire node as a single point mass located at its Center of Mass.
- **If the threshold is exceeded ($L/d \geq \theta$):** The node is considered too close for the previous approximation to be accurate. The algorithm opens the node and looks at the smaller sub-cubes inside it, repeating the MAC check. If it reaches the bottom of the tree (a leaf node), it performs a direct Particle-Particle (PP) calculation.

By grouping distant structures into single, coarse computations, the Barnes-Hut algorithm prunes the majority of the tree, dropping the computational cost of gravity from $O(N^2)$ to $O(N \log N)$.

The Multipole Expansion

To justify treating a distant branch as a single point mass, we must look at the Newtonian formula. Suppose we want to calculate the gravitational potential Φ at a location, defined by the vector $\mathbf{r}$, caused by a distant, irregular blob of N particles.

The potential is the sum of the pulls from every individual particle i (with mass m_i and position vector $\mathbf{r}_i$) inside the blob:

$$\Phi(\mathbf{r}) = -G \sum_{i=1}^{N} \frac{m_i}{|\mathbf{r} - \mathbf{r}_i|}$$

Calculating this requires an $O(N)$ loop over every particle in the blob. However, because the MAC guarantees the node is far away, the distance r to the blob is much larger than the distance of any internal particle from the center of the blob ($r \gg r_i$).

Because the ratio $\frac{r_i}{r}$ is much smaller than 1, we can factor the massive distance r out of the denominator and approximate the remaining fraction using a **Taylor series**. By expanding the distance fraction $\frac{1}{|\mathbf{r}-\mathbf{r}_i|}$ based on the small ratio $\frac{r_i}{r}$, the equation unfolds into a **multipole expansion**—a sum of decreasing mathematical terms representing increasingly detailed geometric shapes:

$$\Phi(\mathbf{r}) = \Phi_{\text{monopole}} + \Phi_{\text{dipole}} + \Phi_{\text{quadrupole}} + \dots$$

Here is the breakdown of these terms:

1. The Monopole (0th Order) The first term in the Taylor expansion ignores the internal positions of the particles ($\mathbf{r}_i$):

$$\Phi_{\text{monopole}} = -\frac{G}{r}\sum_{i=1}^{N} m_i = -\frac{GM}{r}$$

This is the equation for a point mass. Its gravitational pull decays as $1/r$.

2. The Dipole (1st Order) The second term introduces the first power of the internal particle positions, describing how "off-center" the mass distribution is. Its strength decays faster, scaling as $1/r^2$:

$$\Phi_{\text{dipole}} = -\frac{G}{r^3}\mathbf{r} \cdot \left(\sum_{i=1}^{N} m_i \mathbf{r}_i\right)$$

Notice the term in the parentheses: the sum of mass times position. By definition, the Center of Mass (CoM) is the point where $\sum m_i \mathbf{r}_i = 0$. By anchoring our Barnes-Hut tree nodes at their Center of Mass, this term conveniently cancels out to zero.

3. The Quadrupole (2nd Order) The third term scales as $1/r^3$. It accounts for the squared positions of the particles, tracking how stretched or squished the blob is along different spatial axes (e.g., spherical, cigar-shaped, pancake-shaped):

$$\Phi_{\text{quadrupole}} = -\frac{G}{2r^5}\sum_{i=1}^{N} m_i \left[3(\mathbf{r} \cdot \mathbf{r}_i)^2 - r^2 r_i^2\right]$$

We can factor this equation to separate the external position vector ($\mathbf{r}$) from the internal properties of the mass blob ($\mathbf{r}_i$). This allows us to encapsulate the blob's internal geometry into a single 3×3 matrix called the **Quadrupole Moment Tensor**:

$$Q_{jk} = \sum_{i=1}^{N} m_i \left(3 r_{i,j} r_{i,k} - r_i^2 \delta_{jk}\right)$$

Where j and k represent the X, Y, and Z coordinate axes (e.g., $r_{i,1}$ is the x-coordinate of particle i), and δ_{jk} is the Kronecker delta (which equals 1 if $j = k$, and 0 otherwise). This matrix calculates the variance and covariance of the mass along the different spatial axes.

- If the blob is a sphere, the mass variances along the X, Y, and Z axes are identical, and the matrix cancels out to zero.
- If the blob is shaped like a cigar pointing along the X-axis, the x^2 variance will be much larger than y^2 or z^2.

Once this 3×3 matrix is pre-calculated and stored for a tree node, the quadrupole contribution to the gravitational potential at any distant point $\mathbf{r} = (r_1, r_2, r_3)$ can be evaluated using matrix multiplication:

$$\Phi_{\text{quadrupole}} = -\frac{G}{2r^5}\sum_{j=1}^{3}\sum_{k=1}^{3} Q_{jk} r_j r_k$$

If we wanted an ultra-precise Barnes-Hut tree, we could calculate the Quadrupole Moment Tensor (3×3 matrix) for every single node in the tree and store it. When calculating gravity, we would compute the monopole (point mass) and then add the quadrupole correction using matrix multiplication.

However, matrix multiplication is computationally expensive. Because the quadrupole effect decays rapidly as $1/r^3$, its physical influence vanishes over short distances. The Multipole Acceptance Criterion (MAC) ensures we only evaluate nodes where r is so massive that the $1/r^3$ quadrupole fraction shrinks below our acceptable margin of error. Enforcing the MAC allows us to process gravity using fast, simple point masses.

The Linear Bounding Volume Hierarchy (LBVH)

To leverage the power of a GPU, we must flatten the 3D spatial partitioning of an oct-tree into a continuous, 1D array of data.

This structure is known as a **Linear Bounding Volume Hierarchy (LBVH)**. In an LBVH, the nodes of the tree are packed together in a single array. A parent node stores the integer index of its children (e.g., `left_child = 52`). Because arrays are contiguous blocks of memory, the GPU can stream this data into its processors at maximum bandwidth.

To flatten a 3D tree into a 1D array while ensuring that particles that are next to each other in the 3D box sit next to each other in the 1D array we can use a technique developed in the 1960s: **Morton Codes**.

Morton Codes and the Z-Order Curve

A Morton code maps multi-dimensional data onto one dimension while preserving spatial locality. It achieves this by interleaving the binary bits of a particle's coordinates.

To do this, we first take a particle's floating-point coordinates (x, y, z) and normalize them to the size of the simulation box. We then scale these coordinates into integers according to the number of bits that will be used per dimension. To fit inside a standard 64-bit integer, we use 21 bits for each dimension (since $21 \times 3 = 63$ bits, leaving one bit unused).

Let's look at an example. Imagine we have a particle at integer coordinates $X = 5$, $Y = 3$, and $Z = 6$. If we write these numbers in binary (using just 3 bits for this example), we get:

- $X = 101_2$
- $Y = 011_2$
- $Z = 110_2$

To generate the Morton code, we interleave these bits, taking one bit from Z, one from Y, and one from X, over and over.

- We take the first bits: **1** from Z, **0** from Y, **1** from X $\rightarrow$ `101`
- We take the second bits: **1** from Z, **1** from Y, **0** from X $\rightarrow$ `110`
- We take the third bits: **0** from Z, **1** from Y, **1** from X $\rightarrow$ `011`

Stitching them together gives us the final Morton code: `101110011`.

For every particle, we generate a unique 64-bit integer representing its position in 3D space. The Morton codes are then used as sorting keys: we reorder our particle list from the smallest Morton code to the largest to get the flat 1D array.

To handle collisions, the sorting algorithm utilizes a deterministic tie-breaker. If two Morton codes are identical, the algorithm compares the particles' original, unique ID numbers to decide which one goes first.

If we trace the path created by counting upward through these Morton codes, we get a distinct "Z" shape that repeatedly subdivides space, creating a fractal pattern known as the **Z-Order Curve**.

This curve acts like a string folded back and forth filling the whole simulation box. The convenience of this string is that if two particles are physically close to each other in the 3D box, their Morton codes will (in the majority of cases) be close to each other.

The Karras Tree Construction

A sorted list of particles based on their Morton codes is not a tree. We still need to build the hierarchy of bounding boxes—the internal nodes—that group these particles together.

In 2012, Tero Karras published an algorithm that allowed to build a **Binary Radix Tree** (a tree where every node has two children, sequentially providing the 8 divisions of an oct-tree node) in parallel. He realized that if the particles are sorted by their Morton codes, the geometric structure of the oct-tree is *already implicitly defined* by the binary numbers themselves.

Because Morton codes interleave spatial coordinates, the **Longest Common Prefix** (the number of identical bits at the beginning of two codes) tells us how close two particles are. If two adjacent particles share a long sequence of identical starting bits, it means they sit deep inside the same sub-region of the simulation box. The first bit where they differ indicates the geometric plane where they split into separate branches.

Here is how we implemented the Karras algorithm in our code:

1. Defining the Topology (Parallel Split)

For N particles (the leaves), a binary tree will have $N-1$ internal branching nodes. In our flat arrays, indices 0 to $N-2$ are internal nodes, and indices $N-1$ to $2N-2$ are the particle leaves.

Because there are $N-1$ internal nodes, we can launch one GPU thread for every internal node simultaneously. Each thread looks at its corresponding index in the sorted Morton array and performs three lock-free steps:

1. **Find the Range:** It compares adjacent Morton codes to determine the upper and lower bounds of the specific cluster of particles it is responsible for by evaluating a delta function, which counts the number of matching leading bits between any two Morton codes. > As an example, imagine a simulation with just $N = 4$ particles. > * **Internal Nodes:** Indices 0, 1, and 2. > * **Leaf Nodes (Particles):** Indices 3, 4, 5, and 6. > * Assume the particles have been sorted by their binary Morton codes: > * Particle 0: `001` > * Particle 1: `011` > * Particle 2: `100` > * Particle 3: `110`
   > > Thread 0 (responsible for Internal Node 0) looks at its neighbors:
   > > * Direction: Thread 0 compares its Morton code (Particle 0) with its immediate neighbors. Looking right to Particle 1 (011), they share 1 leading bit (the first 0). Looking left, there is no particle, which the algorithm treats as a delta of -1. Because it shares more bits to the right than to the left ($1 > -1$), Thread 0 knows its cluster expands to the right. The "outside" boundary of this cluster is therefore defined by that left-side baseline: delta_min = -1.
   > * Number of elements: The thread searches to the right, continuing as long as the delta between Particle 0 and the target particle remains greater than the delta_min boundary. It checks Particle 1 (delta = 1), Particle 2 (delta = 0), and Particle 3 (delta = 0). Because $1, 0$, and 0 are all greater than -1, they all belong to the cluster. The search hits the right edge of the array, so it stops. Thread 0 has determined it owns the full root range [0, 3].
2. **Find the Split:** Using a binary search algorithm over the Morton codes, it finds the index where the highest differing bit occurs. This is the spatial plane that divides its cluster into a left half and a right half. > Following the example: Looking at the range `[0, 3]`, Thread 0 sees the highest differing bit is the leftmost bit (Particle 0 starts with a `0`, Particle 3 starts with a `1`). It binary searches the range to find where this bit flips from `0` to `1`. It finds the flip happens between Particle 1 (`011`) and Particle 2 (`100`). Therefore, the split divides the cluster into a left half **[0, 1]** and a right half **[2, 3]**.
3. **Assign Children:** It assigns the left side to its `left_child` and the right side to its `right_child`. > In our example, because the left side `[0, 1]` contains multiple particles, Thread 0 cannot point to a leaf. Instead, it sets its `left_child` to **Internal Node 1**. Because the right side `[2, 3]` also contains multiple particles, it sets its `right_child` to **Internal Node 2**. > Conversely, when Thread 1 processes the `[0, 1]` range, it splits between 0 and 1. Because the left side is Particle 0, Thread 1 sets its `left_child` to **Leaf Index 3**. Because the right side is Particle 1, it sets its `right_child` to **Leaf Index 4**.

Because this relies only on reading the sorted Morton array, no thread needs to wait for another to finish, and the entire hierarchy can be built in parallel.

2. Bottom-Up Aggregation

Once the topology is wired, the nodes are just empty shells. We must fill them with physical data: the Axis-Aligned Bounding Box (AABB) for hydrodynamics, and the Mass and Center of Mass for gravity.

We launch a thread for every particle leaf. The thread populates its leaf with the particle's mass and coordinates, and then begins "walking" up the tree toward the root.

When a thread arrives at a parent node, it uses a hardware-level **atomic counter** (a specialized flag).

- If it is the *first* thread to arrive from either the left or right child, it increments the flag and terminates. It cannot compute the parent's bounding box because the other child hasn't arrived yet.
- If it is the *second* thread to arrive, it knows both children have finished their calculations. It reads the bounding boxes and masses of both children, merges them together to define the parent, and then continues walking up to the next level of the tree.

This bottom-up cascade aggregates the geometry and mass of the universe up to the root node.

Stack-Based Traversal

With the LBVH fully constructed as a flat 1D array of nodes, we are ready to compute the physics. To search this binary tree, we use an **Iterative Stack-Based Traversal**.

At the start of the physics calculation, every thread allocates a tiny, fixed-size array in its local memory. In our code, this is a 128-element integer array: `int stack[128]`. Because the depth of our LBVH is bounded by the number of bits used to generate the Morton codes, a 128-element stack (63 Morton code + 64 index tie breaker) is enough to traverse the entire universe without overflowing.

The traversal then operates as a flat `while` loop:

1. The thread pushes the root node (index 0) onto the stack: `stack[stack_ptr++] = 0;`.
2. The loop begins. The thread pops a node off the top of the stack.
3. This is the culling step:
    - **For Hydrodynamics:** It checks if the node's bounding box overlaps the gas particle's search radius. If it doesn't, the thread issues a `continue` statement, culling the entire branch and moving on to the next node on the stack.
    - **For Gravity:** It calculates the distance to the node's Center of Mass and checks the Multipole Acceptance Criterion (MAC). If the node is far enough away, it applies the bulk gravitational pull of the entire node and issues a `continue` statement.
4. If the node must be opened, the thread pushes the `left_child` and `right_child` integers onto its local stack and repeats the loop.

The TreePM Architecture

Combining the Particle-Mesh (PM) grid and the Linear Bounding Volume Hierarchy (LBVH) results in the **TreePM** (Tree-Particle-Mesh) architecture.

In the TreePM framework, the gravitational potential is split into short-range and long-range contributions. This allows us to assign the strengths of each algorithm to the scales where they excel:

- Long-range forces are calculated using a particle mesh. The PM solver handles these forces using Fast Fourier Transforms to efficiently bypass the $O(N^2)$ bottleneck.
- Short-range forces are calculated following the oct-tree algorithm. The LBVH takes over at the sub-grid level, traversing the bounding boxes to resolve the gravitational dynamics of collapsing halos.

To ensure gravity is not double-counted where these two regimes overlap, the forces must be seamlessly blended together. To achieve this, the simulation applies a continuous spatial filter to the gravitational equations. When traversing the LBVH for short-range interactions, the Newtonian force is multiplied by a complementary error function, or `erfc`. This smoothly scales the tree's gravitational pull down to zero at the cutoff radius where the PM grid's forces take over.

Key Literature & Further Reading
For the Barnes-Hut Algorithm:
Barnes, J., & Hut, P. (1986). *A hierarchical O(N log N) force-calculation algorithm.* Nature, 324(6096), 446-449. Available at https://www.nature.com/articles/324446a0

For Morton Codes (Z-Order Curve):
Morton, G. M. (1966). *A computer oriented geodetic data base; and a 7 1/4 transverse Mercator projection.* IBM Research Report.

For the Karras Tree & LBVH Construction:
Karras, T. (2012). *Maximizing parallelism in the construction of BVHs, octrees, and k-d trees.* Proceedings of the Fourth ACM SIGGRAPH / Eurographics conference on High-Performance Graphics, 33-37. Available at https://research.nvidia.com/sites/default/files/pubs/2012-06_Maximizing-Parallelism-in/karras2012hpg_paper.pdf

For the Integration of LBVH/Oct-Trees in GPU N-Body Solvers:
Bédorf, J., Gaburov, E., & Portegies Zwart, S. (2012). *A sparse octree gravitational N-body code that runs entirely on the GPU processor.* Journal of Computational Physics, 231(7), 2825-2839. Available at https://arxiv.org/pdf/1106.1900

Individual particle timesteps

Cosmological simulations are defined by their extreme dynamic range. To solve this, the spatial discretization of the fluid must be matched by a temporal discretization. Particles can be granted individual timesteps determined

by their immediate environment.

Bounding the Local Clock

Before a particle can step forward in time, the system evaluates the absolute maximum time it can safely wait before re-evaluating its surroundings. This "ideal" timestep is defined by several constraints:

1. **The Courant-Friedrichs-Lewy (CFL) Condition:** In a fluid, information propagates at the local signal velocity, which is a combination of the sound speed and the relative velocities of converging gas flows. To prevent fluid volumes from interpenetrating or missing a shock boundary, a particle's timestep is restricted so that physical information cannot cross its smoothing radius in a single leap.
2. **Gravitational Acceleration:** Particles experiencing immense gravitational pulls—such as those falling into a dark matter halo—must integrate their trajectories frequently to accurately resolve the curvature of their orbits. This constraint is tightly coupled to the gravitational softening length to ensure particles do not artificially fling themselves out of bound states.
3. **Radiative Cooling:** When dense gas collapses, it radiates thermal energy rapidly. If a timestep is too large, the numerical solver might artificially drain all of a particle's internal energy in one step, plunging it below the cosmic temperature floor. The clock is therefore restricted so that a particle loses no more than a small fraction (e.g., 10%) of its total internal energy per step.

The smallest of these three values becomes the particle's ideal physical timestep.

The Power-of-Two Hierarchy

While giving every particle its ideal timestep seems optimal, doing so creates a chaotic, asynchronous timeline. If Particle A has a timestep of 102 years and neighboring Particle B has a timestep of 57 years, calculating the momentum exchange between them becomes complicated because of the misaligned clocks.

Instead, the simulation imposes a quantized hierarchy. The maximum allowable global timestep (Δt_{max}) acts as the root of the hierarchy. Every particle's ideal timestep is progressively halved until it bounds the physical requirement.

This means a particle's actual timestep is always $\Delta t_{max}/2^n$, placing it into a specific "time bin". This structure guarantees synchronization. A particle on a small timestep will always land on the synchronization tick of a particle on a larger timestep, allowing them to exchange physical fluxes.

The Active Timeline and Integration

With the hierarchy established, the simulation scans the timeline for the nearest upcoming synchronization point and advances the master clock to that precise moment.

At this tick, only the particles whose block steps end at the current time are flagged as "active". The rest are considered "sleeping." This division requires a modified approach to the Kick-Drift-Kick (KDK) symplectic integrator:

- **Drifting the Universe:** Because hydrodynamics relies on precise spatial geometries, evaluating the density field requires the true positions of every particle. Therefore, *all* particles—both active and sleeping—are drifted forward in space to the current master clock time.
- **Kicking the Active:** Only the active particles are subjected to the heavy computational lifting. The tree searches, density evaluations, gradient matrix inversions, and Riemann solver flux calculations are executed only for the active subset. Their velocities and internal energies are then "kicked" using the newly computed forces.

Asymmetric Fluxes and Conservation Trade-offs

When an active particle calculates a fluid interface with a neighbor, a unique scenario arises if that neighbor is currently sleeping. According to Newton's Third Law, if the active particle loses momentum, the sleeping particle must gain it.

However, because the sleeping particle is locked in a much larger time bin, abruptly altering its momentum mid-step corrupts its temporal integration. To resolve this, the system processes the Riemann problem but applies the resulting flux only to the active particle. By deliberately ignoring the reverse flux onto the sleeping

particle, the code accepts a microscopic violation of machine-precision energy conservation in exchange for maintaining the mathematical integrity of the hierarchical timeline.

The Wake-Up Failsafe

A block time-stepping scheme is vulnerable to the approach of a supersonic shockwave. If a cold, quiescent particle in a cosmic void is assigned a massive timestep, it goes to sleep, expecting nothing to happen for millions of years. If a shockwave generated by a distant supernova crosses the void and impacts the sleeping particle mid-step, the particle will be unable to react, leading to numerical instability.

To protect the fluid continuum, the scheme employs a "Wake-Up" failsafe. During the hydrodynamic force evaluation, active particles constantly scan their neighbors. If an active particle detects a sleeping neighbor, it compares their signal velocities. If the active particle's signal velocity exceeds the sleeping neighbor's by a critical threshold (for example a factor of 3), an override is triggered.

The sleeping particle receives a thread-safe wake-up signal. Its massive timestep is truncated to match the time it has actually experienced, and it is flagged as active for the current clock tick. On the subsequent step, the newly awakened particle drops into a smaller power-of-two time bin, correctly resolving the incoming shockwave.

Analytical Requirements for Resolution and Volume

Because computational resources are finite, choosing the parameters of the simulation requires a trade-off between the overall size of the simulated box and the size of the individual grid cells and number of particles (the resolution). If we choose poorly, our virtual universe will fail to represent the actual cosmos.

Terminology

Finite Volume Effects

This refers to the physical limitations introduced into a simulation because the box has a finite size L_{box}. Since finite size simulations also use periodic boundary conditions, any density perturbation (wave) larger than the box is not simulated. This lack of gravitational large-scale perturbations influences the results of the simulations.

Bagla, J. S., & Prasad, J. (2006). *Effects of the size of cosmological N-Body simulations on physical quantities - I: Mass Function.*, Monthly Notices of the Royal Astronomical Society, 370(2), 993-1002. arXiv:astro-ph/0601320. Available at https://arxiv.org/abs/astro-ph/0601320

Cosmic Variance

Cosmic variance is the statistical uncertainty that arises because we are only simulating (or observing) a finite patch of that universe. A box might randomly end up in a slightly overdense or underdense region. Any measurement we make in such box will have a statistical scatter around the global cosmic mean.

Sinigaglia, F., & Kitaura, F.-S. (2026). *Cosmic variance or galaxy bias? Disentangling finite-volume and galaxy formation effects in cosmological analysis.* arXiv preprint, arXiv:2606.04830. Available at https://arxiv.org/abs/2606.04830

Super-Sample Covariance (SSC)

The waves that are larger than the box (super-sample modes) in reality act as a uniform background overdensity (δ_b) or underdensity that shifts the local expansion rate and modulates the growth of small-scale structures. In a simulation, these missing modes introduce a correlated error across all the non-linear power spectrum bins.

Note how these three terms tie together: because we have a Finite Volume, we have a statistical scatter known as Cosmic Variance, and we are missing large modes. When we measure the power spectrum, the absence of these modes manifests itself as Super-Sample Covariance, because they would couple to the small scales, as they do in reality.

Takada, M., & Hu, W. (2013). *Power spectrum super-sample covariance.* Physical Review D, 87(12), 123504. arXiv:1302.6994. Available at https://arxiv.org/abs/1302.6994

Baryon Acoustic Oscillations (BAO)

Baryon Acoustic Oscillations are periodic fluctuations in the density of the visible baryonic matter of the universe. They originated as acoustic (sound) waves propagating through the hot, dense primordial plasma of

the early universe. When the universe expanded and cooled enough for neutral hydrogen to form (the epoch of recombination), the pressure driving these sound waves vanished, "freezing" the waves in place. This left behind spherical shells of slightly higher matter density at a characteristic physical radius of roughly 150 Mpc (which manifests as the peak on a graph showing the BAO distribution). Astronomers use this 150 Mpc preferred clustering scale as a "standard ruler" to measure the expansion history of the universe.

Galaxy Formation Bias

Galaxy Formation Bias (often simply called "galaxy bias") is the statistical discrepancy between the spatial distribution of visible galaxies and the underlying distribution of dark matter. Galaxies tend to form and reside within the deepest gravitational potential wells (the most massive dark matter halos). Consequently, galaxies are "biased" tracers of the universe's mass, appearing more clustered than the total matter field actually is. This bias arises from the complex, non-linear processes driving galaxy formation, which depend on the characteristics of the host dark matter halo and the surrounding cosmic web environment.

Expected Discrepancies in the Physics of Small Boxes

As explained before, a finite box with periodic boundary conditions imposes a fundamental mode $k_{min} = 2\pi/L_{box}$, which effectively truncates the power spectrum and alters the physics.

This produces a series of observable discrepancies:

- The Halo Mass Function & Gas Temperatures:
  - Because large-scale modes are missing, the hierarchical merging process is delayed.
  - The number of high-mass halos is underestimated, and the number of low-mass halos is overestimated (because they fail to merge into larger structures).
- The Matter Power Spectrum & Mode Coupling:
  - Non-linear growth at small scales is accelerated when embedded in large-scale overdensities. Without these super-sample modes, the late-time power spectrum inside the box will be suppressed compared to CAMB.
  - Super-Sample Covariance (SSC): Missing background modes introduce correlated errors in the power spectrum.
- Baryon Acoustic Oscillations (BAO) Shifts:
  - Cosmic variance in small volumes can induce artificial shifts in the BAO peak position. This shift is degenerate (producing the same observable effect) with physical galaxy formation bias and non-linear gravitational growth (Sinigaglia & Kitaura, 2026).

Although there are ways to deal with these limitations, these are not covered in this text. Here is a brief summary of the most common strategies:

- Analytical Corrections: Theoretical expectations (like the Press-Schechter or Sheth-Tormen curves) can be mathematically adjusted to match the finite volume of the simulation, for example using the variance correction terms derived by Bagla & Prasad (2006).

- Separate Universe Simulations: The missing long-wavelength modes can be injected into the simulation by altering the background cosmology (changing Ω_m and the Hubble expansion). Wagner et al. (2015) explains how this is used to measure the power spectrum response function $R(k)$—a metric that quantifies how much the small-scale clustering shifts in response to a large-scale background overdensity.

- Variance Suppression Techniques: Sinigaglia & Kitaura (2026) discuss how fixing the initial amplitude of Fourier modes and running pairs of phase-inverted simulations can suppress the uncertainty on clustering statistics (like the BAO scale) by a factor of 4 to 5.

Bagla, J. S., & Ray, S. (2005). *Finite volume effects in cosmological N-body simulations. Monthly Notices of the Royal Astronomical Society*, 358(3), 1076-1082. arXiv:astro-ph/0410373. Available at https://arxiv.org/abs/astro-ph/0410373

Sirko, E. (2005). *Initial conditions to cosmological N-body simulations, or how to run an ensemble of simulations. The Astrophysical Journal*, 634(2), 728-743. arXiv:astro-ph/0503106. Available at https://arxiv.org/abs/astro-ph/0503106

Wagner, C., Schmidt, F., Chiang, C.-T., & Komatsu, E. (2015). *Separate universe simulations. Monthly Notices of the Royal Astronomical Society: Letters*, 448(1), L11-L15. arXiv:1409.6294. Available at https://arxiv.org/abs/1409.6294

The Box Size

To simulate a "representative" patch of the universe, the statistical properties of the simulation box should look identical to any other randomly selected patch of the real universe of the same size.

Minimum Size

The minimum box size of a simulation is dictated by finite volume effects and cosmic variance. It must be chosen so that the amplitude of fluctuations at the box scale (and at larger scales) is ignorable. Otherwise the simulation won't correctly represent the model being studied.

To determine if a chosen box size is sufficiently large, we can quantify the amount of "missing" power truncated by the periodic boundary conditions. Bagla and Prasad (2006) established a formal methodology based on the variance of density fluctuations. Let $\sigma_0^2(r)$ be the total expected theoretical variance of the mass fluctuations at a specific physical scale of interest r. The variance lost due to the finite box size (L_{box}) can be approximated by a first-order correction term, $C_1(L_{box})$.

For the finite volume effects to be considered ignorable, the ratio of this missing variance to the total expected variance must be minimized:

$$\frac{C_1(L_{box})}{\sigma_0^2(r)} \ll 1$$

This ratio can be bound to a specific threshold to ensure a given statistical accuracy. For example, for 10% it would be:

$$\frac{C_1(L_{box})}{\sigma_0^2(r)} \leq 0.1$$

Because the denominator $\sigma_0^2(r)$ depends on r, the required box size scales with the size of the objects the simulation is attempting to resolve. For example, simulating massive galaxy clusters (where $r \approx 2$ Mpc) requires a larger L_{box} to satisfy this threshold than simulating the internal structure of individual galaxies.

To compute the expected theoretical variance $\sigma_0^2(r)$ for a specific physical scale r, we integrate the linear matter power spectrum over all possible wavelengths (from zero to infinity) using a smoothing filter.

$$\sigma_0^2(r) = \int_0^\infty \frac{dk}{k} \frac{k^3 P(k)}{2\pi^2} W^2(kr)$$

Where:

- k: The wave number (which is related to the physical wavelength by $k = 2\pi/\lambda$).
- $P(k)$: The theoretical matter power spectrum, which defines the expected amplitude of density fluctuations as a function of scale for the chosen cosmological model (e.g., ΛCDM). Because calculating this requires solving the complex physics of the early universe, $P(k)$ is typically obtained using standard public Boltzmann codes (such as CAMB or CLASS) or by applying established analytical approximations (such as the Eisenstein & Hu transfer function).
- $W(kr)$: The Fourier transform of the window function used to "smooth" the density field over the scale r.

The power that gets left behind, the $C_1(L_{box})$ term, is defined as the same integral but limited to the missing large-scale modes:

$$C_1(L_{box}) = \int_0^{2\pi/L_{box}} \frac{dk}{k} \frac{k^3 P(k)}{2\pi^2} W^2(kr)$$

Bagla and Prasad specifically use a **real-space spherical top-hat window function**. Its Fourier transform is:

$$W(kr) = \frac{3(\sin kr - kr \cos kr)}{(kr)^3}$$

By integrating from $k = 0$ to $k = \infty$, this formula calculates the variance of an infinite universe. A simulated box can only integrate from $k = 2\pi/L_{box}$. The power that gets left behind is the $C_1(L_{box})$ error term we are trying to minimize.

Maximum Size

The maximum, on the other hand, is dictated by General Relativity. In a Newtonian Poisson solver, gravity acts instantaneously across the entire domain. For this approximation to be valid, the comoving length of the box (L) must remain sub-horizon ($L \ll c/H$).

Today, the Hubble radius (c/H) is roughly 4,300 Mpc. If the box size is close to this scale, the instantaneous gravity approximation violates the relativistic physics, allowing causally disconnected regions to interact. Therefore, the box size should be kept below the Hubble radius.

A Reference for Cosmic Scales

Because the Megaparsec (Mpc) is a vast unit of distance (1 Mpc $\approx$ 3.26 million light-years), it can be difficult to build an intuition for the scale of a simulation grid. Here is a quick-reference guide to the approximate diameters and masses of common astronomical structures according to *Galaxy Formation and Evolution* (Mo, van den Bosch & White, 2010):

Cosmic Structure	Typical Size (Mpc)	Typical Mass ($M_\odot$)	Description
Earth-Sun Distance (1 AU)	$\sim 5 \times 10^{-12}$ Mpc		
The Solar System	~ 0.00001 Mpc		Reaching out to the edge of the Oort Cloud.
Dwarf Galaxy (Visible)	$\sim 0.00001 - 0.0009$ Mpc		Derived from the text stating sizes range from "a few tens to several hundreds of parsec".
Milky Way (Visible Stellar Disk)	~ 0.03 Mpc	$\sim 5 \times 10^{10} M_\odot$	Described as a thin stellar disk with an "overall diameter of ~ 30 kpc".
Milky Way (Dark Matter Halo)	> 0.2 Mpc	$\sim 10^{12} M_\odot$	The dark halo is "thought to extend well beyond 100 kpc from the Galactic center" (Radius > 0.1 Mpc, Diameter > 0.2 Mpc).
Typical Galaxy (Visible)	~ 0.02 Mpc	$\sim 10^{10} - 10^{12} M_\odot$	The text states a "typical bright galaxy... has a diameter (~20kpc)", with stellar masses rarely exceeding $10^{12} M_\odot$.
Typical Galaxy (DM Halo)	> 0.2 Mpc	$\sim 10^{11} - 10^{13} M_\odot$	Galaxy-galaxy lensing shows they have "extended dark matter halos with masses 10-100 times more massive than the galaxies themselves".
The Local Group	~ 1 Mpc		Defined as a "loose association of galaxies which fills an irregular region just over 1 Mpc across".
Galaxy Group (DM Halo)	$\sim 0.29 - 2.9$ Mpc	$4.5 \times 10^{12} - 1.4 \times 10^{14} M_\odot$	Defined as having "radii in the range $(0.1 - 1)$ h^{-1} Mpc".
Galaxy Cluster (DM Halo)	$\sim$ a few Mpc	$\sim 1.4 \times 10^{15} M_\odot$	Clusters contain more than 50 relatively bright galaxies "in a volume only a few megaparsecs across".

Cosmic Structure	Typical Size (Mpc)	Typical Mass ($M_\odot$)	Description
Cosmic Filaments	Up to 100 Mpc (Length)		The bridges of dark matter and gas connecting clusters.
Cosmic Voids	Up to $\sim$ 100 Mpc		Defined as regions "with diameters up to $\sim$ 100 Mpc that contain very few, or no, galaxies".
BAO Scale	~150 Mpc		The Baryon Acoustic Oscillation scale.
Scale of Homogeneity	~370 Mpc		The scale at which the universe finally appears homogeneous.

Scale of Homogeneity reference:
Yadav, J. K., Bagla, J. S., & Khandai, N. (2010). *Fractal dimension as a measure of the scale of homogeneity.* Monthly Notices of the Royal Astronomical Society, 405(3), 2009-2015.

Full-size Representative Simulations

When designing a cosmological simulation, the specific physics implemented dictate the bounds for the box size (L) and the grid dimension (N). To run a simulation that captures the correct tendencies we must balance several constraints.

The Cooling Threshold

In any hydrodynamical setup, gas is allowed to radiate thermal energy away, but this cooling is halted at a specific temperature floor ($T_{\rm floor}$). Depending on the simulation, this floor might represent a physical barrier, such as the limit of primordial atomic cooling.

For the fluid solver to demonstrate the complete cycle of hierarchical structure formation (infall, shock-heating, and subsequent core condensation), the gravity solver must be capable of forming dark matter halos massive enough to shock-heat the infalling gas to at least this $T_{\rm floor}$ threshold.

The analytical relationship between a dark matter halo's mass ($M_{\rm vir}$) and the temperature of the gas shock-heated within its potential well ($T_{\rm vir}$) is derived from the Virial Theorem. By equating the thermal kinetic energy of the gas to the gravitational binding energy of the halo, we get the equation for virial temperature:

$$T_{\rm vir} = \frac{\mu m_p G M_{\rm vir}}{2 k_B R_{\rm vir}} \approx 1.98 \times 10^4 \text{ K} \left(\frac{M}{10^8 M_\odot} \right)^{2/3}$$

Where:

- μ is the mean molecular weight of the gas (roughly **0.6** for fully ionized primordial gas).
- m_p is the mass of a proton.
- G is the gravitational constant.
- k_B is the Boltzmann constant.
- $R_{\rm vir}$ is the virial radius of the collapsed dark matter halo.

To use this equation practically, we must eliminate the radius variable ($R_{\rm vir}$) and express it in terms of mass. According to the Spherical Collapse Model, a halo is defined by a specific virial overdensity (the background density of the universe being known at any given epoch).

Substituting this mass-radius relationship into the virial equation, yields a parameterized formula that links temperature and mass. Rearranging this relation allows us to isolate the minimum halo mass ($M_{\rm min}$) required to reach a specific temperature floor ($T_{\rm floor}$):

$$M_{\rm min} \approx 10^8 M_\odot \left(\frac{T_{\rm floor}}{10^4 \text{ K}} \right)^{3/2}$$

To trigger the chosen cooling curve, the simulation's volume and mass resolution must be capable of forming halos of at least $M_{\rm min}$.

Mass Resolution

Knowing the minimum halo mass (M_{min}) required to trigger our specific thermodynamics, we must now ensure the grid is fine enough to actually resolve it. In a grid-based solver utilizing Cloud-In-Cell (CIC) mass assignment or similar schemes, structures that are only one or two cells wide are heavily distorted by numerical diffusion and grid noise. A dark matter halo must span a minimum number of grid cells, N_{halo}, to be considered physically resolved and dynamically stable.

Therefore, the maximum allowable mass resolution (m_{cell}, the mean mass of an unperturbed background cell) is constrained by:

$$m_{\text{cell}} \leq \frac{M_{\text{min}}}{N_{\text{halo}}}$$

The mass resolution can be defined also as a function of the total background comoving matter density of the simulated universe (ρ_m), the physical box size (L), and the 1D grid resolution (N):

$$m_{\text{cell}} = \rho_m \left(\frac{L}{N}\right)^3$$

By equating these two principles, we can obtain the maximum allowable physical width of a single grid cell ($\Delta x = L/N$):

$$\Delta x_{\text{max}} \leq \left(\frac{M_{\text{min}}}{N_{\text{halo}}\rho_m}\right)^{1/3}$$

Final Synthesis

By dividing the minimum volume by the maximum cell size, we arrive at the equation for the minimum required grid dimension (N):

$$N \geq \frac{L_{\text{min}}}{\Delta x_{\text{max}}}$$

Substituting our derived mass resolution into this equation yields the analytical requirement:

$$N \geq L_{\text{min}} \left(\frac{N_{\text{halo}}\rho_m}{M_{\text{min}}}\right)^{1/3}$$

If a simulation is run at a lower resolution, the mass of a single cell artificially swells beyond M_{min} and the physics engine becomes blind to the smallest structures it is attempting to resolve.

Key Literature & Further Reading

Truelove, J. K., Klein, R. I., McKee, C. F., Holliman II, J. H., Howell, L. H., & Greenough, J. A. (1997). *The Jeans condition: a new constraint on spatial resolution in simulations of isothermal self-gravitational hydrodynamics.* The Astrophysical Journal Letters, 489(2), L179. Available at https://iopscience.iop.org/article/10.1086/310975

Barkana, R., & Loeb, A. (2001). *In the beginning: the first sources of light and the reionization of the universe.* Physics Reports, 349(2), 125-238. Available at https://arxiv.org/abs/astro-ph/0010468

Architectures of Cosmological Simulations

Because computational power is finite, it is physically impossible to simulate the entire visible universe at the resolution required to resolve individual star systems or even individual molecular clouds. Cosmological simulation is fundamentally an exercise in managing the "computational trade-off" between volume and resolution.

To answer different distinct physical questions, cosmologists have developed several distinct architectures, or strategies, for setting up their simulation domains. These architectures dictate how the initial conditions are generated and how computational resources are allocated across the grid.

Large-Volume "Uniform Box" Simulations

This is the standard architecture we have been assuming throughout this text. In a uniform box simulation, a massive cubic volume of space (often hundreds of Megaparsecs across) is simulated at a consistent, uniform mass and spatial resolution throughout the entire domain.

- **The Goal:** These simulations are designed to capture the statistical properties of the large-scale structure. They are the ideal tool for studying the cosmic web, determining the mass function of galaxy clusters, measuring the sizes of voids, and testing how different dark energy models alter the expansion history of the universe.
- **The Trade-off:** Because the simulated volume is so massive, the computational resolution is relatively coarse. A uniform box simulation can reliably predict *where* a dark matter halo will form and how much mass it contains, but it lacks the resolution to model the intricate, internal physics of the galaxy residing inside it, such as the structure of its spiral arms.
- **Notable Examples:** The Millennium Simulation, IllustrisTNG, EAGLE.

Zoom-in Simulations

If uniform boxes provide the macro-view of the universe, zoom-in simulations are the cosmic microscope. They allow developers to dedicate nearly all of their computational resources to a single, highly resolved object—such as a Milky Way-sized galaxy—while still embedding it within a true cosmological environment.

Creating a zoom-in simulation requires a multi-step computational technique:

1. **The Base Run:** First, the developer runs a fast, low-resolution, dark matter-only simulation of a large uniform volume.
2. **Target Selection:** Once the run reaches $z = 0$, the developer searches the data to find a specific, interesting structure (for example, an isolated halo with a mass of $10^{12} M_{\odot}$).
3. **Tracing the Lagrangian Region:** The code identifies every particle that ended up inside that target halo and traces them backward in time to their exact starting positions in the initial conditions (e.g., at $z = 100$). This specific patch of the initial grid is called the target's *Lagrangian region*.
4. **Refining the Initial Conditions:** The initial conditions generator is run a second time. However, this time, high-frequency density waves (fine-grained details) are injected *only* into that specific Lagrangian region. The rest of the box is left at low resolution.
5. **The Final Run:** The simulation is re-run. The high-resolution particles collapse into a highly detailed galaxy in the center of the box, while the massive, low-resolution particles on the outskirts simply act as boundary conditions, providing the correct long-range gravitational tidal forces.

- **The Goal:** To study the intricate, internal physics of individual galaxies, such as supernova feedback, supermassive black hole accretion, and the formation of galactic disks.
- **The Trade-off:** Because they are intensely expensive, they lack statistical power. A zoom-in simulation usually models only one or a handful of galaxies. If the chosen target happened to be a statistical outlier, it might lead to incorrect assumptions about general galaxy formation.
- **Notable Examples:** The FIRE (Feedback In Realistic Environments) project, the Auriga simulations.

Constrained Simulations

In standard uniform boxes and zoom-in simulations, the initial conditions are generated using a random mathematical seed. This creates a statistically valid but completely random, generic patch of space. Constrained simulations, however, aim to simulate our exact physical neighborhood.

Using advanced mathematical techniques (such as Wiener filtering), cosmologists run complex algorithms backwards to reverse-engineer the initial density field based on actual 3D telescopic surveys of the real sky.

- **The Goal:** To force the primordial density ripples to collapse into exact replicas of the Milky Way, Andromeda, the Virgo Cluster, and the Local Void, in the exact coordinates where we observe them today. This allows cosmologists to test structure formation theories against the most well-observed region of the universe.
- **The Trade-off:** The reverse-engineering of the density field is mathematically grueling, heavily dependent on the accuracy of observational data, and is currently only viable for the relatively local universe.
- **Notable Examples:** The CLUES project (Constrained Local UniversE Simulations), SIBELIUS.

The Physics Split: Gravity-Only vs. Hydrodynamics

Beyond spatial architecture, simulations are also categorized by the physical forces they compute.

Dark Matter-Only (N-body) simulations completely ignore baryonic gas, stars, and radiation. Everything is treated as collisionless particles governed purely by the Poisson solver and the expansion of space. Because they are computationally inexpensive, they can simulate enormous Gigaparsec-scale volumes, making them the primary tool for testing pure cosmology and General Relativity.

Hydrodynamical simulations introduce the complex fluid dynamics of normal matter. By integrating the Euler equations alongside gravity, these codes can model shock waves, the heating and cooling of gas, and "sub-grid" physics like star formation. While they are required to understand what the universe actually "looks" like to telescopes, adding hydrodynamics increases the computational cost by orders of magnitude. For this reason, hydrodynamics are most frequently paired with Zoom-in architectures or smaller uniform boxes.

Diagnosing Cosmological Simulations

This chapter outlines the standard diagnostic plots used to evaluate the health of a cosmological simulation, detailing the underlying physics and the expected theoretical behaviors.

Cosmic Expansion History

The most basic cross-check of a cosmological simulation is verifying its background geometry. Because the simulation box represents a comoving volume of the universe, its physical size must expand according to the Friedmann equations.

To validate this, the scale factor a is plotted against the physical simulation time (typically in Gigayears). The shape of this curve is dictated by the cosmological parameters chosen for the run, specifically the matter density parameter (Ω_m) and the dark energy density parameter (Ω_Λ).

In a matter-dominated universe, the expansion decelerates over time. However, in a standard ΛCDM cosmology ($\Omega_\Lambda > 0$), dark energy eventually dominates. On the plot, this manifests as a curve that initially decelerates but gradually bends upward at late times (around $a \approx 0.5$ to 1.0), reflecting the accelerated expansion of the universe.

Structure Growth and Linear Theory

We must confirm that dark matter is clumping together at the correct rate. This can be tracked by plotting the **Dark Matter Density Variance** (σ^2) against the scale factor.

The Physics of Density Variance

The density contrast, δ, defines how much denser or emptier a specific region is compared to the cosmic mean density $\bar{\rho}$:

$$\delta(\mathbf{x}) = \frac{\rho(\mathbf{x}) - \bar{\rho}}{\bar{\rho}}$$

To calculate the variance of the entire simulation, the continuous distribution of dark matter mass is mapped onto a uniform spatial grid. For every discrete volume element in the simulation, the local density contrast δ is calculated. The variance σ^2 is simply the mean of the squared density contrasts across the entire volume:

$$\sigma^2 = \langle \delta^2 \rangle$$

This single number (σ^2) represents the global "clumpiness" of the universe. A completely smooth universe has a variance of zero.

Linear Perturbation Theory

To know if the simulated variance is correct, it is compared against Cosmological Linear Perturbation Theory. In the very early universe, density fluctuations are microscopic ($\delta \ll 1$). Under these conditions, the equations of dark matter can be linearized. These small perturbations grow uniformly according to a Linear Growth Factor, $D(a)$:

$$\delta(\mathbf{x}, a) = \delta_0(\mathbf{x}) \frac{D(a)}{D(a_0)}$$

In a purely matter-dominated universe (an Einstein-de Sitter cosmology), the growth factor is directly proportional to the scale factor ($D(a) \propto a$), meaning the variance simply grows with the square of the scale factor ($\sigma^2 \propto a^2$).

However, in a ΛCDM universe, the accelerated stretching of spacetime fights against the gravitational pull of the dark matter halos, causing the rate of structure growth to slow down.

To validate the numerical solver in both cases, the theoretical model for the variance curve is driven by the integral of the expanding Hubble metric $H(a)$:

$$D(a) \propto H(a) \int_0^a \frac{da'}{(a'H(a'))^3}$$

With:

$$H(a) = H_0 \sqrt{\Omega_{m,0} a^{-3} + \Omega_{\Lambda,0}}$$

Usually approximated as:

$$D(a) \propto \frac{5\Omega_m(a)}{2} \left[\Omega_m(a)^{4/7} - \Omega_\Lambda(a) + \left(1 + \frac{\Omega_m(a)}{2}\right)\left(1 + \frac{\Omega_\Lambda(a)}{70}\right) \right]^{-1} \cdot a$$

By plotting this growth curve against the discrete numerical variance calculated from the grid, we can prove that the N-body gravity solver is capturing both the early epoch of rapid hierarchical assembly and the late-stage suppression of structure formation caused by Dark Energy.

Expected Simulation Behavior

- **The Linear Regime:** At early times, the simulated variance must track the theoretical line. The initial smooth distribution of matter gently amplifies exactly as linear equations predict.
- **The Non-Linear Regime:** At later times (typically around $a > 0.4$), the simulated variance will curve upward, breaking away from linear theory. As dense clumps form ($\delta > 1$), the linear approximation fails. Gravity becomes highly localized and non-linear, pulling matter into dark matter halos much faster than the background linear theory suggests.

The 1-Point Density PDF

While the variance provides a single number for global structure growth, the **1-Point Density Probability Distribution Function (PDF)** provides a complete statistical picture of the matter distribution. Most commonly plotted as a **Volume-Weighted PDF**, this histogram shows the volume fraction of the universe occupied by gas at various overdensities ($\rho/\bar{\rho}$).

By plotting the simulation's PDF at different evolutionary stages (e.g., $a = 0.1$, $a = 0.5$, $a = 1.0$) against analytical models, we can verify the migration of mass.

The Early Universe: The Gaussian Model

In the linear regime of the early universe, primordial density fluctuations are symmetrical. The probability $P(\delta)$ of finding a region with a specific density contrast δ follows a classic Gaussian distribution, dictated by the global variance σ^2:

$$P(\delta) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{\delta^2}{2\sigma^2}\right)$$

On a plot, the early-universe PDF will resemble a very narrow, tall bell curve centered at $\rho/\bar{\rho} = 1$.

The Late Universe: The Lognormal Model

As gravity drives the universe into the non-linear regime, the symmetry of the Gaussian curve breaks. Mass evacuates from cosmic voids (which are bounded, as density cannot drop below zero) and compresses into halos (which have no upper density limit). The peak of the PDF shifts slightly to the left, representing the massive volume of empty voids, while the right side develops a massive "tail" stretching into extreme overdensities.

This asymmetric, late-stage density field is well approximated by a Lognormal distribution. If we define a new variable $A = \ln(1 + \delta)$, the lognormal PDF is given by:

$$P(A) = \frac{1}{\sqrt{2\pi\sigma_A^2}} \exp\left(-\frac{(A - \langle A \rangle)^2}{2\sigma_A^2}\right)$$

By comparing these analytical models to the simulation's dynamic histogram, we can verify that the code adheres to theoretical expectations.

The Matter Power Spectrum

The global density variance (σ^2) tells us how "clumpy" the universe is as a single number, but it cannot tell us the *sizes* of those clumps. A universe filled with tiny, dense galaxies could produce the same variance as a universe filled with a few massive superclusters.

To evaluate the structural health of the simulation across all spatial scales, we must decompose the density field into its constituent frequencies using a Fourier Transform. The resulting metric is the **Matter Power Spectrum**, $P(k)$.

The Physics of the Power Spectrum

Instead of looking at the density contrast $\delta(\mathbf{x})$ in physical space, we transform it into k-space (Fourier space) to get $\delta(\mathbf{k})$. The variable k is the **wavenumber**, which is inversely proportional to the physical size of a structure (λ):

$$k = \frac{2\pi}{\lambda}$$

Low values of k represent massive, large-scale structures (like voids and superclusters), while high values of k represent small-scale structures (like individual galaxy halos). The Power Spectrum $P(k)$ is defined as the variance of the density amplitudes at a specific wavenumber:

$$P(k) = V\langle|\delta(\mathbf{k})|^2\rangle$$

where V is the volume of the simulation box. On a conventional plot, the x-axis represents the wavenumber k (typically in units of h Mpc^{-1}), and the y-axis represents the power $P(k)$ (in units of $(h^{-1}\text{Mpc})^3$). Because both the scales and the amplitudes vary by many orders of magnitude, this is always plotted on a log-log scale.

Computing the Power Spectrum from the Simulation

In our simulation the dark matter exists as discrete particles, while the gas is mapped onto a fixed Eulerian grid. To compute the global density variance in Fourier space, we must project the particles onto the gas grid, combining them into a unified density field. The pipeline consists of four steps:

1. Mass Assignment (Cloud-In-Cell) First, the dark matter mass must be distributed to the stationary grid cells. To do this, we can use the Cloud-In-Cell (CIC) technique, explained in a previous chapter. At the end of this step, we obtain the dark matter density field, $\rho_{dm}(\mathbf{x})$.

2. The Unified Density Contrast With the dark matter mapped to the grid, the total mass density in any given cell is the sum of the dark matter and gas densities: $\rho(\mathbf{x}) = \rho_{dm}(\mathbf{x}) + \rho_{gas}(\mathbf{x})$.

We then calculate the dimensionless density contrast for the unified grid:

$$\delta(\mathbf{x}) = \frac{\rho(\mathbf{x}) - \bar{\rho}}{\bar{\rho}}$$

where $\bar{\rho}$ is the mean density of the entire simulation box.

3. Fourier Transform and Deconvolution We pass the grid of $\delta(\mathbf{x})$ through a Fast Fourier Transform to obtain the complex field $\delta(\mathbf{k})$. Then we need to "un-smear" the raw output by applying the CIC deconvolution.

$$\delta_{\text{true}}(\mathbf{k}) = \frac{\delta_{\text{grid}}(\mathbf{k})}{W(\mathbf{k})}$$

$$W(\mathbf{k}) = \left[\frac{\sin(\frac{k_x \Delta x}{2})}{\frac{k_x \Delta x}{2}}\right]^2 \left[\frac{\sin(\frac{k_y \Delta x}{2})}{\frac{k_y \Delta x}{2}}\right]^2 \left[\frac{\sin(\frac{k_z \Delta x}{2})}{\frac{k_z \Delta x}{2}}\right]^2$$

4. Spherical Averaging (Binning) To generate the final 1D Power Spectrum plot, we must collapse the Fourier grid down to a single line. First, we calculate the squared magnitude $|\delta_{\text{true}}(\mathbf{k})|^2$ for every cell in the Fourier grid.

Because the simulated universe is isotropic (statistically identical in all directions), we do not care about the orientation of a given density wave, only its physical scale. For each cell, we calculate its scalar magnitude $k = \sqrt{k_x^2 + k_y^2 + k_z^2}$, which represents its distance from the origin in Fourier space.

We then define the x-axis of the final plot by dividing this continuous 3D space into discrete "bins" (effectively carving Fourier space into concentric spherical shells of increasing radius). By collecting all the grid cells that fall within the boundaries of a given shell and averaging their squared amplitudes together, we isolate the variance at that specific physical scale. Multiplying this average by the volume of the simulation box yields the final 1D power spectrum value, $P(k)$.

Theoretical Comparison

We must test our simulation against established theoretical models, like accurate Boltzmann solvers.

CAMB (Code for Anisotropies in the Microwave Background) is an industry-standard "Einstein-Boltzmann solver". When provided with a set of cosmological parameters (such as Ω_m, Ω_b, and the Hubble constant), CAMB calculates how the primordial quantum fluctuations from the Big Bang evolved over billions of years.

It does this by simultaneously solving the Einstein field equations (which dictate how gravity expands and bends space) alongside the Boltzmann transport equations (which track how dark matter, baryonic gas, photons, and neutrinos interact, scatter, and push against each other). By tracking these continuous fluids as the universe cools, CAMB calculates the theoretical power spectra for the large-scale distribution of matter.

It is useful to plot the power spectrum from multiple simulation snapshots to verify that the simulation's large-scale structures are evolving in step with the theoretical linear growth rate.

Global Gas Energy Inventory

To ensure the fluid solver is stable and accurately capturing energy transformations, we can plot a global energy inventory over time. This displays the total Kinetic Energy, total Thermal Energy, and the cumulative Radiated Energy of the gas. We can also plot the Fractional Energy Error to prove that the Riemann solver is obeying the First Law of Thermodynamics.

The Physics of Energy Conversion

As the universe evolves, the gravitational potential wells created by dark matter halos begin to pull the surrounding gas inward. As the gas accelerates toward the center of these halos, it gains immense velocity, causing an increase in the global Kinetic Energy of the simulation.

However, unlike dark matter, gas is collisional. When flows of gas from opposite sides of a halo converge in the center, they violently collide. These supersonic collisions create massive shockwaves that convert the ordered kinetic energy of the infalling gas into disordered thermal energy.

Once the gas is shock-heated and densely packed in the center of these halos, it begins to emit photons (primarily via Bremsstrahlung radiation), allowing energy to escape the simulation volume.

The Fractional Energy Error

The Fractional Energy Error is a diagnostic metric that reveals whether the numerical Riemann solver is artificially creating or destroying energy. In a static physics simulation, proving energy conservation is as easy as checking if the final energy matches the initial energy. However, because our cosmological box is expanding and gravity is constantly doing work, the total energy of the gas is *supposed* to change.

To calculate the true numerical error, the simulation must maintain a running ledger. At every single time step, the engine integrates the amount of energy added by gravitational work (W_{grav}), the energy drained by cosmic expansion work (W_{exp}), the thermal energy lost to radiation (E_{rad}), and the thermal energy gained by photoheating (E_{heat}).

$$W_{\text{grav}} = \int \left(\int \rho \mathbf{v} \cdot \mathbf{g} \, dV \right) dt$$

$$W_{\text{exp}} = \int \left(\int 3\frac{\dot{a}}{a} P \, dV \right) dt$$

By evaluating these values using comoving code units (to strip away the diluting effects of the expanding metric), we can isolate the fluid solver's numerical error:

$$\text{Absolute Error} = \Delta E_{\text{tot}} - (W_{\text{grav}} - W_{\text{exp}} - E_{\text{rad}} + E_{\text{heat}})$$

Dividing this absolute error by the initial total energy of the system provides the unitless Fractional Energy Error. If the fluid equations are solved correctly, this ledger balances to zero.

Expected Simulation Behavior

On an energy evolution plot, these processes tell a chronological story:

- **The Adiabatic Expansion Phase:** In the early universe, before structures form, the Thermal Energy curve will initially drop. This is the cosmological PdV work: the expansion of the universe forces the primordial gas to expand and cool down adiabatically (losing thermal energy because its volume is increasing, rather than radiating heat away).
- **The Infall Phase:** As gravity begins to win against the expansion, gas falls into emerging potential wells and the Kinetic Energy curve steadily rises.
- **The Shock Phase:** As the first structures collapse, the Kinetic Energy curve plateaus or begins to drop. Simultaneously, the Thermal Energy curve shoots upward. This exchange between the two curves shows the hydrodynamics engine converting kinetic energy into thermal heat during supersonic shocks.
- **The Photoheating Phase:** At a specific redshift dictated by the selected radiation model, the cosmic ultraviolet background (UVB) activates. The cumulative Photoheating Energy curve, which sits flat at zero in the early universe, will suddenly exhibit a steep, rapid climb. Following this initial jump, the cumulative curve will continue to rise steadily for the remainder of the simulation as the background radiation continuously injects heat to maintain the ionization state of the expanding intergalactic medium.
- **The Cooling Phase:** As the gas reaches extreme temperatures and densities, the cumulative Radiated Energy curve begins to rapidly climb. As billions of Ergs are radiated away, the gas loses its thermal pressure support, allowing it to condense into the cold, dense cores necessary for star formation.
- **The Conservation Baseline:** If the Riemann solver and KDK integrators are healthy, Fractional Energy Error line will remain nearly flat at zero (typically within a tolerance of 10^{-4} or 10^{-3}) across billions of years of evolution.

Maximum Gas Density Evolution

To understand how tightly the baryonic matter is compressing, we can track the densest gas cell in the simulation over time. This metric exposes the ongoing battle between gravitational collapse and thermal pressure.

Tracking Hydrogen Number Density

Instead of plotting the raw bulk mass density of the fluid (ρ), it is usual to track the **Hydrogen number density** (n_H), measured in particles per cubic centimeter (cm^{-3}). There are two main reasons for this:

1. **Composition and Collisions:** The primordial universe is roughly 76% hydrogen by mass. The physical processes that drive galaxy evolution—such as radiative cooling and star formation—depend on how frequently individual particles collide, which is dictated by number density, not just overall mass.
2. **Observational Parity:** Astronomers cannot easily "weigh" a distant gas cloud, but they can count hydrogen atoms by measuring the light absorbed or emitted by them. Plotting n_H allows direct comparisons between our simulated universe and telescope data.

To calculate this metric from the simulation data, we must convert our internal code variables into physical units. We first divide the comoving mass density by the expansion factor cubed ($\mathbf{a^3}$) to find the true physical density. We then multiply by the primordial hydrogen mass fraction ($X_H = 0.76$) to isolate the hydrogen mass, and finally divide by the mass of a single proton to count the exact number of atoms per unit volume.

Expected Simulation Behavior

As gas falls into a dark matter halo, gravity forces it into an increasingly smaller volume. However, as the gas shock-heats, its thermal pressure increases, pushing back against the gravitational collapse.

- **The Expansion-Dominated Phase:** In the very early universe, the maximum physical density curve will drop. During this linear regime, the expansion of the universe outpaces the slow gravitational assembly of the initial dark matter clumps, causing the gas to dilute.
- **Turnaround and Collapse:** As dark matter halos grow massive enough to overcome the background Hubble expansion—a threshold known as "turnaround"—the gas is aggressively pulled into these deepening potential wells. At this point, the density curve reverses course and begins to rise steeply, mirroring the non-linear structure growth.
- **Late Epochs and Core Collapse:** In a healthy, fully featured simulation (with active radiative cooling and high spatial resolution), this curve will continue to climb dramatically. As the gas radiates away its shock-heated thermal energy, it loses outward pressure support and collapses deeply into the halo cores, achieving the extreme densities necessary to trigger star formation. *Conversely*, in a purely adiabatic simulation (without cooling) or one limited by coarse grid resolution, this curve will prematurely flatten out and hit an artificial ceiling. In those restricted scenarios, the unabated thermal pressure prevents the gas from compressing any further than a few grid cells across.

Maximum Temperature Evolution

Tracking the maximum temperature—the single hottest cell in the universe at any given time—helps verifying the simulation's shock-capturing capabilities.

The Physics of Shock Heating

In the void regions of the universe, gas naturally cools due to adiabatic expansion ($T \propto a^{-2}$). However, inside halos, the gravitational potential is so deep that infalling gas reaches velocities far exceeding the local speed of sound. When this gas collides, the resulting shockwaves heat the medium to the **virial temperature** of the halo (the temperature a cloud of gas naturally reaches when it falls into a gravitational well and its inward fall is violently halted by collisions, transforming all that gravitational energy into heat), which scales with the halo's mass. Massive galaxy clusters can possess deep enough gravity wells to heat gas to temperatures exceeding 10^7 K.

Expected Simulation Behavior

- **The First Light (First Shocks):** At the beginning of the simulation, the maximum temperature remains low. Suddenly, when the very first non-linear structures collapse, the plot will show a violent, near-vertical spike. The maximum temperature jumps from a few tens of Kelvin to hundreds of thousands or millions of Kelvin in a fraction of a Gigayear.
- **Virial Equilibrium:** After the initial spike, the curve levels off, forming a stable, slightly rising plateau. This plateau corresponds to the virial temperature of the most massive dark matter halo that has formed within the simulation box.

The Temperature-Density Phase Diagram

The **Temperature-Density Phase Diagram** maps the thermodynamic state of every single gas cell in the universe. Plotted as a 2D histogram (often with logarithmic scales), the x-axis represents the gas overdensity ($\rho/\bar{\rho}$), and the y-axis represents the temperature. The position of gas in this phase space reveals which physical processes are dominating its behavior.

The Physics of the Phase Space

Gas in the universe naturally segregates into distinct thermodynamic regimes based on its environment:

- **The Diffuse Intergalactic Medium (IGM):** In the extremely low-density voids, gas simply expands with the universe. Because it is doing work as it expands, it cools adiabatically. In the phase diagram, this gas forms a tight, diagonal line at low densities and low temperatures, defined by the relationship $T \propto \rho^{\gamma-1}$ (the **Adiabatic Track**).
- **The Warm-Hot Intergalactic Medium (WHIM):** As gas falls into filaments and dark matter halos, it compresses and shocks. This shock-heated gas breaks away from the adiabatic track, scattering upward into a broad cloud of high-temperature, moderate-density material.

- **The Condensed Cores:** If the simulation includes micro-physics like Bremsstrahlung (free-free) or line cooling, gas that reaches high densities can radiate its thermal energy away as photons. Because the cooling rate scales with the square of the density ($\Lambda \propto \rho^2$), cooling becomes fiercely efficient in the deepest parts of the dark matter halos.

Expected Simulation Behavior

- **In an Adiabatic Simulation:** Without radiative cooling, gas that gets shocked into the high-temperature regime stays hot. As it compresses into halos, it moves to the right on the phase diagram (higher density) but remains in a thick, hot plateau above the adiabatic track. Its thermal pressure eventually physically halts further compression.
- **In a Radiative Cooling Simulation:** When cooling physics is enabled (and spatial resolution is high enough to allow dense cores to form), a dramatic structural shift occurs. At high overdensities, the cooling time of the gas drops below the age of the universe. The hot gas loses its pressure support and plummets downward on the graph, forming a vertical "cooling waterfall." This gas pools at the bottom right of the phase diagram, hitting the artificial temperature floor (often set around **10,000 K** for primordial cooling, or lower if molecular cooling is simulated).

Cold Dense Gas Fraction (The Star Formation Precursor)

Cosmological simulations are fundamentally attempting to explain galaxy formation. However, individual stars are too small to simulate in a box that is millions of parsecs wide. Instead, astrophysicists use sub-grid models to spawn "star particles" out of gas that is physically ready to condense. The **Cold Dense Gas Fraction** tracks the total mass of this eligible raw material over time.

The Physics of Condensation

For a cloud of gas to collapse and form stars, it must overcome its own internal thermal pressure. This requires two specific conditions to be met simultaneously:

1. **High Density:** The gas must be deep inside a gravitational potential well (typically an overdensity $\delta > 100$), ensuring gravity is strong enough to pull it together.
2. **Low Temperature:** The gas must have successfully radiated away its shock-heated thermal energy (typically dropping below **10,000 K**), sapping the outward pressure that would otherwise resist collapse.

Expected Simulation Behavior

Tracking the mass fraction of gas that meets these two exact criteria serves as the ultimate benchmark for the simulation's thermodynamic pipeline.

- **Pure Hydrodynamics:** In a simulation without radiative cooling, or one crippled by low resolution, this curve remains entirely flat at zero. The gas never simultaneously achieves high density and low temperature.
- **Complete Physics:** In a high-resolution simulation with active cooling, this curve will remain at zero during the early epochs. However, shortly after the first halos collapse and the cooling "waterfall" triggers in the phase diagram, this line will rapidly spike upward. This rising curve represents the exact mass of gas that has decoupled from the hot halo and is successfully condensing into proto-galactic disks, acting as the immediate fuel source for the universe's first stars.

The Invariant Layzer-Irvine Energy

In an expanding Friedmann-Robertson-Walker (FRW) cosmology, the physical energy of a comoving volume is not conserved. As the fabric of space expands, the peculiar velocities of particles naturally decay due to "Hubble drag," cooling the kinetic energy of the system. Simultaneously, the gravitational potential energy evolves as the background density dilutes. The expanding universe effectively does thermodynamic work on the matter within it.

To validate a cosmological N-body code, we must account for the work done by expansion and evaluate a specific cosmological invariant: **The Layzer-Irvine Equation**.

Named after David Layzer and William Irvine, who formalized the cosmic energy equation in the 1960s, this equation governs the thermodynamic evolution of an expanding universe. It bridges the gap between the discrete Newtonian mechanics of our simulated particles and the continuous general relativistic expansion of the background space.

For a purely collisionless dark matter system, the instantaneous physical energy is simply $E(t) = K(t) + W(t)$. According to the Layzer-Irvine equation, the rate at which this energy decays is proportional to the Hubble parameter $H(t)$:

$$\frac{dE}{dt} + H(t)\left[2K(t) + W(t)\right] = 0$$

If the simulation includes hydrodynamics (gas), the expansion also performs PdV work on the thermal internal energy $U(t)$, and the equation expands to include the gas adiabatic index γ:

$$\frac{dE}{dt} + H(t)\left[2K(t) + W(t) + 3(\gamma - 1)U(t)\right] = 0$$

By rearranging and integrating this equation from the start of the simulation (t_0) to the current time (t), we can define a quantity that *must* remain perfectly constant—a **Layzer-Irvine Invariant** (I):

$$I = E(t) - E(t_0) + \int_{t_0}^{t} H(t')\left[2K(t') + W(t')\right] dt' = 0$$

The integral term represents the cumulative "Expansion Work" (W_{exp}) exchanged between the clustering matter and the expanding spacetime. In a mathematically perfect simulation, the instantaneous energy drift $E(t) - E(t_0)$ will be exactly balanced by W_{exp}.

Numerical Integration: The Trapezoidal Rule

To prove that a simulation conserves this invariant, we must evaluate the expansion work integral. However, simulations do not output continuous data; they write data in discrete snapshots at specific times $t_1, t_2, \ldots, t_n$.

Because we only know the exact physical state of the universe at these discrete snapshot intervals, we must solve the integral numerically. We can define the instantaneous "expansion power" at any snapshot i as:

$$P_i = H_i(2K_i + W_i)$$

To integrate this power over time, we use the **Trapezoidal Rule**. The Trapezoidal Rule approximates the area under the curve of $P(t)$ by treating the space between two consecutive snapshots as a trapezoid. The work done between snapshot $i-1$ and snapshot i is calculated using the average power over that interval, multiplied by the time step Δt:

$$\Delta W_{exp,i} = \frac{1}{2}(P_i + P_{i-1}) \times (t_i - t_{i-1})$$

The total cumulative expansion work up to the current snapshot n is simply the sum of these discrete trapezoids:

$$W_{exp}(t_n) \approx \sum_{i=1}^{n} \frac{1}{2}(P_i + P_{i-1})(t_i - t_{i-1})$$

Interpreting the Fractional Error

With the total energy and the cumulative expansion work calculated, we can formulate a dimensionless fractional error to evaluate the health of the simulation:

$$\text{Fractional Error} = \frac{E(t_n) - E(t_0) + W_{exp}(t_n)}{|E(t_0)|}$$

Plotting this fractional error across the lifespan of a simulation acts as the ultimate diagnostic of numerical integrity. Because N-body codes generally rely on symplectic integrators (like the Leapfrog method), the fractional error should not exhibit long-term secular growth. Instead, it should hover near zero.

If the error curve remains flat and bounded (e.g., within 0.1%), it proves that the Hamiltonian of the system is preserved and the equations of motion are accurately resolving the deepest density collapses. Conversely, a rapidly growing fractional error immediately flags numerical artifacts, such as inadequate force softening, timestep violations, or non-physical force discontinuities at periodic boundaries.

Appendix

Cosmological Unit Conversion Cheatsheet

Physical Concept	Dimensions	Factor from comoving to physical
Mass	M	No change
Position / Length	L	a
Volume	L^3	a^3
Velocity	L/T	a
Acceleration	L/T^2	a
Force	$M \cdot L/T^2$	a
Density	M/L^3	a^{-3}
Pressure	$M/(L \cdot T^2)$	a^{-1}
Specific Internal Energy *(MFM)*	L^2/T^2	a^2
Specific Kinetic Energy *(MFM)*	L^2/T^2	a^2
Volumetric Energy Density *(Eulerian)*	$M/(L \cdot T^2)$	a^{-1}
Volumetric Momentum Density *(Eulerian)*	$M/(L^2 \cdot T)$	a^{-2}
Entropic Function ($S = P/\rho^\gamma$)	$M^{1-\gamma} \cdot L^{3\gamma-1}/T^2$	$a^{3\gamma-1}$
Time	T	No change (tracked physically)
Temperature	Θ	No change (converted internally)